WaitLess: A Digital Queue Management System for Zimbabwean Public Hospitals

Case Study: Chinhoyi Provincial Hospital

Submitted by

Team member	Email address
Shadreck Mapuranga	mapurangashadreck03@gmail.com
Norris Marava	norrismarava44@gmail.com
Kelvin Ncube	kevincube19@gmail.com

Supervisor: H. Musabayeka
Institution: Chinhoyi University of Technology
School: School of Engineering Science and Technology
Department: Department of ICT and Electronics
Place: Chinhoyi, Zimbabwe

Date: August 2026

A project submitted in partial fulfilment of the requirements for the Bachelor of Science Honours Degree in Information Technology.

Acknowledgements

We gratefully acknowledge our supervisor, H. Musabayeka, for the direction and critical questions that helped us turn an ambitious idea into a more disciplined project. We also thank the School of Engineering Science and Technology and the Department of ICT and Electronics for the academic foundation and facilities that supported the work.

The project was shaped by the everyday reality of hospital service delivery. We therefore acknowledge the healthcare workers whose routines, constraints, and commitment informed the way we thought about patient flow. Finally, we thank our families, classmates, and friends for their patience, encouragement, and practical support throughout the long cycles of design, coding, testing, correction, and documentation.

Dedication

This work is dedicated to patients who spend precious time waiting for care, and to the healthcare workers who keep services moving despite limited resources. It is also dedicated to our families, whose support made the project possible.

Declaration and Release Form

We declare that this project documentation is our original work except where sources have been acknowledged through citation and reference. The software artefact, analysis, diagrams, testing evidence, and written discussion were prepared for the WaitLess final-year project. We understand the University's requirements concerning academic integrity and accept responsibility for the accuracy of the submission.

We grant Chinhoyi University of Technology permission to make this project available for academic consultation and examination, subject to applicable privacy, intellectual-property, and institutional policies.

Name	Signature	Date
Shadreck Mapuranga	_____	_____
Norris Marava	_____	_____
Kelvin Ncube	_____	_____

Approval Form

This project documentation has been submitted for examination with the approval of the project supervisor.

Role	Name	Signature	Date
Project Supervisor	H. Musabayeka	_____	_____
Department/Examiner	_____	_____	_____

Originality and Antiplagiarism Statement

The authors confirm that all ideas, findings, diagrams, and words drawn from other sources have been acknowledged. An institution-generated similarity or antiplagiarism report should be attached in accordance with Chinhoyi University of Technology policy before the final bound submission. No similarity score has been invented or inserted into this document.

Abstract

Public hospitals in Zimbabwe often coordinate registration, triage, and queues through paper records and verbal calls, making patient flow difficult to monitor and missed turns difficult to prevent. This project developed WaitLess, a web-based digital queue management system using Chinhoyi Provincial Hospital as a case study. A pragmatic methodology combined workflow observation, document review, literature review, design science research, iterative software development, and prototype evaluation. The resulting artefact integrated patient registration, colour-coded triage, priority-aware departmental queues, real-time staff dashboards, patient tracking, anonymous queue boards, notification logging and retry, reporting, appointments, and selected pharmacy, laboratory, and OI workflows. Verification produced 12 passing automated backend tests, 39 controlled functional walkthroughs, and a successful frontend production build. The evaluation showed that the prototype could represent an end-to-end patient journey and provide shared operational visibility. It did not yet prove a reduction in hospital waiting time or improved clinical outcomes because a live authorised pilot had not been completed. Production deployment still required persistent sessions, restricted CORS, rate limiting, security headers, provider verification, backup and restore testing, load testing, and user acceptance. The project concluded that WaitLess provided a credible pilot-ready foundation for context-appropriate digital queue management, subject to those controls and evaluations.

DOCUMENT NAVIGATION

Table of Contents

Chapter page numbers begin with Chapter 1. Front matter uses Roman numerals in the consolidated PDF.

Chapter 1: Introduction 1

1.0 Introduction 1

1.1 Background of the study 1

1.2 Problem Statement 2

1.3 Project Aims and Objectives 4

1.4 Significance of the Study 5

1.5 Limitations and Delimitations of the Study 5

1.6 Definition of terms 6

1.7 Conclusion 8

Chapter 2: Literature Review 9

2.0 Introduction 9

2.1 Queue Management Theory in Healthcare 9

2.2 Digital Queue Management Systems in Hospitals 10

2.3 Patient Notification Systems in Healthcare 11

2.4 Digital Health Adoption in Zimbabwe and Sub-Saharan Africa 11

2.5 Design Science Research as a Methodological Foundation 12

2.6 Technology Frameworks for Real-Time Web Applications 12

2.7 Review of Commercial Queue Management Systems 13

2.8 Identified Gaps in the Literature 14

2.9 Conclusion 14

Chapter 3: Methodology 16

3.0 Introduction 16

3.1 Research Philosophy and Approach 16

3.2 Research Design 16

3.3 Design Science Research Method 17

3.4 Case Study and Unit of Analysis 19

3.5 Data Sources and Collection Methods 20

3.6 Requirements Analysis Procedure 21

3.7 System Development Method 21

3.8 Technology Selection Procedure 22

3.9 Evaluation Strategy 22

3.10 Data Analysis 23

3.11 Validity, Reliability, and Trustworthiness 23

3.12 Ethical and Privacy Considerations 24

3.13 Implementation Plan and Milestones 24

3.14 Methodological Limitations 25

3.15 Chapter Summary 25

Chapter 4: System Design and Implementation 26
4.0 Introduction 26
4.1 Requirements Analysis 26
4.2 Use Case Diagrams 31
4.3 Use Case Descriptions 34
4.4 System Architecture 41
4.5 Data Design 44
4.6 Process Design 52
4.7 User Interface Design 59
4.8 Security Design 64
4.9 Implementation Plan and Milestones 66
4.10 Technology Stack 66
4.11 System Structure 69
4.12 Backend Implementation 73
4.13 Frontend Implementation 79
4.14 Database Implementation 84
4.15 Integration Implementation 86
4.16 Deployment Configuration 87
4.17 Summary 90
Chapter 5: Presentation of Results 91
5.1 Introduction 91
5.2 Testing Approach 91
5.3 Test Plan and Test Cases 92
5.4 Test Results 96
5.5 Usability Evaluation 99
5.6 Performance Analysis 100
5.7 Security Evaluation 102
5.8 Test Coverage Analysis 104
5.9 Interpretation of Results Against the Problem 105
5.10 Summary 105
Chapter 6: Summary, Maintenance and Recommendations 107
6.1 Introduction 107
6.2 Project Summary 107
6.3 Achievement of Objectives 108
6.4 Limitations 110
6.5 Lessons Learned 111
6.6 System Maintenance and Recommendations 112
6.7 Future Research Directions 113
6.8 Project Significance 114
6.9 Conclusion 114
References 116
Technical Documentation 117

Appendices 118

Appendix A: Submission Deliverables 118

Appendix B: Verification Command Summary 118

Appendix C: Pre-Deployment Acceptance Checklist 118

Appendix D: Patient and User Manual 119

Appendix E: Staff User Manual 119

VISUAL INDEX

List of Figures

Figure 3.1: Research Design 17
Figure 3.2: Demonstration and Evaluation 18
Figure 3.3: Requirements Analysis Procedure 21
Figure 4.1: System-Wide Use Case Diagram 31
Figure 4.2: Patient Registration Use Case Diagram 32
Figure 4.3: Triage Use Case Diagram 33
Figure 4.4: Queue Management Use Case Diagram 33
Figure 4.5: Notification Use Case Diagram 34
Figure 4.6: High-Level Architecture 41
Figure 4.7: Deployment Architecture 43
Figure 4.8: Entity-Relationship Diagram 44
Figure 4.9: DFD Level 0 — Context Diagram 50
Figure 4.10: DFD Level 1 — Major Processes 51
Figure 4.11: DFD Level 2 — Patient Registration Detail 51
Figure 4.12: Sequence Diagram: Patient Registration 52
Figure 4.13: Sequence Diagram: Triage Assessment 53
Figure 4.14: Sequence Diagram: Call Next Patient 54
Figure 4.15: Sequence Diagram: Notification Dispatch with Retry 55
Figure 4.16: Activity Diagram: Patient Journey 56
Figure 4.17: Activity Diagram: Triage Workflow 57
Figure 4.18: Activity Diagram: Notification Retry Lifecycle 58
Figure 4.19: Navigation Structure 61
Figure 4.20: Backend Module Map 72
Figure 4.21: Frontend Component Hierarchy 72

List of Tables

Table 3.1: Research Design 16
Table 3.2: Existing-System and Code Evidence 20
Table 3.3: Evaluation Strategy 22
Table 3.4: Implementation Plan and Milestones 24
Table 4.1: Module 1: Patient Registration 26
Table 4.2: Module 2: Patient Self-Service 26
Table 4.3: Module 3: Triage 27
Table 4.4: Module 4: Queue Management 27
Table 4.5: Module 5: Department Routing 27
Table 4.6: Module 6: Staff Dashboard 28
Table 4.7: Module 7: Notifications 28
Table 4.8: Module 8: Queue Display Screens 28
Table 4.9: Module 9: Reporting 29
Table 4.10: Module 10: Analytics 29
Table 4.11: Module 11: Security and Access Control 29
Table 4.12: Module 12: Privacy and Confidentiality 29
Table 4.13: Module 13: Offline and Connectivity 29
Table 4.14: Non-Functional Requirements 30
Table 4.15: User Requirements by Stakeholder Role 31
Table 4.16: Use Case UC-01: Register New Patient 35
Table 4.17: Use Case UC-02: Triage Assessment 35
Table 4.18: Use Case UC-03: Call Next Patient 36
Table 4.19: Use Case UC-04: Missed Turn and Recall 36
Table 4.20: Use Case UC-05: Queue Transfer 37
Table 4.21: Use Case UC-06: Patient Self-Tracking 38
Table 4.22: Use Case UC-07: Kiosk Self-Check-in 38
Table 4.23: Use Case UC-08: Send WhatsApp Notification 39
Table 4.24: Use Case UC-09: View Reports 39
Table 4.25: Use Case UC-10: View Audit Trail 40
Table 4.26: Layered Architecture Description 42
Table 4.27: Technology Architecture 43
Table 4.28: Table: departments 45
Table 4.29: Table: patientprofiles 46
Table 4.30: Table: tickets 46
Table 4.31: Table: notifications 47
Table 4.32: Table: notificationattempts 48
Table 4.33: Table: staffusers 48
Table 4.34: Table: patientusers 49
Table 4.35: Table: auditevents 49

Table 4.36: Table: systemcounters 50
Table 4.37: Colour Palette 60
Table 4.38: Typography 60
Table 4.39: Role-Based Access Control Matrix 65
Table 4.40: Privacy Measures 66
Table 4.41: Frontend Technologies 67
Table 4.42: Backend Technologies 67
Table 4.43: Development Tools 67
Table 4.44: Reporting Endpoints 79
Table 4.45: State Management 80
Table 4.46: UI Component Library 83
Table 4.47: Indexing Strategy 85
Table 4.48: Environment Variables 88
Table 5.1: Testing Strategy Overview 91
Table 5.2: Test Environment 91
Table 5.3: Automated Test Framework 92
Table 5.4: Test Suite 1: Audit Service Tests (audit-service.test.js) 92
Table 5.5: Test Suite 2: Staff Authentication Security Tests (staff-auth-security.test.js) 93
Table 5.6: Test Suite 3: Returning Patient Tests (returning-patient.test.js) 93
Table 5.7: Module: Authentication 93
Table 5.8: Module: Patient Registration 94
Table 5.9: Module: Triage Assessment 94
Table 5.10: Module: Queue Management 95
Table 5.11: Module: Notifications 95
Table 5.12: Module: Queue Display 95
Table 5.13: Module: Patient Tracking 96
Table 5.14: Module: Reports 96
Table 5.15: Module: Security and Audit 96
Table 5.16: Automated Test Results 97
Table 5.17: Manual Test Results 97
Table 5.18: Build Verification 98
Table 5.19: API Endpoint Verification 98
Table 5.20: System Usability Scale (SUS) 99
Table 5.21: User Feedback Categories 100
Table 5.22: Task-Based Evaluation 100
Table 5.23: Frontend Bundle Analysis 100
Table 5.24: API Response Times 101
Table 5.25: Real-Time Update Latency 101
Table 5.26: Database Performance 101
Table 5.27: Low-Bandwidth and Offline Considerations 102
Table 5.28: OWASP Top 10 Compliance Checklist 102
Table 5.29: Security Measures Verified 103
Table 5.30: Remaining Security Recommendations 103

Table 5.31: Requirements Traceability 104
Table 5.32: Test Type Coverage 104
Table 5.33: Interpretation of Results Against the Problem 105
Table 6.1: Technical Architecture Summary 108
Table 6.2: Original Objectives Assessment 108
Table 6.3: Requirements Coverage at Documentation Freeze 109
Table 6.4: Technical Limitations 110
Table 6.5: Functional Limitations 110
Table 6.6: Operational Limitations 111
Table 6.7: Immediate Actions (Pre-Production) 112
Table 6.8: Short-Term Pilot Work (Next 3 Months) 112
Table 6.9: Medium-Term Enhancements (3–6 Months) 113
Table 6.10: Long-Term Vision (6–12 Months) 113
Table A.1: Appendix A: Submission Deliverables 118
Table A.2: Appendix B: Verification Command Summary 118

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ART	Antiretroviral Therapy
CORS	Cross-Origin Resource Sharing
CUT	Chinhoyi University of Technology
DFD	Data Flow Diagram
DQMS	Digital Queue Management System
DSR	Design Science Research
EHR	Electronic Health Record
ERD	Entity-Relationship Diagram
HIS	Hospital Information System
ICT	Information and Communication Technology
LAN	Local Area Network
OI	Opportunistic Infection
OPD	Outpatient Department
QR	Quick Response
RBAC	Role-Based Access Control
SSE	Server-Sent Events
SUS	System Usability Scale
UAT	User Acceptance Testing

Chapter 1: Introduction

1.0 Introduction

Healthcare delivery in public hospitals is strongly influenced by the way patients are registered, prioritised, and moved between services. At the time of the study, many Zimbabwean public health facilities still relied on manual, paper-based processes for patient registration, triage, and queue management. These processes contributed to long waits, crowded waiting areas, weak operational visibility, and an increased risk that urgent cases could be delayed. This context provided the practical motivation for developing WaitLess.

This chapter introduced WaitLess, a digital queue management system designed for Zimbabwean public hospitals. The project was informed by a case study at Chinhoyi Provincial Hospital, where patient-flow processes, triage protocols, departmental workflows, and patient entry routes were documented and analysed. The work made the problem concrete: staff were not simply managing a line of people, but a set of clinically different journeys that crossed several departments and changed as new information became available.

A key feature of WaitLess was its integrated patient-notification workflow. The prototype supported WhatsApp delivery states and browser-based notification interactions so that a patient could receive queue updates without remaining beside a display screen. This directly addressed a gap observed during the case study: once a patient moved away from a waiting area, staff had no reliable way to communicate an approaching turn. The notification design therefore focused on timely updates, consent, retry visibility, and low operating cost. Actual production charges remained dependent on the messaging provider's policy and hospital configuration.

This chapter is organised as follows: Section 1.1 presents the background to the study, outlining the broader context of queue management in healthcare and the specific challenges observed in the Zimbabwean setting. Section 1.2 articulates the problem statement, contrasting the ideal situation with the reality observed at Chinhoyi Provincial Hospital and the resulting consequences. Section 1.3 states the aim and objectives of the project. Section 1.4 discusses the significance of the study, while Section 1.5 outlines its limitations and delimitations. Section 1.6 defines key terms used throughout the chapter, and Section 1.7 concludes with a summary and a preview of the next chapter.

1.1 Background of the study

Queue management in healthcare is defined as the systematic process of controlling the flow of patients from entry to exit within a healthcare facility (Preater, 2002). Effective queue management is critical to the functioning of any hospital, as it directly influences patient satisfaction, clinical outcomes, and staff productivity. In high-income countries, digital queue management systems (DQMS) are widely deployed, incorporating features such as electronic patient registration, automated triage prioritisation, real-time wait-time displays, and digital notifications to patients (Cayirli & Veral, 2003).

In Sub-Saharan Africa, including Zimbabwe, the healthcare system faces unique challenges. According to the World Health Organization (2019), the African region bears over 24% of the global disease burden while having access to only 3% of health workers and less than 1% of world health expenditure. The resulting strain manifests in overcrowded outpatient departments, inefficient manual patient registration, and chaotic queuing that can endanger critically ill patients who do not receive timely care.

Zimbabwe's public hospitals have historically operated with paper-based patient records and manual queue management. Evidence from other public-hospital settings showed that digitised records and queue analysis could reveal bottlenecks that were difficult to see in a paper workflow (Cho et al., 2017). Bidari et al. (2021) further showed that a queue-management intervention could reduce both actual and perceived waiting time while improving patient satisfaction.

The case study conducted at Chinhoyi Provincial Hospital corroborated these findings. The hospital operates a colour-coded triage system categorised into four priority levels: Priority I (Red) for critically ill patients such as those with respiratory arrest or severe burns, directed to the Casualty Resuscitation Room; Priority II (Yellow) for seriously ill but non-life-threatening cases directed to the Casualty Observation Ward; Priority III (Green) for ambulant patients with minor injuries directed to the Casualty Waiting Area; and Priority IV (Black) for deceased patients held until transfer to the mortuary. Despite this structured triage framework, the absence of a digital system to support these processes means that patient sorting relies entirely on manual observation and handwritten records.

The hospital's OPD Patient Flow Map revealed a structured but entirely paper-based pathway: patients began at registration (5 minutes), proceeded to nurse observation (5 minutes) and nurse consultation (10 minutes), waited at the doctor's queue line (10 minutes), then underwent doctor's consultation (10 minutes) before branching to laboratory (20 minutes), radiology (30 minutes), or direct admission. Results were then reviewed by a doctor (5 minutes), followed by pharmacy (10 minutes), the treatment room (10 minutes), and departure. The total expected journey time for a straightforward OPD visit was thus approximately 80 minutes under ideal conditions. In practice, the absence of digital tracking means bottlenecks are invisible to management and cannot be proactively addressed.

The hospital also operates an Opportunistic Infection (OI) Clinic, which serves HIV/AIDS patients requiring TB services, antiretroviral therapy (ART) resupply, viral load testing, cervical cancer screening, and counselling. Managing these multiple parallel service streams without a digital system creates confusion, missed appointments, and inequitable service distribution.

Digital workflow interventions have also produced measurable benefits in Sub-Saharan African hospitals. Mwanswila, Mollel and Mushi (2024) reported a 45% reduction in median outpatient waiting time after a programme of technical and process changes at Kilimanjaro Christian Medical Centre in Tanzania. At the same time, Furusa and Coleman (2018) found that e-health implementation in Zimbabwean public hospitals was shaped by infrastructure, staff ICT skills, technical support, security concerns, funding, and policy. WaitLess was therefore designed around local operational realities rather than assuming constant internet access, unlimited hardware, or specialist technical staff.

1.2 Problem Statement

1.2.1 Ideal Situation

In an ideal healthcare environment, every patient who enters a public hospital should be swiftly registered, electronically triaged according to the severity of their condition, and directed to the appropriate service area with minimal waiting time. Real-time digital dashboards should enable clinical staff and hospital administrators to monitor patient flow across all departments, identify bottlenecks instantly, and dynamically reallocate resources to prevent congestion. Patients should be notified when their turn is approaching via WhatsApp or web notification, even if they have temporarily left the hospital premises. Critically ill patients should be automatically flagged and fast-tracked, and all patient journey data should be recorded digitally for reporting, quality improvement, and health system planning.

1.2.2 Reality

The current reality at Chinhoyi Provincial Hospital, as evidenced by the case study, is significantly different. Patient registration, triage classification, and queue management are entirely manual and paper-based. There is no system to digitally track a patient's journey from the point of entry through triage, consultation, investigations, pharmacy, and discharge. Triage officers rely on visual assessment and handwritten records to assign patients to the colour-coded priority categories (Red, Yellow, Green, Black), with no digital support to flag deteriorating patients or compute waiting times.

The OPD flow map, while conceptually well-designed, is not supported by any digital infrastructure. Queue bottlenecks, such as delays at the laboratory (20 minutes) or radiology (30 minutes), cannot be monitored in real time. The OI Clinic manages multiple complex patient streams through paper-based files with no automated scheduling or queue prioritisation. Patient sources include both unreferred walk-in cases and referred cases from clinics, peripheral hospitals, and private practitioners, adding further complexity to patient flow management. Critically, patients who leave the hospital premises while waiting have no way of being notified when it is their turn, leading to no-shows and further disruption of the queue.

1.2.3 Consequences

The consequences of the current manual system include:

- Prolonged patient waiting times that regularly exceed 3-5 hours for a standard OPD visit, far beyond the expected 80 minutes under ideal conditions.
- Unmanaged congestion in OPD and casualty areas, with risk of adverse clinical outcomes when critically ill patients are not prioritised.
- Staff burnout due to manual administrative burden, including physical movement between departments to coordinate patient referrals.
- Inability to generate real-time or historical performance reports, preventing evidence-based staffing and resource decisions.
- Loss of institutional data due to paper records being misfiled or damaged.

- No-shows caused by patients who leave the premises without knowing when their turn will come, as there is no notification mechanism.
- Patients from referred facilities who arrive without prior digital notification experience additional, avoidable delays. 1.2.4 Proposed Solution This project developed of WaitLess, a web-based digital queue management system tailored for Zimbabwean public hospitals. The system digitised patient registration and triage, automate patient-to-department routing based on triage priority, provide real-time queue dashboards for clinical and administrative staff, generate waiting time estimates, and notify patients when their turn is approaching via WhatsApp or web push notification. It also produced automated reports for hospital management. The system was designed to function on low-bandwidth networks using locally hosted infrastructure, making it suitable for resource-constrained hospital settings. Problem Statement: Chinhoyi Provincial Hospital, like most public hospitals in Zimbabwe, manages patient registration, triage, and queue flow using entirely manual, paper-based processes. No digital system exists to track patient movement from entry through triage, consultation, investigations, and discharge, nor to monitor waiting times, alert staff to congestion, or notify patients of their queue status. This results in inequitable service delivery, preventable delays in care, and loss of operational data. A web-based digital queue management system tailored to Zimbabwe's triage protocols and hospital workflows would enable real-time patient tracking, automated priority routing, patient notifications via WhatsApp and web push, and data-driven hospital management.

1.2.4 Proposed Solution

The project developed WaitLess, a web-based digital queue management system tailored to Zimbabwean public hospitals. The system digitised patient registration and triage, routed patients into departmental queues, preserved clinical-priority ordering, displayed live queue state, supported patient notifications, and produced operational reports. It was designed for browser-based use on ordinary hospital computers and for local-area-network deployment so that core queue work could continue even when external connectivity was unreliable.

The problem addressed by the project can therefore be stated as follows: Chinhoyi Provincial Hospital managed registration, triage, and queue flow through disconnected paper processes that did not provide a live view of the patient's journey or a dependable notification channel. This reduced staff visibility, made delays difficult to investigate, and created avoidable missed turns. WaitLess addressed this problem through a single, role-aware workflow for patient registration, triage, departmental routing, live monitoring, and communication.

1.3 Project Aims and Objectives

1.3.1 Aim

The aim of this project was to design, develop, and evaluate a web-based digital queue management system for Zimbabwean public hospitals, using Chinhoyi Provincial Hospital as the primary case study.

1.3.2 Objectives

The following objectives were defined to guide the achievement of the project aim:

1. To analyse current patient flow, triage, and queue management processes at Chinhoyi Provincial Hospital and identify key system requirements, including the need for a patient notification mechanism, by May 2026.
2. To design and develop a web-based digital queue management system that supports colour-coded triage prioritisation, multi-department patient routing, real-time queue monitoring, and a WhatsApp and web push patient notification engine, by August 2026.
3. To evaluate the system's usability, efficiency, and suitability for deployment in low-bandwidth, resource-limited Zimbabwean hospital settings, including patient satisfaction with the notification system, by October 2026.

1.4 Significance of the Study

This study is significant in terms of societal impact, healthcare service improvement, academic contribution, and alignment with national digital health objectives.

- Societal Impact: Long patient waiting times in Zimbabwean public hospitals remain a major healthcare challenge. By digitising and streamlining queue management processes at Chinhoyi Provincial Hospital, the proposed system has the potential to improve patient experience and reduce congestion in waiting areas. The integration of patient notifications further allows patients to temporarily leave waiting areas without losing their queue position.
- Healthcare Worker Efficiency: The automation of queue coordination tasks reduces the administrative burden currently placed on nurses and hospital staff. This allows healthcare personnel to focus more on clinical responsibilities, thereby improving operational efficiency and potentially enhancing job satisfaction.
- Evidence-Based Management: The system generates both real-time and historical data relating to patient flow, waiting times, and departmental throughput. Such information can support hospital management in making informed decisions regarding staffing, scheduling, and resource allocation.
- Reduction of Missed Queue Turns: The integration of WhatsApp and web push notifications addresses the common problem of patients missing their queue turn after temporarily leaving the hospital premises. By notifying patients when their turn is approaching, the system improves queue continuity and reduces service disruptions.
- Academic Contribution: The study contributes to the growing body of knowledge in digital healthcare systems within sub-Saharan Africa, particularly in Zimbabwe. The project may serve as a reference model for similar queue management implementations in other public healthcare institutions.
- Alignment with National Health Objectives: The project aligns with Zimbabwe's National Health Strategy 2021–2025 and the Digital Economy Blueprint, both of which encourage the adoption of digital technologies to improve healthcare service delivery and patient experience in public institutions.

1.5 Limitations and Delimitations of the Study

1.5.1 Limitations of the Study

The following limitations are acknowledged in this study:

- The case study was conducted at a single hospital, Chinhoyi Provincial Hospital, which may not fully represent the diversity of patient flow processes and infrastructure conditions across all Zimbabwean public hospitals.
- Intermittent internet connectivity at Chinhoyi Provincial Hospital may affect the WhatsApp notification and web push features of the system. However, the core queue management system is designed to function offline within the hospital local area network, with notifications queued for delivery once internet connectivity is restored.
- Variations in patient and staff digital literacy levels may affect system usability, thereby necessitating simple user interfaces and staff orientation sessions.
- Access to hospital records and patient flow data for baseline comparison may be limited due to the manual nature of existing systems and data privacy considerations.
- Intermittent power supply and limited hardware resources at the hospital may affect system deployment and testing activities.
- The limited timeframe allocated for the final-year project may restrict the depth of user acceptance testing and system evaluation conducted.

1.5.2 Delimitations of the Study

The study is delimited as follows:

- The system focuses exclusively on Chinhoyi Provincial Hospital as the case study institution. Generalisation to other hospitals is proposed but falls outside the scope of this project.
- The system covers five departments: Outpatient Department (OPD), Casualty/Triage, Pharmacy, Laboratory/Radiology, and Opportunistic Infection (OI) Clinic. Inpatient ward management and theatre scheduling are excluded from the scope of the project.
- Patient notifications are limited to WhatsApp Business API notifications and web push notifications only. SMS notifications and dedicated mobile applications are not included in this phase of the project.
- Public and department-specific queue boards were included in the prototype; procurement and physical installation of hospital display hardware remained outside the project scope.
- The system does not integrate with existing Hospital Information Systems (HIS) or Electronic Health Records (EHR) platforms in this phase.
- The project focuses on prototype development and pilot evaluation rather than full-scale production deployment.

1.6 Definition of terms

Queue Management System: A digital system used to organise, monitor, and manage the flow of patients waiting for healthcare services in order to improve efficiency and reduce waiting times.

Triage: The process of prioritising patients according to the severity and urgency of their medical condition to ensure timely medical attention.

Outpatient Department (OPD): A hospital department that provides registration, consultation, and treatment services to patients who do not require overnight admission.

Casualty Department: The hospital department responsible for the initial reception, triage, and emergency treatment of patients requiring urgent medical attention.

Opportunistic Infection (OI) Clinic: A specialised hospital clinic that provides care for HIV/AIDS patients, including antiretroviral therapy resupply, TB services, viral load testing, cervical cancer screening, and counselling.

Digital Health System: A technology-based healthcare solution designed to improve healthcare service delivery, patient management, and operational efficiency through the use of digital tools and platforms.

Web Push Notification: A browser-based notification sent from a web application to a user's device to provide alerts or updates, even when the application is not actively open.

WhatsApp Business API: A communication platform provided by Meta that enables businesses and organisations to send automated messages and notifications to users through WhatsApp.

Local Area Network (LAN): A computer network that connects devices within a limited geographical area such as a hospital, office, or campus to enable communication and resource sharing.

Patient Flow: The movement of patients through different stages of healthcare service delivery, from registration and triage to consultation, treatment, and discharge.

Real-Time System: A system capable of processing and updating information immediately or with minimal delay as events occur.

Hospital Information System (HIS): An integrated digital system used by healthcare institutions to manage administrative, financial, and clinical operations.

Electronic Health Record (EHR): A digital version of a patient's medical history and health information maintained over time by healthcare providers.

WebSocket: A communication protocol that enables continuous two-way communication between a client application and a server in real time.

Prototype: An initial working version of a system developed for testing, evaluation, and demonstration purposes before full deployment.

Application Programming Interface (API): A set of rules and protocols that allows different software applications or system components to communicate and exchange data with each other.

Departmental Queue: A queue assigned to a specific hospital department where patients wait to receive services related to that department.

Patient Notification System: A subsystem responsible for sending alerts and updates to patients regarding their queue status, appointment status, or service availability.

1.7 Conclusion

This chapter introduced the development of WaitLess, a digital queue management system designed for Zimbabwean public hospitals, with Chinhoyi Provincial Hospital serving as the case study institution. The chapter discussed the existing manual patient flow and triage processes currently used at the hospital and identified key inefficiencies associated with the absence of a digital queue management system.

The chapter further highlighted the significance of integrating digital queue coordination and patient notification mechanisms into public healthcare service delivery. Particular emphasis was placed on the use of WhatsApp Business API notifications and web push notifications to improve patient convenience and reduce missed queue turns without introducing recurring messaging costs to the hospital.

The aim, objectives, significance, limitations, and delimitations of the study were also presented to establish the boundaries and direction of the project. Collectively, these elements provide the foundation for the design and development of an affordable, scalable, and contextually appropriate queue management solution for Zimbabwean public healthcare institutions.

The next chapter reviews related literature and existing digital healthcare solutions relevant to queue management systems, patient flow optimisation, and healthcare digitalisation.

Chapter 2: Literature Review

2.0 Introduction

This chapter presents a review of literature relevant to the development of WaitLess. It examines existing scholarship on queue management theory in healthcare, digital queue management system implementations in hospital settings, patient notification mechanisms, digital health adoption in Zimbabwe and the wider Sub-Saharan African region, design science research as a methodological foundation for information systems projects, and the technology frameworks suited to real-time web applications. The chapter closes by synthesising the gaps that emerge from this body of work and explaining how WaitLess is positioned to address them.

The chapter is organised thematically. Section 2.1 begins with the theoretical foundations of queue management. Section 2.2 reviews specific digital queue management system implementations in hospital contexts, both globally and within Sub-Saharan Africa. Section 2.3 focuses on patient notification systems, a central feature of WaitLess. Section 2.4 examines digital health adoption in Zimbabwe. Section 2.5 discusses design science research as the methodological foundation adopted for this project, and Section 2.6 reviews the technology frameworks relevant to its implementation. Section 2.7 reviews existing commercial queue management systems, and Section 2.8 synthesises the gaps identified across the reviewed literature. Section 2.9 concludes the chapter.

2.1 Queue Management Theory in Healthcare

Queue management, as a field of academic enquiry, draws on queuing theory, a branch of operations research concerned with the mathematical study of waiting lines. Preater (2002) traced the growing application of queuing theory within healthcare and medicine, arguing that queue modelling provides a foundational analytical tool for understanding and managing patient flow within hospitals. This early framing established queue management not merely as an operational inconvenience to be minimised, but as a legitimate object of systematic study with direct implications for clinical safety and service quality.

Building on this foundation, Cayirli and Veral (2003) conducted a comprehensive review of outpatient scheduling and appointment systems literature. Their review concluded that the primary drivers of patient waiting time were unpredictable patient arrival patterns, variable service durations, and the absence of real-time monitoring tools, and argued that digital scheduling and queue management systems were essential tools for addressing these inefficiencies.

Fomundam and Herrmann (2007) extended this line of enquiry through a broader survey of queuing theory applications across the healthcare sector. Their survey concluded that the primary barriers to effective queue management in public hospitals were the lack of real-time data visibility and the absence of coordinated patient tracking systems spanning multiple departments. This finding mirrors the conditions documented

during the case study conducted at Chinhoyi Provincial Hospital, where patient movement between the OPD, laboratory, radiology, and pharmacy is coordinated entirely through handwritten records with no shared visibility across departments.

Taken together, these three works establish queue management as a well-theorised problem domain, but one whose proposed solutions have historically remained largely conceptual or confined to high-income, well-resourced healthcare settings. None of the three studies reviewed in this section report on an implemented system evaluated in a low-resource, Sub- Saharan African hospital, a gap addressed by the empirical implementations reviewed in the following section.

2.2 Digital Queue Management Systems in Hospitals

Where Section 2.1 established the theoretical case for digital queue management, a growing body of empirical literature has evaluated implemented systems directly. Cho et al. (2017) applied queueing analysis to outpatient data before and after electronic medical records were introduced in public hospitals. Their results showed reductions in outpatient waiting time ranging from 44% to 78%. The study demonstrated the value of timestamped digital records for identifying delays, but it did not address triage-aware routing or patient notifications.

Bidari et al. (2021) provided some of the strongest causal evidence available in this literature through a prospective randomised controlled trial evaluating the effect of a queue management system on patient satisfaction in a hospital emergency department. The trial involved 236 patients allocated to intervention and control groups. The results showed that both actual and perceived waiting times were significantly lower in the intervention group, and that patient satisfaction improved markedly with the introduction of the queue management system. The randomised design of this study lends particular methodological weight to the broader claim, made throughout this literature, that queue management interventions produce measurable improvements in patient experience rather than merely a perception of improvement.

Kanchanawanishkul et al. (2018) designed a wearable wireless queue device for a university medical service centre. The prototype allowed patients to move away from a fixed display while still receiving visual and audible queue updates. Although the setting and hardware differed from WaitLess, the work supported the principle that notification should be treated as part of queue management rather than as a separate convenience feature.

Within the African context specifically, an evaluation of technical and workflow interventions at Kilimanjaro Christian Medical Centre (KCMC) in Tanzania, reported by Mwanswila, Mollel and Mushi (2024), found that KCMC had historically recorded the longest outpatient waiting times among Tanzania's zonal and national referral hospitals, with patients waiting an average of six hours to receive OPD services. In response, the hospital implemented a multi-year programme of interventions, including the expansion of physical infrastructure, the streamlining of patient flow processes, the deployment of patient ushers, and increases in clinical staffing, which together produced a substantial and sustained reduction in patient waiting times. This case is instructive for the present study in two respects: first, it confirms that resource-constrained East African referral hospitals can achieve measurable waiting-time reductions through deliberate intervention; second, it

demonstrates that the most effective intervention documented in a comparable regional setting relied primarily on process redesign and staffing changes rather than on a dedicated, purpose-built digital queue management platform of the kind proposed in this project.

Collectively, these four studies demonstrate that digital and process-based queue management interventions are both feasible and effective across a range of healthcare settings, from high-income emergency departments to resource-constrained African referral hospitals. However, only Kanchanawanishkul et al. (2018) explicitly integrated patient notifications, and none of the four studies combined patient notification with a colour-coded triage prioritisation model of the kind required by Zimbabwean public hospitals. This combination represents a specific, unaddressed gap in the reviewed literature.

2.3 Patient Notification Systems in Healthcare

A recurring theme across the queue management literature reviewed in Section 2.2 is the growing recognition that patient notification is not a peripheral feature but a core determinant of system effectiveness. Despite this, the specific mechanism of notification varies considerably across implementations, and few studies address notification design in a manner appropriate to a resource-constrained, low-connectivity environment such as Chinhoyi Provincial Hospital.

The case study conducted at Chinhoyi Provincial Hospital identified the complete absence of any mechanism to notify patients of their queue status, particularly patients who leave hospital premises while waiting, as a critical gap in the existing manual process. WaitLess addresses this gap through two complementary notification channels selected specifically to minimise or eliminate ongoing cost to the hospital.

The primary external channel selected for the prototype was the WhatsApp Business Platform because WhatsApp was already familiar to many potential users and could carry short queue updates without requiring a separate mobile application. Provider pricing and conversation rules can change, so the project did not treat zero-cost messaging as a permanent technical assumption. Instead, notification delivery was isolated behind a provider adapter, allowing the hospital to configure the most affordable approved channel available at deployment time. This design decision responded to the funding constraints identified by Furusa and Coleman (2018).

The secondary channel was browser notification. It was suited to patients who registered through the web application or kept the tracking page open on a personal device. The prototype implemented the patient-facing browser workflow; full production push delivery still required deployment-specific service-worker keys and a push provider.

A complementary channel was the public queue board. The software provided anonymous token displays for general and department-specific waiting areas, while the purchase and installation of physical screens remained a hospital deployment decision.

The design of the WaitLess notification engine therefore responds directly to two distinct gaps identified elsewhere in this chapter: the general under-prioritisation of patient notification observed in Section 2.2, and the specific absence of any notification mechanism documented in the Chinhoyi Provincial Hospital case study.

2.4 Digital Health Adoption in Zimbabwe and Sub-Saharan Africa

Despite the demonstrated benefits of digital queue management systems reviewed in Section 2.2, adoption of digital health tools more broadly across Sub-Saharan Africa, and within Zimbabwe specifically, remains uneven. Furusa and Coleman (2018) used qualitative multiple-case research, including interviews, direct observation, and policy review, to study e-health implementation in Zimbabwean public hospitals. They identified infrastructure, ICT skills, technical support, security, funding, and policy as important influences on adoption.

This last barrier is of particular relevance to the present study. Much of the digital queue management literature reviewed in Section 2.2, and the commercial systems reviewed in Section 2.7, originates from or is designed for higher-resource contexts. Systems built for reliable broadband connectivity, well-staffed IT departments, or predictable electricity supply are not necessarily transferable, without substantial redesign, to a Zimbabwean provincial hospital operating under intermittent connectivity and constrained infrastructure. Furusa and Coleman's finding therefore provides direct empirical support for the project's core methodological decision to base system requirements on a physical case study conducted at Chinhoyi Provincial Hospital, rather than adapting an existing solution designed elsewhere.

These barriers aligned closely with the operating conditions documented at Chinhoyi Provincial Hospital, where intermittent internet connectivity and constrained hardware resources had to be treated as design conditions rather than exceptional cases.

2.5 Design Science Research as a Methodological Foundation

Beyond the substantive literature on queue management and digital health, this project also drew on methodological literature to justify its research approach, which is described fully in Chapter 3. Hevner et al. (2004) established a foundational framework for design science research (DSR) within the information-systems discipline. They argued that DSR involved the creation of purposeful artefacts, such as software systems, models, or methods, that addressed real organisational problems, and that the utility, quality, and efficacy of those artefacts had to be evaluated rather than simply asserted.

Bisandu (2019) further contextualised design science research specifically within computer science and information systems research, emphasising its particular relevance for projects that both develop and evaluate a software artefact as a direct response to a documented organisational problem. This framing maps closely onto the structure of the present project: the physical case study conducted at Chinhoyi Provincial Hospital documents a specific organisational problem, and the WaitLess system constitutes the designed artefact proposed as its solution.

The relevance of DSR to this project lies not only in its theoretical fit but in the explicit emphasis both Hevner et al. (2004) and Bisandu (2019) place on iterative evaluation. This has directly informed the decision to structure system development around distinct, evaluable phases, namely requirements analysis, design, implementation, testing, and evaluation, as detailed in Chapter 3, rather than pursuing a single-pass build with no structured evaluation checkpoint.

2.6 Technology Frameworks for Real-Time Web Applications

The implementation of WaitLess depends on technology choices capable of supporting real-time, concurrent updates across multiple hospital departments simultaneously. React and Node.js have emerged as a widely adopted full-stack combination for building this class of application. Aggarwal (2018) demonstrated that React's component-based architecture is particularly well suited to building live dashboard interfaces that must reflect dynamic, frequently changing data, a requirement central to the WaitLess staff dashboard, which must reflect queue state changes across the OPD, Casualty, Pharmacy, Laboratory/Radiology, and OI Clinic departments in near real time.

On the server side, Tilkov and Vinoski (2010) argued that Node.js's event-driven, non-blocking input/output model makes it well suited to handling large numbers of concurrent, lightweight updates of the kind generated by multiple hospital departments simultaneously reporting queue state changes. This architectural characteristic directly informs the choice of Node.js with Express.js and WebSocket support as the backend foundation for WaitLess, since a traditional synchronous, thread-per-request server architecture would be comparatively poorly suited to sustaining many simultaneous, low-latency department-to-dashboard update streams on the modest server hardware available within a hospital local area network.

While Aggarwal (2018) and Tilkov and Vinoski (2010) establish the general suitability of this stack for real-time applications, neither study addresses deployment within a low-bandwidth, intermittently connected environment of the kind found at Chinhoyi Provincial Hospital. This is not a shortcoming of either work, both of which concern general web application architecture rather than healthcare deployment specifically, but it does mean that the low-bandwidth, offline-tolerant configuration of WaitLess's technology stack, detailed in Chapter 3, extends beyond what either source directly evaluates.

2.7 Review of Commercial Queue Management Systems

Beyond the academic literature reviewed above, a review of existing commercial queue management systems provides additional context for the gap this project addresses. Established commercial platforms such as Qmatic, Qtrac, and Wavetec are feature-rich, mature products with a strong track record of deployment in high-income country settings, including retail, banking, and some private healthcare environments. However, three characteristics limit their suitability for a Zimbabwean public hospital context. First, licensing and deployment costs are typically structured for well-resourced private or high-income public institutions, placing them outside the budget of a Zimbabwean public hospital such as Chinhoyi Provincial Hospital. Second, these platforms generally assume consistent, high-bandwidth internet connectivity, an assumption that does not hold reliably at Chinhoyi Provincial Hospital. Third, and most significantly for this project, none of these commercial platforms are designed around the specific colour-coded triage protocols (Red, Yellow, Green, Black) or the multi-department referral workflows that characterise Zimbabwean public hospital operations; their prioritisation and routing logic is generic rather than aligned to this clinical context.

This review of commercial systems is necessarily limited to publicly available product information rather than independent, peer-reviewed evaluation, and should be read as market context rather than as literature in the academic sense. Even accounting for this limitation, the pattern is consistent with the academic literature reviewed in Sections 2.1 and 2.2: mature, well-resourced solutions exist, but none combine affordability, offline tolerance, and Zimbabwean-triage-aware routing in a single system.

2.8 Identified Gaps in the Literature

The literature reviewed in this chapter establishes that queue management is a well-theorised problem (Section 2.1), that digital and process-based interventions have been implemented and evaluated with measurable success across a range of healthcare settings (Section 2.2), that patient notification is increasingly recognised as a core rather than peripheral system feature (Section 2.3), and that infrastructural and contextual barriers continue to limit digital health adoption specifically within Zimbabwe (Section 2.4). Read together, this body of work reveals a specific combination of gaps that no single reviewed source addresses simultaneously.

First, none of the implementations reviewed in Section 2.2 combined patient notification with colour-coded triage prioritisation aligned to the four-tier classification used at Chinhoyi Provincial Hospital. Kanchanawanishkul et al. (2018) explored portable notifications without clinical routing, while Cho et al. (2017) examined digital records and waiting-time analysis rather than multi-department queue control.

Second, the reviewed notification approaches depended either on dedicated hardware or an external communication service. This created an unresolved design question for a resource-constrained public hospital: how to provide timely alerts while keeping recurring cost and technical maintenance manageable.

Third, the commercial systems reviewed in Section 2.7 demonstrate that feature-complete queue management platforms exist, but at a cost and connectivity requirement that places them outside the reach of Zimbabwean public hospitals, while the academic literature on affordable, locally developed digital health tools (Section 2.4) has not, to date, produced a queue management system specifically aligned to Zimbabwean triage protocols.

Fourth, the Tanzanian evaluation by Mwanswila, Mollel and Mushi (2024) demonstrated that deliberate technical and process interventions could reduce waiting time in a regional referral hospital, but it did not evaluate the triage-aware notification architecture proposed by WaitLess.

WaitLess was positioned within this combined gap: a web-based queue-management system that integrated colour-coded triage-aware routing, real-time multi-department visibility, and configurable patient notifications while remaining suitable for the infrastructural and budgetary realities of a Zimbabwean public hospital.

2.9 Conclusion

This chapter has reviewed literature across six interconnected areas: the theoretical foundations of queue management in healthcare, empirical implementations of digital queue management systems in hospital settings, patient notification mechanisms, digital health adoption within Zimbabwe and the wider Sub-Saharan African region, design science research as a methodological foundation, and the technology frameworks suited to real-time web application development. A review of existing commercial queue management platforms provided additional market context.

Across this literature, a consistent pattern emerged: digital interventions in queue management are both feasible and effective, but no single reviewed system combines triage-aware patient routing, real-time multi-department visibility, and a zero-recurring-cost patient notification engine within a system designed for the

infrastructural and budgetary constraints of a Zimbabwean public hospital. This combination of gaps, detailed in Section 2.8, defines the specific contribution WaitLess is intended to make.

The following chapter presents the methodology adopted to design, develop, and evaluate WaitLess, structured according to the design science research framework established by Hevner et al. (2004) and discussed in Section 2.5 of this chapter.

Chapter 3: Methodology

3.0 Introduction

This chapter explains how the WaitLess project was investigated, designed, built, and evaluated. The work combined a single-site case study with design science research because the project had two connected goals: to understand a practical patient-flow problem in its real setting and to produce a working software artefact that responded to that problem. The methodology was deliberately iterative. Requirements were refined as hospital workflows became clearer, and each implemented module was checked against the problem it was intended to solve.

The chapter describes the research design, case selection, data sources, requirements-analysis procedure, development method, evaluation strategy, ethical safeguards, and limitations of the approach. It also distinguishes evidence gathered from the hospital context from results generated through prototype testing. This distinction was important because the project did not claim a full clinical deployment or a large-scale patient trial.

3.1 Research Philosophy and Approach

The project adopted a pragmatic research position. The central question was not whether one research tradition was universally preferable, but which combination of methods could produce a useful and defensible response to the queue-management problem. Qualitative evidence was used to understand workflow, roles, constraints, and exceptions. Quantitative measures were used where the prototype could be tested objectively, including automated-test results, build status, API behaviour, queue metrics, and response-time observations.

The overall approach was therefore an applied, mixed-evidence design rather than a statistical population study. The case-study evidence explained the local problem; design science guided the construction of the artefact; and software evaluation established what the prototype could and could not do.

3.2 Research Design

The research design combined three mutually supporting methods.

Table 3.1: Research Design

Method	Purpose in the project	Main output
Single-site case study	To understand patient movement, triage, departmental hand-offs, and infrastructure constraints at Chinhoyi Provincial Hospital	Context-specific problem definition and workflow requirements
Design science research	To design and construct a useful software artefact in response to the observed problem	WaitLess web application, data model, queue logic, and notification workflow

Method	Purpose in the project	Main output
Software-system evaluation	To test functional correctness, usability characteristics, security controls, build quality, and deployment suitability	Test results, requirements traceability, limitations, and recommendations

The stages were connected rather than strictly linear. A workflow observation could lead to a revised requirement; a failed test could lead to an implementation change; and an implementation constraint could require a more realistic interpretation of the original requirement.

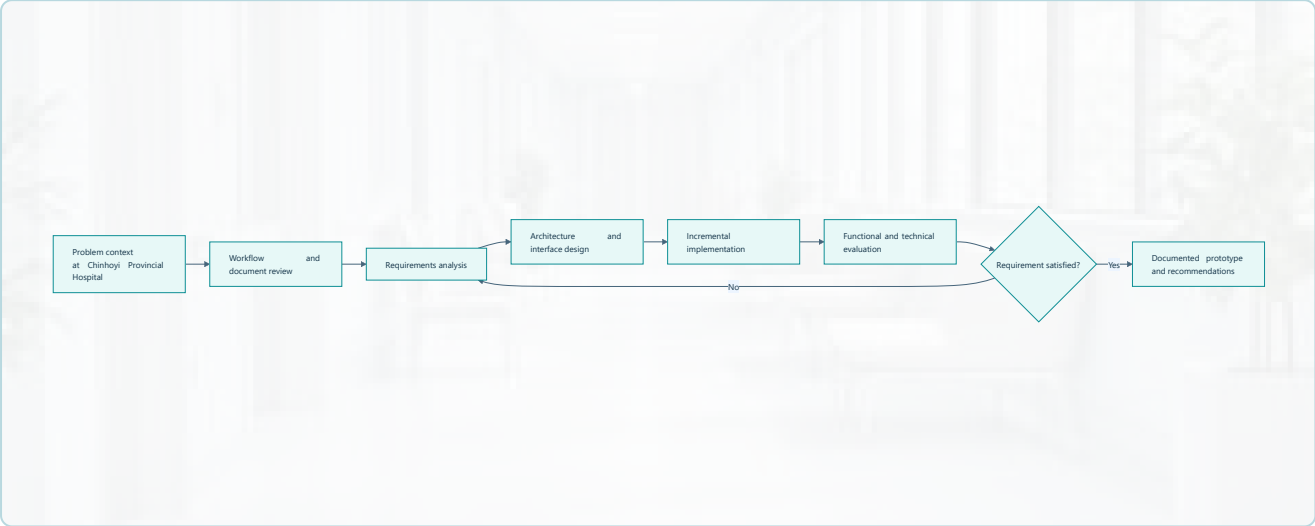


Figure 3.1: Research Design

3.3 Design Science Research Method

Design science research was selected because the principal outcome was a software artefact designed to solve an identified organisational problem. Hevner et al. (2004) described design science as the creation and evaluation of purposeful artefacts, while Peffers et al. (2007) organised the method around problem identification, objectives, design and development, demonstration, evaluation, and communication. These stages closely matched the work carried out in WaitLess.

3.3.1 Problem Identification

The initial problem was expressed broadly as long and poorly coordinated queues. Case-study analysis refined it into several connected issues: registration and triage records were disconnected; queue position was not visible across departments; clinical urgency and arrival order had to be reconciled; patients could miss turns when they left a waiting area; and management lacked dependable operational data.

3.3.2 Objectives of the Artefact

The artefact was expected to provide one traceable patient-flow record from registration through triage and departmental service. It also had to preserve clinical priority, support anonymous public queue displays, provide role-appropriate staff controls, communicate queue events, and remain usable on an ordinary hospital local-area network.

3.3.3 Design and Development

The design was expressed through use cases, data-flow diagrams, an entity-relationship model, sequence diagrams, activity diagrams, access-control rules, and interface specifications. Development then proceeded by module, beginning with the queue core and extending into triage, dashboards, notifications, reporting, appointments, laboratory and pharmacy workflows, and administrative controls.

3.3.4 Demonstration and Evaluation

The prototype was demonstrated through realistic but non-identifying test records. Evaluation combined automated backend tests, manual end-to-end scenarios, production-build verification, API checks, interface review, requirements traceability, and a security-control review. Findings were used to separate completed behaviour from partially implemented or deployment-dependent behaviour.

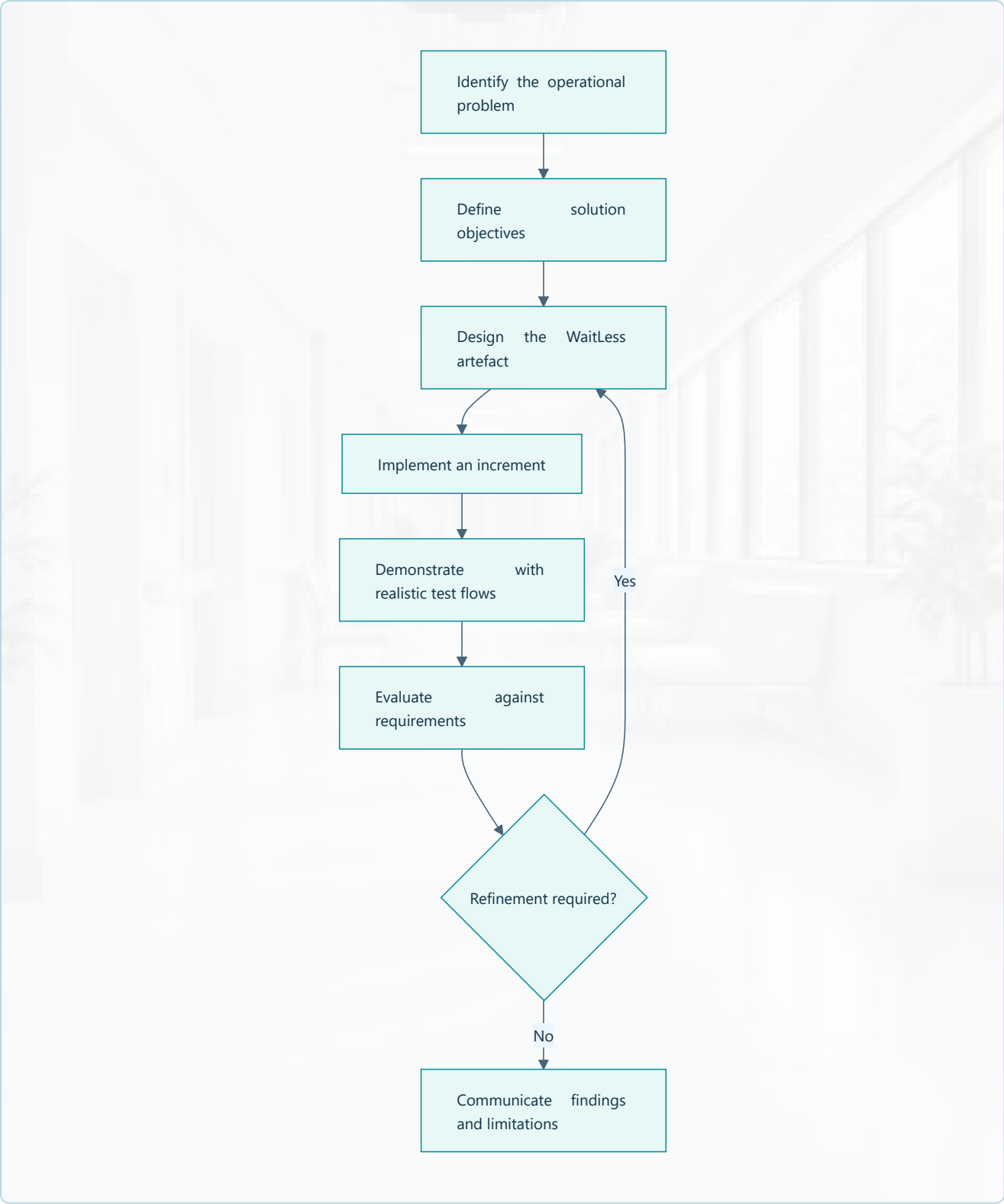


Figure 3.2: Demonstration and Evaluation

3.4 Case Study and Unit of Analysis

Chinhoyi Provincial Hospital was selected purposively because it provided a realistic public-hospital setting with outpatient, casualty, diagnostic, pharmacy, and OI service streams. The hospital was not treated as a statistically representative sample of every Zimbabwean health facility. It was treated as a bounded case

through which a context-appropriate artefact could be developed.

The main unit of analysis was the patient journey through the queue-management process. Supporting units included the registration desk, triage point, departmental queue, public display, notification event, staff action, and management report. This framing kept the project focused on coordination and waiting flow rather than attempting to replace a complete electronic health-record system.

3.5 Data Sources and Collection Methods

3.5.1 Direct Workflow Observation

Non-participant observation was used to understand how patients entered the hospital, moved between service points, and waited for the next action. Attention was given to hand-offs, decision points, repeated data entry, visible bottlenecks, and the difference between ordinary and urgent cases. Notes from this process informed the problem statement and the early use cases.

3.5.2 Hospital Workflow Artefacts

Available workflow artefacts were reviewed, including the OPD patient-flow map, colour-coded casualty triage categories, service-time indications, referral routes, and OI Clinic streams. These materials helped translate a general idea of a queue into concrete system states, department names, routing rules, and safety thresholds.

3.5.3 Literature and Policy Review

Published work on queueing theory, patient satisfaction, hospital waiting time, design science, electronic records, and e-health adoption was reviewed. Zimbabwe's National Health Strategy 2021-2025 was also considered to place the project within the wider direction of digital health and service improvement. Literature was used both to justify the problem and to challenge assumptions made during design.

3.5.4 Existing-System and Code Evidence

During implementation, evidence was also drawn from the system itself. API responses, audit events, test fixtures, queue-state transitions, notification attempts, production-build output, and source-code structure were examined. These sources supported technical evaluation but were not presented as clinical outcome data.

Table 3.2: Existing-System and Code Evidence

Evidence source	Information extracted	Use in the project
Direct observation	Patient movement, waiting points, staff hand-offs	Problem definition and workflow modelling
OPD and triage artefacts	Stages, departments, priority categories, indicative times	Requirements and routing rules
OI service-flow information	ART, TB, viral-load, counselling, and screening streams	Confidential routing and module scope
Academic literature	Prior findings, research gap, evaluation approaches	Literature synthesis and methodological justification
Source code and API output	Implemented behaviour and constraints	Verification and traceability

Evidence source	Information extracted	Use in the project
Automated and manual tests	Pass/fail evidence and observed defects	Presentation of results

3.6 Requirements Analysis Procedure

Requirements were derived through a traceable sequence. Observed workflow needs were first written as plain-language problems. Each problem was converted into a user or system requirement, assigned a priority, and mapped to one or more use cases. The corresponding design model was then linked to an implementation module and a test case.

For example, the observation that a critically ill patient could not be represented safely by a simple first-in-first-out line became a requirement for four-level triage priority. That requirement was reflected in the triage use case, queue-ordering rule, priority data field, dashboard presentation, and tests for call-next behaviour. This chain reduced the risk of adding attractive features that did not address the hospital problem.

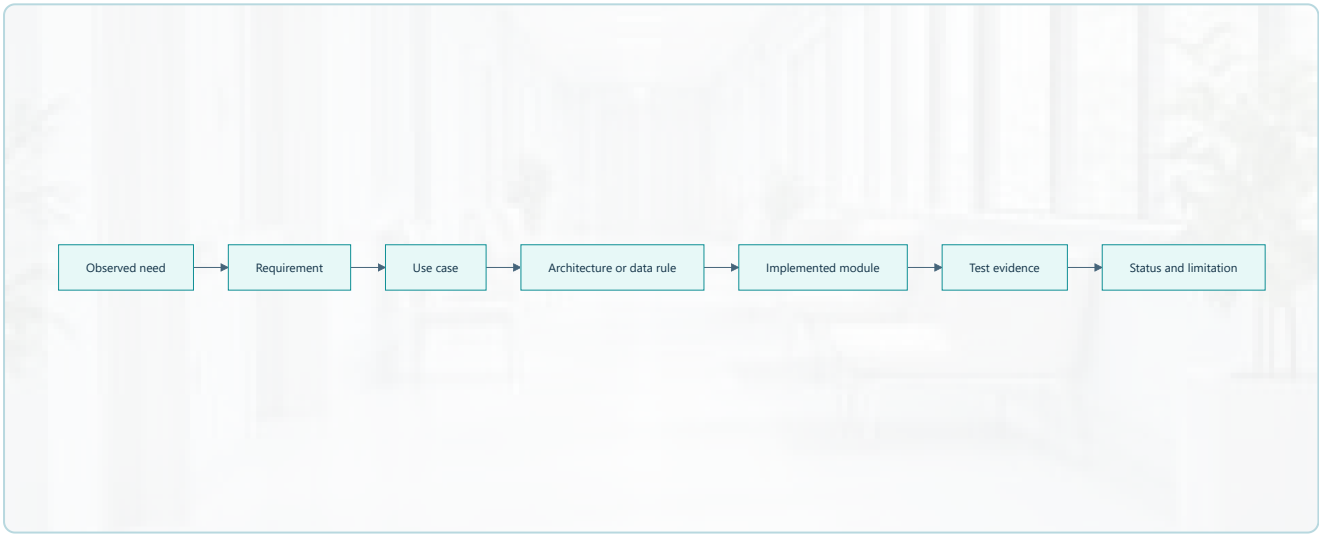


Figure 3.3: Requirements Analysis Procedure

Requirements were classified as functional or non-functional and prioritised as high, medium, or low. High-priority work covered registration, triage, queue ordering, staff authentication, anonymous public display, and operational traceability. Lower-priority analytics and deployment enhancements were implemented only after the core patient-flow path was stable.

3.7 System Development Method

An iterative and incremental development method was used. A rigid single-pass process would have been poorly suited to a project in which queue rules, interface behaviour, and deployment assumptions were clarified during construction. Work was organised into short feature increments, each of which passed through clarification, design, implementation, verification, and review.

The main increments were:

1. Patient registration, ticket generation, and basic queue state.
2. Digital triage and priority-aware queue ordering.
3. Staff dashboard, call-next actions, missed turns, recall, and transfer.
4. Patient tracking, public queue boards, and department displays.
5. Notification logging, retry handling, and delivery analytics.
6. Reports, audit history, appointments, pharmacy, laboratory, and OI support.
7. Authentication hardening, profile management, navigation, and deployment preparation.

Version control was used to preserve implementation history. Reusable components and service boundaries were preferred so that changes to one workflow did not require unrelated pages to be rewritten. Defects found during testing were corrected within the same increment where practical; unresolved work was recorded as a limitation or future recommendation.

3.8 Technology Selection Procedure

Technology choices were judged against the project context rather than popularity alone. The frontend required responsive, component-based interfaces capable of frequent state updates. The backend required lightweight concurrent connections, structured APIs, and real-time event delivery. Data storage had to support a production relational database while retaining a simple development mode. The selected React, Node.js, Express, Server-Sent Events, and MySQL-oriented architecture met these needs and matched the skills and hardware available to the project team.

Other criteria included maintainability, licence cost, local development support, browser compatibility, and the ability to deploy on a hospital LAN. The detailed justification and resulting architecture are presented in Chapter 4.

3.9 Evaluation Strategy

Evaluation was structured around the three project objectives.

Table 3.3: Evaluation Strategy

Objective	Evaluation question	Evidence used
Analyse the existing workflow	Were the important actors, stages, priorities, and constraints represented?	Case-study notes, workflow artefacts, requirements catalogue, use cases
Design and develop the system	Did the prototype implement the required patient-flow and staff-control behaviour?	Source-code inspection, API verification, automated tests, manual scenarios, build output
Evaluate suitability	Was the prototype understandable, responsive, secure enough for demonstration, and realistic for constrained deployment?	Interface review, task scenarios, performance observations, security checklist, limitation analysis

3.9.1 Functional Testing

Backend unit tests checked queue ordering, registration, status transitions, notification retries, authentication-related behaviour, and report calculations. Manual tests exercised cross-page journeys such as registration to triage, call-next to patient tracking, missed turn to recall, and notification failure to retry.

3.9.2 Usability Evaluation

Usability was examined through task-based walkthroughs and responsive interface inspection. The System Usability Scale questionnaire was prepared as a repeatable instrument for pilot users; however, the documentation did not present an invented population score where a formal sample had not been completed. Interface findings were therefore reported as prototype evidence rather than as a hospital-wide usability claim.

3.9.3 Performance and Reliability Evaluation

The frontend production build, API response behaviour, real-time update path, and persistence modes were checked. Low-bandwidth suitability was considered through asset size, browser-based deployment, local-network operation, and graceful notification failure. Full production load testing with hospital-scale concurrent users remained future work.

3.9.4 Security Evaluation

The evaluation reviewed password hashing, session handling, role checks, audit logging, public-display privacy, consent, input validation, and configuration risks. Security findings were documented honestly: prototype controls that were present were distinguished from deployment controls that still required production secrets, hardened hosting, rate limiting, backup procedures, and operational monitoring.

3.10 Data Analysis

Qualitative evidence was analysed thematically. Observations and workflow materials were grouped into recurring concerns such as registration delay, triage safety, department hand-off, missed turns, visibility, reporting, confidentiality, and infrastructure. These themes became requirement groups and later provided categories for evaluating the prototype.

Quantitative prototype evidence was summarised descriptively. Counts, percentages, waiting-time calculations, test pass rates, build sizes, and notification statuses were reported without implying statistical generalisation beyond the tested data. Where real hospital baseline data were unavailable, the report stated that limitation instead of constructing unsupported before-and-after claims.

3.11 Validity, Reliability, and Trustworthiness

Several measures were used to strengthen the study. Requirements were triangulated across observation, hospital workflow artefacts, literature, and the implemented system. A requirements-traceability matrix connected needs to tests. Repeatable automated tests reduced dependence on a single manual

demonstration. Diagrams, API contracts, and data definitions made design decisions inspectable by another developer or supervisor.

Threats remained. The single-case design limited generalisation, observation could miss infrequent events, and prototype test data could not reproduce every clinical or organisational condition. These limitations were mitigated by avoiding claims of clinical effectiveness and by presenting future pilot evaluation as a necessary next step.

3.12 Ethical and Privacy Considerations

The project dealt with healthcare workflows and therefore applied privacy-by-design principles even though it remained a prototype. Demonstration records used synthetic identities. Public queue boards displayed ticket numbers rather than patient names. Staff actions were attributed through authenticated accounts, sensitive routes were role-protected, notification consent was recorded, and OI-related information was treated as confidential.

No diagnosis or treatment decision was delegated to the software. Triage logic supported authorised staff and provided escalation cues; it did not replace clinical judgement. Formal hospital approval, production data-governance agreements, and patient-participant consent would be required before live clinical deployment or a patient outcome study.

3.13 Implementation Plan and Milestones

The implementation plan was organised around observable deliverables and completion criteria, as required by the project guide.

Table 3.4: Implementation Plan and Milestones

Milestone	Deliverable	Completion criterion	Target period
Problem analysis	Case-study notes and problem statement	Core workflow and constraints documented	March-April 2026
Requirements	Prioritised functional and non-functional requirements	Requirements reviewed and mapped to actors	April-May 2026
System design	Architecture, ERD, DFDs, use cases, sequences, and activity models	Major workflows represented consistently	May-June 2026
Core implementation	Registration, triage, queues, dashboards, tracking	End-to-end patient flow demonstrable	June-July 2026
Extended implementation	Notifications, reports, appointments, department modules	Priority features integrated with the queue core	July-August 2026
Verification	Automated tests, manual scenarios, build and security review	Critical test paths pass and limitations are recorded	August 2026
Documentation	Dissertation, diagrams, user guides, and deployment notes	Submission package generated and reviewed	August 2026
Pilot preparation	UAT plan, deployment checklist, training material	Ready for authorised hospital pilot	Post-submission

Project deliverables included the frontend and backend source code, database schema, environment templates, automated tests, system diagrams, deployment guidance, patient and staff user manuals, and the completed dissertation.

3.14 Methodological Limitations

The methodology produced a well-documented prototype, but it did not substitute for a controlled clinical evaluation. The project did not have a sufficiently large verified dataset to measure a statistically reliable reduction in hospital waiting time. It also did not complete a representative survey of patients and staff. External services such as WhatsApp and web push required production credentials and provider configuration that could not be fully validated in an offline demonstration environment.

These limitations shaped the conclusions. The project could demonstrate functional feasibility, traceable design, and readiness for pilot preparation; it could not yet claim population-level patient satisfaction, long-term operational savings, or improved clinical outcomes.

3.15 Chapter Summary

This chapter described the pragmatic case-study and design science methodology used to develop WaitLess. The method connected observed workflow problems to requirements, design models, implementation modules, and test evidence. It also established the boundaries of the evaluation so that the final claims remained proportional to the evidence. Chapter 4 presents the resulting system design and implementation in detail.

Chapter 4: System Design and Implementation

4.0 Introduction

This chapter translates the requirements identified in Chapters 1 and 2, and the methodology described in Chapter 3, into the design and implementation of the WaitLess prototype. It first presents the requirements, use cases, architecture, data structures, process flows, interface approach, and security model. It then explains how those decisions were implemented in the frontend, backend, storage, real-time update, notification, reporting, and deployment layers. The chapter deliberately distinguishes a feature represented in the prototype from a control that still requires production configuration or hospital acceptance.

4.1 Requirements Analysis

The status values in this section describe the prototype at the documentation freeze on 10 August 2026. A status of **Complete** means that the feature was represented in the prototype and available for controlled verification; it does not by itself indicate production accreditation or completion of a live hospital pilot. **Partial** identifies a usable foundation that still depended on provider configuration, broader acceptance testing, infrastructure hardening, or another stated control.

4.1.1 Functional Requirements

The functional requirements are organised by module, with each requirement assigned a unique identifier consistent with the project's amendments and traceability audit.

Module 1: Patient Registration

Table 4.1: Module 1: Patient Registration

ID	Requirement	Priority	Status
1.1	The system shall allow staff to electronically register patients with demographics, national ID, phone, address, category, next of kin, chief complaint, department, and notification consent.	High	Complete
1.2	The system shall support returning-patient search by name, phone, national ID, patient number, previous ticket, or scanned QR code, and reuse existing records without duplication.	High	Complete
1.3	The system shall generate a unique, immutable queue token (e.g., OPD-205, LAB-206) consistent across print, phone, WhatsApp, and display board.	High	Complete
1.4	The system shall generate a QR patient card for self-service identification and reception lookup.	Medium	Partial
1.5	The system shall classify patients by category: walk-in, referred, emergency, elderly, pregnant, child, disabled, and chronic-care.	Medium	Complete

Module 2: Patient Self-Service

Table 4.2: Module 2: Patient Self-Service

ID	Requirement	Priority	Status
2.1	The system shall allow online pre-registration with arrival activation by reception staff.	Medium	Partial
2.2	The system shall provide a self-service kiosk for check-in by QR code scan or phone number lookup, collecting department and visit reason, and issuing a queue token.	High	Complete
2.3	The system shall provide a live patient queue-status tracking page showing token, department, position, patients ahead, ETA, progress, directions, and notification history.	High	Complete
2.4	The system shall provide a patient feedback form for satisfaction, cleanliness, communication, and wait-time feedback.	Low	Complete

Module 3: Triage

Table 4.3: Module 3: Triage

ID	Requirement	Priority	Status
3.1	The system shall support digital triage assessment capturing chief complaint, symptoms, vitals (oxygen saturation, heart rate, blood pressure, temperature, respiratory rate), pain score, and severity.	High	Complete
3.2	The system shall classify patients into four priority levels: Red (Critical), Yellow (Serious), Green (Ambulant), and Black (Deceased).	High	Complete
3.3	The system shall automatically fast-track Red priority cases based on emergency vital thresholds and trigger escalation alerts.	High	Complete
3.4	The system shall generate reassessment alerts for Red and Yellow patients who have been waiting beyond defined thresholds.	High	Complete
3.5	The system shall allow authorised triage staff to manually override the priority assignment, recording the before/after priority and the staff actor.	High	Complete

Module 4: Queue Management

Table 4.4: Module 4: Queue Management

ID	Requirement	Priority	Status
4.1	The system shall automatically assign patients to a department queue after registration or triage.	High	Partial
4.2	The system shall maintain department-specific queues for OPD, Casualty/Triage, Pharmacy, Laboratory, Radiology, and OI Clinic.	High	Complete
4.3	The system shall call the next patient by priority (Red > Yellow > Green > Black), then by arrival time, within a selected department.	High	Complete
4.4	The system shall mark a called patient as missed if they do not present for service.	High	Complete
4.5	The system shall support fair recall of missed patients with configurable reinsertion rules.	Medium	Partial
4.6	The system shall support queue transfer of a patient to another department with patient notification.	High	Complete
4.7	The system shall calculate dynamic estimated waiting time based on historical service time and queue position.	Medium	Partial
4.8	The system shall detect and display queue bottlenecks based on department load and thresholds.	Medium	Partial

Module 5: Department Routing

Table 4.5: Module 5: Department Routing

ID	Requirement	Priority	Status
5.1	The system shall support automatic department routing based on chief complaint keyword matching.	Medium	Partial
5.2	The system shall support doctor-to-laboratory test order workflow with lifecycle states (ordered, sample-collected, in-progress, completed, result-ready).	Medium	Complete
5.3	The system shall support doctor-to-pharmacy prescription workflow with lifecycle states (prescribed, dispensing, ready, collected).	Medium	Complete
5.4	The system shall support OI Clinic service streams (ART, TB, viral load, counselling, screening).	Low	Complete

Module 6: Staff Dashboard

Table 4.6: Module 6: Staff Dashboard

ID	Requirement	Priority	Status
6.1	The system shall provide a live queue dashboard with real-time metrics, queue tables, patient actions, alerts, and automatic refresh.	High	Complete
6.2	The system shall provide department-scoped dashboards that filter data by staff department assignment.	Medium	Complete
6.3	The system shall provide an administrator dashboard with staff management, audit trail, queue overview, reports, and settings.	High	Complete
6.4	The system shall display operational alerts including critical metrics, reassessment alerts, and notification failures.	Medium	Complete

Module 7: Notifications

Table 4.7: Module 7: Notifications

ID	Requirement	Priority	Status
7.1	The system shall send WhatsApp notifications for registration, turn readiness, call, missed turn, and transfer events.	High	Partial
7.2	The system shall support web push notifications.	Low	Partial
7.3	The system shall provide automatic and manual notification retry with configurable retry delays and attempt limits.	High	Complete
7.4	The system shall maintain a notification delivery log with status, timestamps, provider message ID, and error details.	High	Complete
7.5	The system shall send a missed-turn notification when a called ticket is marked missed.	High	Complete

Module 8: Queue Display Screens

Table 4.8: Module 8: Queue Display Screens

ID	Requirement	Priority	Status
8.1	The system shall provide a public waiting-area display board showing now-serving, waiting, and missed token numbers without patient names.	High	Complete
8.2	The system shall provide department-specific display boards accessible via URL with department query parameter.	Medium	Complete
8.3	The system shall support audio calling of token numbers.	Low	Complete

Module 9: Reporting

Table 4.9: Module 9: Reporting

ID	Requirement	Priority	Status
9.1	The system shall generate a daily report by department, service, gender, age, and triage priority.	Medium	Complete
9.2	The system shall generate an average waiting time report by department with date filtering.	High	Complete
9.3	The system shall generate a department performance report with throughput, missed rate, and priority breakdown.	High	Complete
9.4	The system shall generate a no-show report with notification delivery correlation per missed ticket.	High	Complete
9.5	The system shall generate a triage report with triage rate, timing, outcomes, overrides, and trends.	High	Complete
9.6	The system shall support PDF, Excel, and CSV export of reports.	Low	Complete

Module 10: Analytics

Table 4.10: Module 10: Analytics

ID	Requirement	Priority	Status
10.1	The system shall provide peak-hours analysis of patient arrivals and service patterns.	Low	Complete
10.2	The system shall provide queue-congestion prediction.	Low	Partial
10.3	The system shall provide resource-allocation suggestions based on current load.	Low	Partial

Module 11: Security and Access Control

Table 4.11: Module 11: Security and Access Control

ID	Requirement	Priority	Status
11.1	The system shall enforce role-based access control with roles: admin, reception, triage, and clinician.	High	Complete
11.2	The system shall provide secure individual login with bcrypt password hashing and action attribution.	High	Complete
11.3	The system shall maintain an immutable, hash-chained audit trail of all operational actions.	High	Complete
11.4	The system shall implement automatic idle logout with inactivity timeout.	Medium	Complete

Module 12: Privacy and Confidentiality

Table 4.12: Module 12: Privacy and Confidentiality

ID	Requirement	Priority	Status
12.1	The system shall display anonymous token numbers on public displays, never patient names.	High	Complete
12.2	The system shall capture and honour notification consent preferences.	High	Complete
12.3	The system shall provide confidential OI mode with masked routing and restricted data views.	Low	Partial

Module 13: Offline and Connectivity

Table 4.13: Module 13: Offline and Connectivity

ID	Requirement	Priority	Status
13.1	The system shall support local-network hosting within a hospital LAN.	Medium	Partial
13.2	The system shall maintain core queue operation without internet connectivity.	Medium	Partial
13.3	The system shall queue notifications for delivery when internet connectivity returns.	Medium	Partial
13.4	The system shall provide local form auto-save with IndexedDB draft recovery.	Low	Complete

Module 14–21: Additional Modules

Additional modules covering Appointment Management, OI Clinic, Pharmacy, Laboratory/Radiology, Staff Management, Patient Experience, Global System Features, and AI/Smart Assistance are defined in the project amendments. Their current implementation status is documented in the traceability matrix (Section 3.2.3).

4.1.2 Non-Functional Requirements

Table 4.14: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The system shall load the public queue board within 3 seconds on a standard hospital LAN connection.
NFR-02	Performance	The system shall deliver real-time queue updates to connected clients within 2 seconds of a state change via Server-Sent Events.
NFR-03	Performance	The frontend production JavaScript output shall remain below 3 MB uncompressed and shall use lazy-loaded chunks for major routes.
NFR-04	Security	All passwords shall be hashed using bcrypt with a minimum cost factor of 12.
NFR-05	Security	Staff sessions shall expire after 8 hours of inactivity.
NFR-06	Security	The audit trail shall be tamper-evident using SHA-256 hash chaining.
NFR-07	Security	Public displays shall never expose patient names or personally identifiable information.
NFR-08	Usability	The system shall be usable on desktop, tablet, and mobile devices with responsive design.
NFR-09	Usability	The system shall use a consistent visual language with medical teal (#0F766E) as the primary brand colour.
NFR-10	Reliability	The system shall persist all queue state to the database, surviving server restarts without data loss.
NFR-11	Reliability	The system shall support dual-mode storage (MySQL or file-based JSON) for deployment flexibility.
NFR-12	Scalability	The system shall support concurrent connections from at least 50 staff and display board clients.
NFR-13	Maintainability	The system shall use a modular architecture with clear separation of concerns (controller, service, repository layers).
NFR-14	Maintainability	The system shall use a component-based frontend architecture with reusable UI components.

ID	Category	Requirement
NFR-15	Compatibility	The system shall run on Node.js v18+ and modern browsers (Chrome, Firefox, Edge, Safari).

4.1.3 User Requirements by Stakeholder Role

Table 4.15: User Requirements by Stakeholder Role

Role	Key Requirements
Patient	Register online or at kiosk; receive a queue token; track queue status in real-time; receive WhatsApp notifications; view estimated wait time; leave and return without losing position.
Reception Staff	Register new and returning patients; search patient records; scan QR codes; manage queue tokens; transfer patients between departments; view department-scoped queue.
Triage Nurse	Conduct digital triage assessments; assign priority levels; override automatic classifications; view reassessment alerts; fast-track emergency cases.
Clinician/Doctor	Call next patient by priority; mark patients as in-service or completed; prescribe medications; order laboratory tests; transfer patients; view department dashboard.
Administrator	Manage staff accounts and roles; view system-wide audit trail; access all reports and analytics; configure system settings; manage departments.
Hospital Manager	View department performance reports; monitor waiting times and no-show rates; review triage outcomes; assess notification delivery effectiveness.

4.2 Use Case Diagrams

4.2.1 System-Wide Use Case Diagram

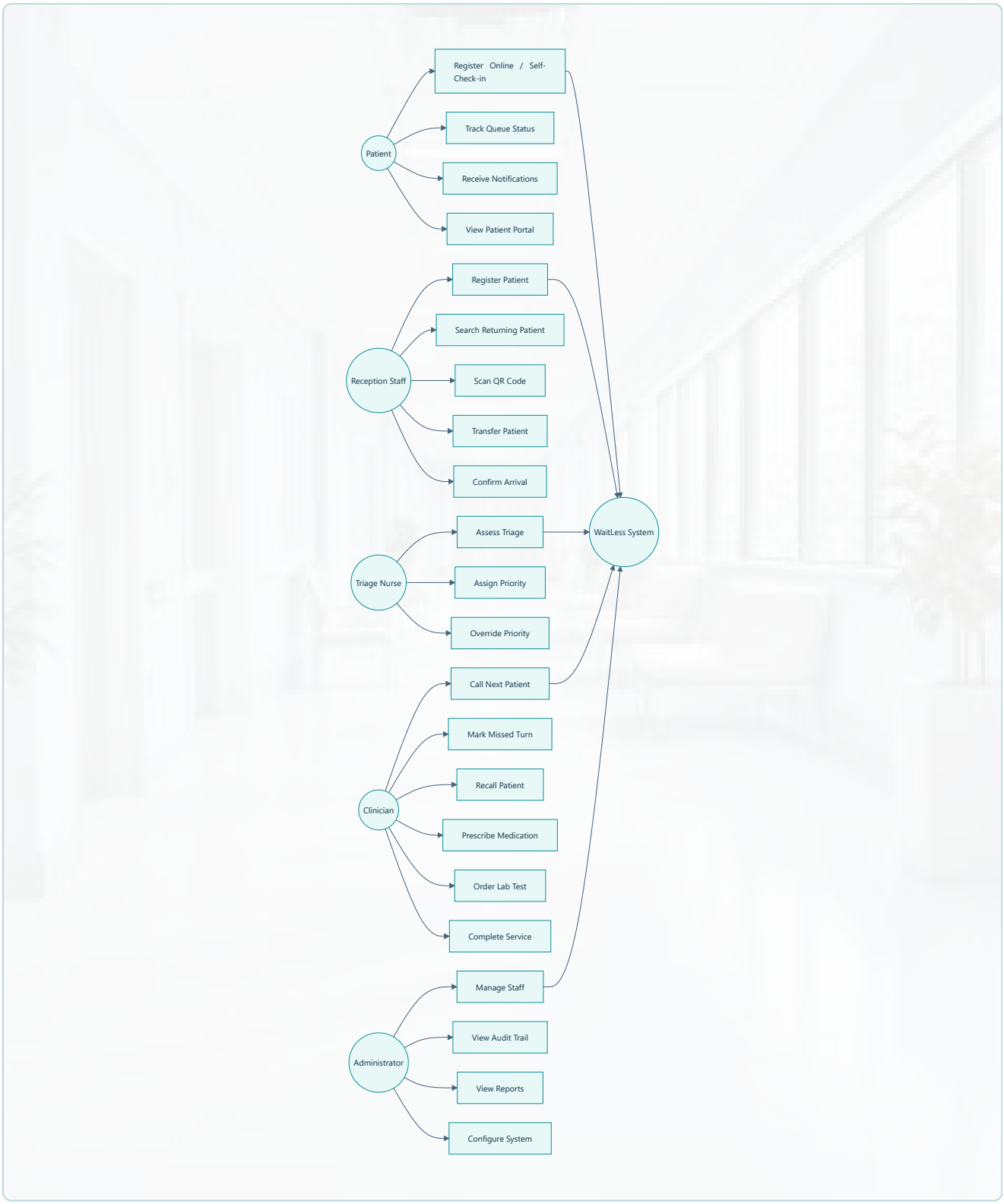


Figure 4.1: System-Wide Use Case Diagram

4.2.2 Patient Registration Use Case Diagram

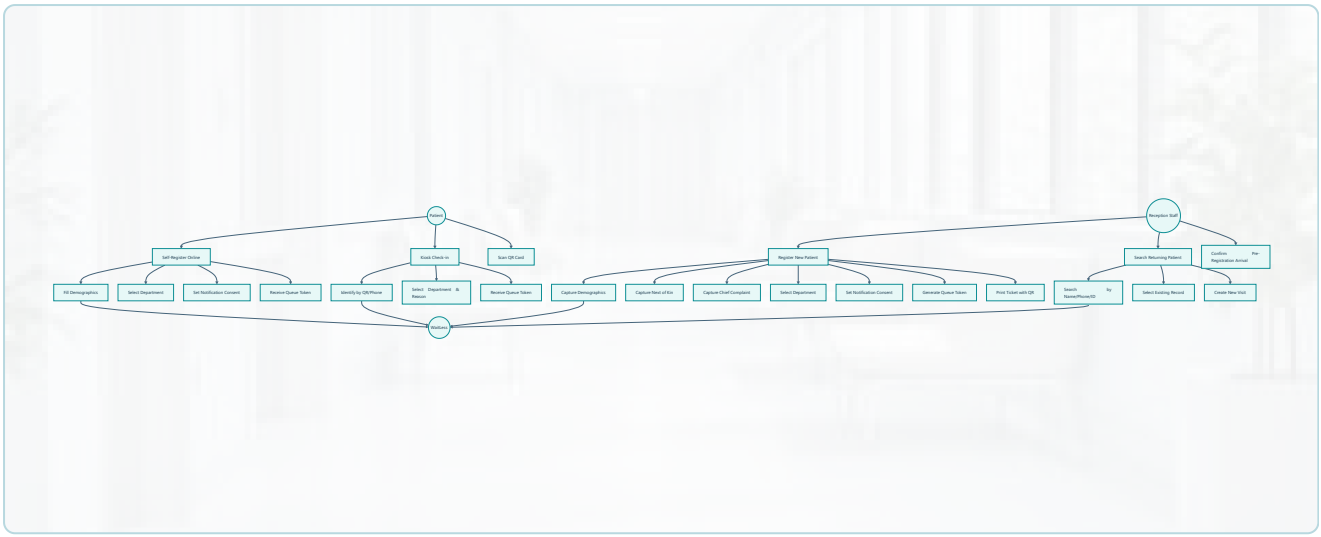


Figure 4.2: Patient Registration Use Case Diagram

4.2.3 Triage Use Case Diagram

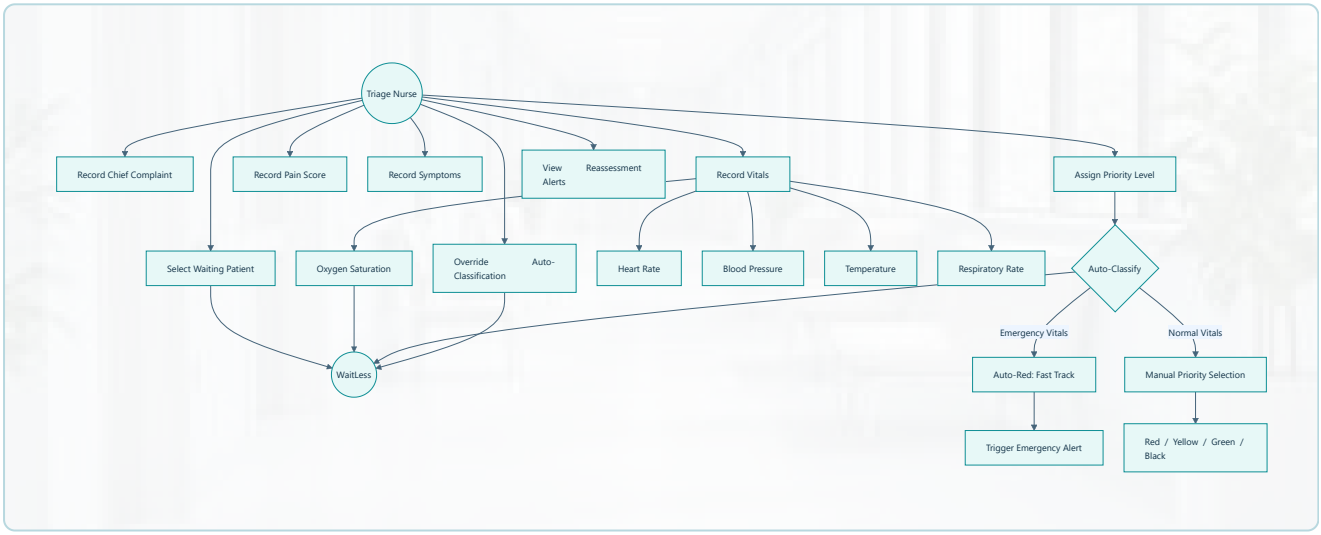


Figure 4.3: Triage Use Case Diagram

4.2.4 Queue Management Use Case Diagram

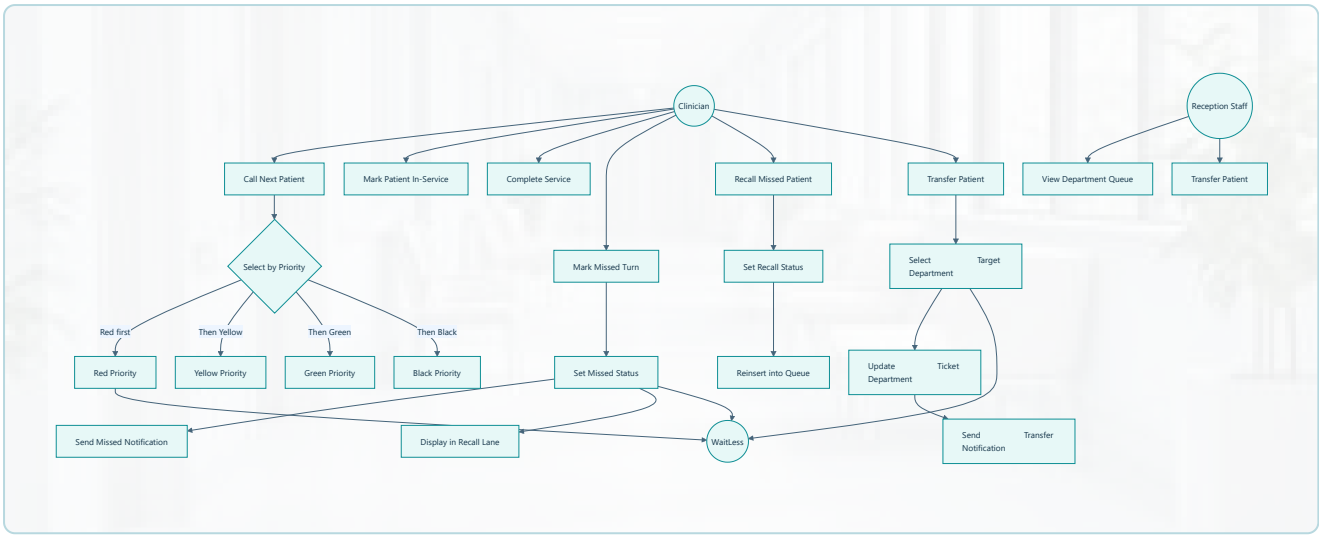


Figure 4.4: Queue Management Use Case Diagram

4.2.5 Notification Use Case Diagram

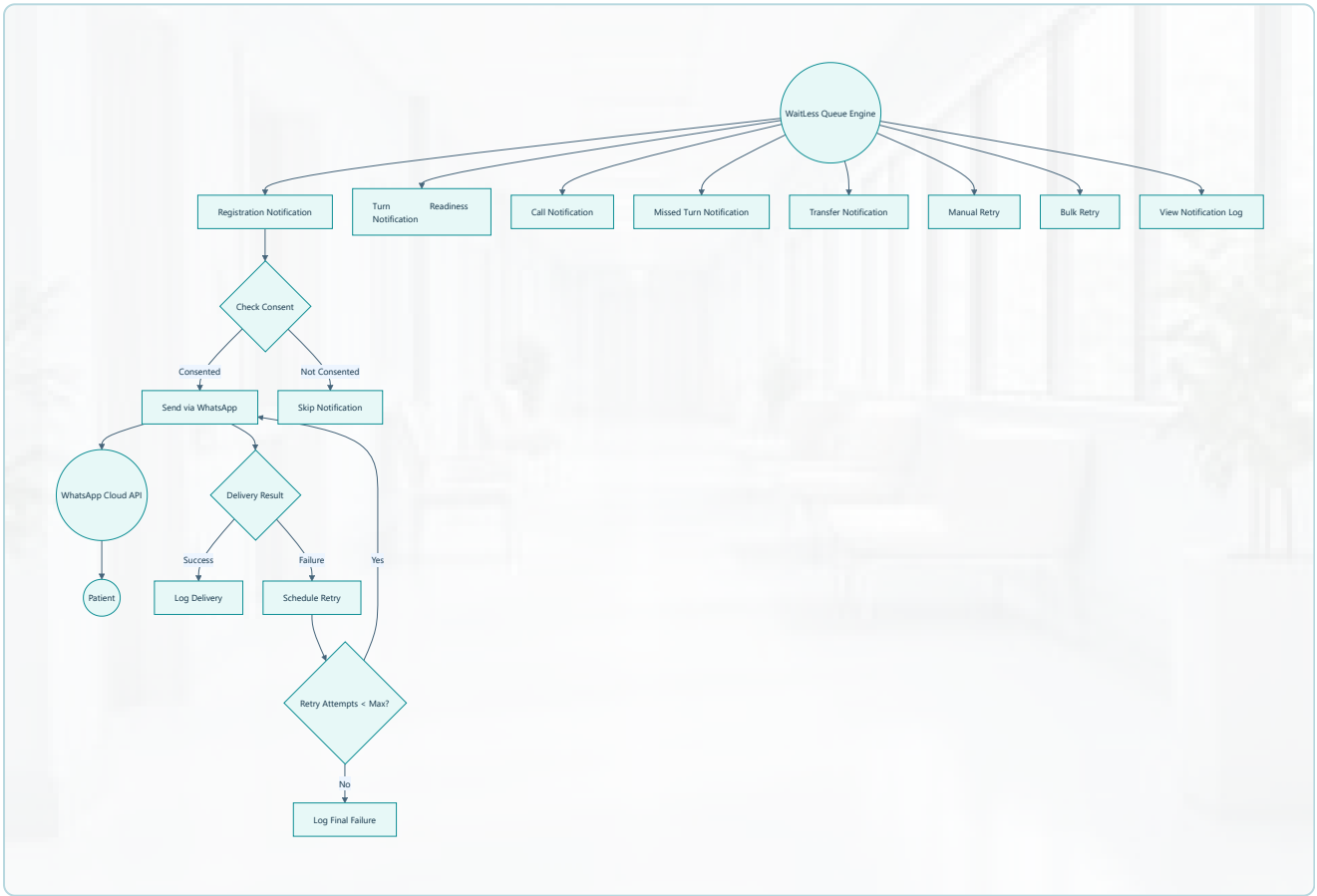


Figure 4.5: Notification Use Case Diagram

4.3 Use Case Descriptions

Use Case UC-01: Register New Patient

Table 4.16: Use Case UC-01: Register New Patient

Field	Description
Use Case ID	UC-01
Use Case Name	Register New Patient
Actor(s)	Reception Staff
Description	A reception staff member registers a new patient entering the hospital, capturing demographics, next of kin, chief complaint, and department selection. The system generates a unique queue token and prints a ticket.
Preconditions	Staff member is authenticated with reception role. Department queue is operational.
Postconditions	Patient is registered in the system. A queue ticket is created with a unique token. Patient appears in the department waiting queue.
Main Flow	1. Staff selects “Register Patient” from the dashboard. 2. Staff enters patient demographics (name, national ID, DOB, gender, phone, address). 3. Staff selects patient category (walk-in, referred, emergency, etc.). 4. Staff enters next of kin details. 5. Staff enters chief complaint. 6. Staff selects destination department. 7. Staff sets notification consent (WhatsApp opt-in). 8. Staff submits the form. 9. System validates inputs. 10. System creates or links a patient profile. 11. System generates a department-specific queue token (e.g., OPD-205). 12. System creates a ticket with status “waiting” and priority “green”. 13. System broadcasts a queue update event via SSE. 14. System sends a registration WhatsApp notification if consented. 15. System displays the ticket with QR code for printing.
Alternate Flows	**A1: Returning patient found.** At step 10, if a matching patient profile is found by name, phone, or national ID, the system links to the existing profile instead of creating a new one. **A2: Validation error.** At step 9, if required fields are missing or invalid, the system displays validation errors and the staff corrects the input.
Exceptions	**E1: Database unavailable.** If the database is unreachable, the system falls back to file-based storage (if configured) or displays an error message.
Business Rules	Queue tokens are immutable once generated. Token prefixes are department-specific (OPD, TRI, PHA, LAB, RAD, OIC). Sequence numbers are persistent and monotonic.
Frequency	High — performed many times daily.

Use Case UC-02: Triage Assessment

Table 4.17: Use Case UC-02: Triage Assessment

Field	Description
Use Case ID	UC-02
Use Case Name	Triage Assessment
Actor(s)	Triage Nurse
Description	A triage nurse conducts a structured assessment of a waiting patient, recording vitals, symptoms, pain score, and severity. The system auto-classifies priority based on emergency vital thresholds, with manual override capability.
Preconditions	Patient exists in the queue with status “waiting”. Triage nurse is authenticated with triage role.
Postconditions	Patient’s priority is updated. Triage assessment is recorded with assessor identity and timestamp. If Red priority, emergency fast-tracking is triggered. Queue order is updated.

Field	Description
Main Flow	1. Nurse selects a waiting patient from the triage queue. 2. Nurse records chief complaint (if not already captured). 3. Nurse records vitals: oxygen saturation, heart rate, blood pressure (systolic/diastolic), temperature, respiratory rate. 4. Nurse records pain score (0–10). 5. Nurse records symptoms and severity notes. 6. System evaluates vitals against emergency thresholds. 7. If any vital exceeds emergency thresholds, system auto-assigns Red priority. 8. If no emergency thresholds are exceeded, nurse manually selects priority (Red/Yellow/Green/Black). 9. System saves the triage assessment with assessor ID and timestamp. 10. System updates the patient's priority in the queue. 11. If Red priority, system triggers emergency fast-tracking (moves patient to front of queue). 12. System broadcasts a queue update event via SSE. 13. System records an audit event with before/after priority.
Alternate Flows	**A1: Manual override.** At step 8, the nurse may override an auto-Red classification if clinical judgement differs. The override is recorded with the original and new priority. **A2: Patient already triaged.** If the patient has a prior triage assessment, the nurse can update it, and both the original and updated values are retained.
Exceptions	**E1: Patient not found.** If the selected patient no longer exists in the queue, the system displays an error and returns to the triage list.
Business Rules	Emergency vital thresholds: SpO2 < 90%, HR outside 40–130 bpm, systolic BP outside 80–180 mmHg, temperature >= 39.5°C, respiratory rate outside 8–30/min, pain score >= 9. Priority order: Red (0) > Yellow (1) > Green (2) > Black (3).
Frequency	High — performed for every Casualty/Triage patient.

Use Case UC-03: Call Next Patient

Table 4.18: Use Case UC-03: Call Next Patient

Field	Description
Use Case ID	UC-03
Use Case Name	Call Next Patient
Actor(s)	Clinician
Description	A clinician calls the next patient in their department queue. The system selects the highest-priority, longest-waiting patient and updates their status to "called".
Preconditions	Clinician is authenticated. At least one patient is waiting in the department queue.
Postconditions	Selected patient's status is "called". Patient is removed from the waiting list. A call notification is sent if consented. Queue display boards update in real-time.
Main Flow	1. Clinician clicks "Call Next" on the dashboard. 2. System retrieves all waiting tickets for the clinician's department. 3. System sorts by priority rank (Red first), then by registration time (earliest first). 4. System selects the top ticket. 5. System updates the ticket status to "called" and records the called_at timestamp. 6. System broadcasts a "ticket.called" event via SSE. 7. System sends a WhatsApp call notification if the patient consented. 8. System dispatches turn-readiness notifications for remaining waiting patients. 9. System displays the called patient's details to the clinician.
Alternate Flows	**A1: No waiting patients.** If no patients are waiting in the department, the system displays "No patients waiting" and no action is taken.
Exceptions	**E1: Concurrent call.** If two clinicians call next simultaneously, the system processes requests sequentially; the second clinician receives the next available patient.
Business Rules	Priority sorting: Red > Yellow > Green > Black. Within the same priority, earliest registration time is called first. Called patients have a configurable window before being marked missed.
Frequency	High — performed continuously during clinic hours.

Use Case UC-04: Missed Turn and Recall

Table 4.19: Use Case UC-04: Missed Turn and Recall

Field	Description
Use Case ID	UC-04
Use Case Name	Mark Missed Turn and Recall
Actor(s)	Clinician
Description	A clinician marks a called patient as missed when they do not present for service. The patient is moved to a recall lane. The clinician can later recall the patient, reinserting them into the queue.
Preconditions	Patient has status “called”.
Postconditions	Patient status is “missed”. Missed notification is sent. Patient appears in the recall lane. On recall, patient is reinserted into the waiting queue.
Main Flow	1. Clinician selects a called patient and clicks “Mark Missed”. 2. System updates the ticket status to “missed” and records the missed_at timestamp. 3. System broadcasts a “ticket.missed” event via SSE. 4. System sends a missed-turn WhatsApp notification if consented. 5. Patient appears in the missed/recall lane on the dashboard and display board. 6. When the patient returns, clinician clicks “Recall”. 7. System updates the ticket status to “waiting” and records the recalled_at timestamp. 8. System increments the recall count. 9. System reinserts the patient into the waiting queue. 10. System broadcasts a “ticket.recalled” event via SSE.
Alternate Flows	**A1: Configurable reinsertion.** The reinsertion position can be configured (e.g., after 3 patients) to balance fairness with flow.
Business Rules	Recall count is tracked per ticket. Multiple recalls are allowed. The registration time may be reset on recall depending on configuration.
Frequency	Medium — occurs when patients step away during their turn.

Use Case UC-05: Queue Transfer

Table 4.20: Use Case UC-05: Queue Transfer

Field	Description
Use Case ID	UC-05
Use Case Name	Transfer Patient Between Departments
Actor(s)	Clinician, Reception Staff
Description	A staff member transfers a patient from one department queue to another, recording the previous department and notifying the patient.
Preconditions	Patient exists in the queue with a non-completed status. Target department is valid.
Postconditions	Patient’s department is updated. Previous department is recorded. Transfer notification is sent. Patient appears in the target department’s waiting queue.
Main Flow	1. Staff selects a patient and clicks “Transfer”. 2. Staff selects the target department from the dropdown. 3. System validates the target department exists. 4. System records the previous department. 5. System updates the ticket’s department and sets transferred_at timestamp. 6. System broadcasts a “ticket.transferred” event via SSE. 7. System sends a transfer WhatsApp notification with the new department and directions. 8. System dispatches turn-readiness notifications for the updated queue.
Alternate Flows	None.
Exceptions	**E1: Invalid department.** If the target department is the same as the current department or does not exist, the system displays an error.
Business Rules	Only non-completed tickets can be transferred. The previous department is retained for audit and tracking purposes.

Field	Description
Frequency	Medium — occurs when patients need services in multiple departments.

Use Case UC-06: Patient Self-Tracking

Table 4.21: Use Case UC-06: Patient Self-Tracking

Field	Description
Use Case ID	UC-06
Use Case Name	Track Queue Status
Actor(s)	Patient
Description	A patient enters their queue token on the public tracking page to view their current queue position, estimated wait time, patients ahead, and notification history.
Preconditions	Patient has a valid queue token.
Postconditions	Patient views their current queue status. No state changes occur.
Main Flow	1. Patient navigates to the tracking page (/track). 2. Patient enters their queue token (e.g., OPD-205). 3. System looks up the ticket by normalised token code. 4. System retrieves the ticket's current status, department, priority, and position. 5. System calculates the estimated wait time based on historical service time and queue position. 6. System retrieves the patient's notification history. 7. System displays: token, department, current status, position in queue, patients ahead, estimated wait time, progress indicator, and department directions. 8. Page updates in real-time via SSE when queue state changes.
Alternate Flows	**A1: Token not found.** If the token does not match any ticket, the system displays "Ticket not found" and prompts re-entry.
Exceptions	None.
Business Rules	Tracking is public and does not require authentication. Only the token code is needed. Patient name is not displayed on the tracking page for privacy.
Frequency	High — patients check frequently during their wait.

Use Case UC-07: Kiosk Self-Check-in

Table 4.22: Use Case UC-07: Kiosk Self-Check-in

Field	Description
Use Case ID	UC-07
Use Case Name	Kiosk Self-Check-in
Actor(s)	Patient
Description	A patient uses a self-service kiosk to check in by scanning a QR card or entering a phone number, selecting a department and visit reason, and receiving a queue token.
Preconditions	Kiosk device is operational and connected to the network.
Postconditions	Patient is checked in. A queue ticket is created. Patient receives a token and optional WhatsApp notifications.

Field	Description
Main Flow	1. Patient approaches the kiosk and taps “Check In”. 2. Patient identifies themselves by scanning a QR card (camera-based) or entering a phone number. 3. System searches for an existing patient profile by QR payload or phone. 4. If found, system displays the patient’s name for confirmation. 5. If not found, patient enters basic demographics. 6. Patient selects the destination department. 7. Patient enters a visit reason / chief complaint. 8. Patient sets notification consent. 9. System creates a queue ticket with the next sequence number. 10. System displays the queue token and estimated wait time. 11. System broadcasts a queue update event via SSE. 12. System sends a registration notification if consented.
Alternate Flows	**A1: QR scan failure.** If the camera cannot read the QR code, the patient can manually enter their phone number or patient number.
Exceptions	**E1: Network error.** If the kiosk cannot reach the server, an error message is displayed and the patient is directed to reception.
Business Rules	Kiosk check-in creates a “waiting” ticket with “green” priority by default. Triage assessment can upgrade the priority later.
Frequency	Medium — used by walk-in patients at the hospital entrance.

Use Case UC-08: Send WhatsApp Notification

Table 4.23: Use Case UC-08: Send WhatsApp Notification

Field	Description
Use Case ID	UC-08
Use Case Name	Send WhatsApp Notification
Actor(s)	System (automated)
Description	The system sends a WhatsApp message to a patient for registration, turn readiness, call, missed turn, or transfer events, with automatic retry on failure.
Preconditions	Patient has notification consent enabled. Patient has a valid phone number. WhatsApp provider is configured (mock or Meta Cloud API).
Postconditions	Notification record is created with delivery status. If successful, message is delivered. If failed, retry is scheduled.
Main Flow	1. A queue event triggers notification dispatch (registration, call, missed, transfer, turn readiness). 2. System checks the patient’s notification consent. 3. System creates a notification record with type, channel, recipient, and message content. 4. System resolves the WhatsApp template for the notification type. 5. System sends the message via the configured provider (mock or Meta Cloud API). 6. If delivery succeeds, system records the provider message ID and sets status to “sent”/“delivered”. 7. If delivery fails, system records the error code and message, and schedules a retry based on configured retry delays (15s, 60s, 300s). 8. System creates a notification attempt record for each attempt. 9. Retry continues until max attempts is reached or delivery succeeds.
Alternate Flows	**A1: Manual retry.** Staff can manually retry a failed notification from the dashboard. **A2: Bulk retry.** Staff can retry multiple failed notifications simultaneously.
Exceptions	**E1: Provider unavailable.** If the WhatsApp API is unreachable, the notification is queued for retry. **E2: Invalid phone number.** If the phone number is invalid, the notification is marked as failed with an error code.
Business Rules	Retry delays: 15 seconds, 60 seconds, 300 seconds (configurable). Max attempts: configurable per notification. Mock provider simulates delivery with configurable delay and failure rate for development.
Frequency	High — triggered by every queue event for consented patients.

Use Case UC-09: View Reports

Table 4.24: Use Case UC-09: View Reports

Field	Description
Use Case ID	UC-09
Use Case Name	View Operational Reports
Actor(s)	Administrator, Clinician
Description	A staff member views operational reports including average waiting time, department performance, no-show analysis, triage outcomes, and WhatsApp status, with date range filtering.
Preconditions	Staff member is authenticated.
Postconditions	Report data is displayed. No state changes occur.
Main Flow	1. Staff navigates to the Reports page. 2. Staff selects a report tab (Overview, Average Wait Time, Department Performance, No-Show, Triage, WhatsApp Status). 3. Staff selects a date range (from/to). 4. System fetches report data from the API with date filters. 5. System displays the report with summary metrics, department breakdowns, and detailed tables.
Alternate Flows	None.
Exceptions	**E1: No data.** If no tickets exist in the selected date range, the system displays an empty state message.
Business Rules	Reports are read-only. Date filtering applies to ticket registration date. Department scoping applies to non-admin staff.
Frequency	Medium — used for daily/weekly operational review.

Use Case UC-10: View Audit Trail

Table 4.25: Use Case UC-10: View Audit Trail

Field	Description
Use Case ID	UC-10
Use Case Name	View Audit Trail
Actor(s)	Administrator
Description	An administrator views the immutable audit trail of all system actions, including actor, timestamp, action type, entity, before/after state, and hash chain verification.
Preconditions	Administrator is authenticated with admin role.
Postconditions	Audit events are displayed. No state changes occur (audit trail is read-only).
Main Flow	1. Administrator navigates to the Audit Trail page. 2. System retrieves audit events from the database, ordered by occurrence time (most recent first). 3. System displays each event with: timestamp, actor name/role/department, action, entity type/ID, outcome, and before/after state. 4. Administrator can filter by action type, actor, or date range. 5. System displays the hash chain verification status.
Alternate Flows	None.
Exceptions	**E1: Hash chain broken.** If a tamper event is detected, the system flags the compromised event(s) in the audit display.
Business Rules	Audit events are append-only and cannot be modified or deleted. Each event's hash incorporates the previous event's hash, creating a verifiable chain. Sensitive data (passwords, tokens) is redacted in audit records. Only administrators can view the audit trail.
Frequency	Low — used for security review and compliance.

4.4 System Architecture

4.4.1 High-Level Architecture

The WaitLess system follows a client-server architecture with a React single-page application frontend, a Node.js backend with a custom HTTP router, a dual-mode data layer (MySQL or file-based JSON), and integration with the WhatsApp Cloud API for external notifications.

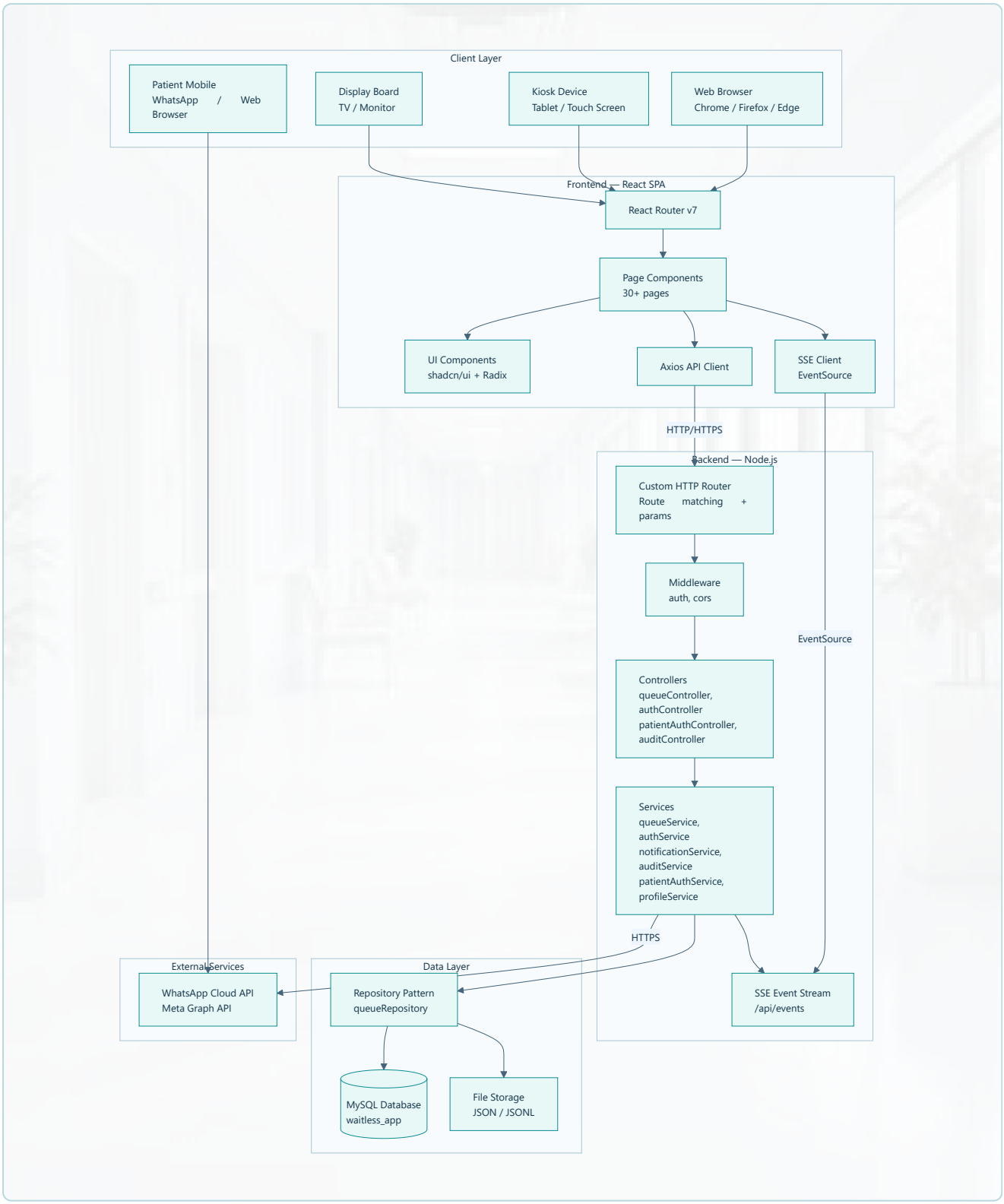


Figure 4.6: High-Level Architecture

4.4.2 Layered Architecture Description

The system follows a strict layered architecture with clear separation of concerns:

Table 4.26: Layered Architecture Description

Layer	Responsibility	Key Files
Presentation Layer	Renders UI, captures user input, displays real-time updates	`frontend/src/pages/`, `frontend/src/components/`
API/Controller Layer	Receives HTTP requests, validates input, calls services, formats responses	`backend/src/controllers/`, `backend/src/index.js`
Service/Business Logic Layer	Implements domain rules, queue algorithms, notification dispatch, authentication	`backend/src/services/`
Repository/Data Access Layer	Abstracts data persistence, supports MySQL and file-based modes	`backend/src/repositories/`
External Integration Layer	Communicates with WhatsApp Cloud API	`backend/src/services/whatsAppProvider.js`

4.4.3 Technology Architecture

Table 4.27: Technology Architecture

Component	Technology	Version
Frontend Framework	React	19.2.0
Build Tool	Vite	7.3.1
Routing	React Router DOM	7.10.1
Styling	Tailwind CSS	4.2.1
UI Components	Radix UI (shadcn/ui)	Various
Charts	Recharts	2.15.4
HTTP Client	Axios	1.7.9
Form Validation	Zod + react-hook-form	3.24.2 / 7.71.2
Icons	Lucide React	0.575.0
Toast Notifications	Sonner	2.0.7
QR Code	qrcode	1.5.4
QR Scanning	html5-qrcode	2.3.8
Backend Runtime	Node.js	18+ (ESM)
Database Driver	mysql2	3.6.0
Password Hashing	bcryptjs	2.4.3
Environment Config	dotenv	16.3.1
Real-time Protocol	Server-Sent Events (SSE)	Native
Notification Provider	WhatsApp Cloud API (Meta Graph)	v23.0

4.4.4 Deployment Architecture

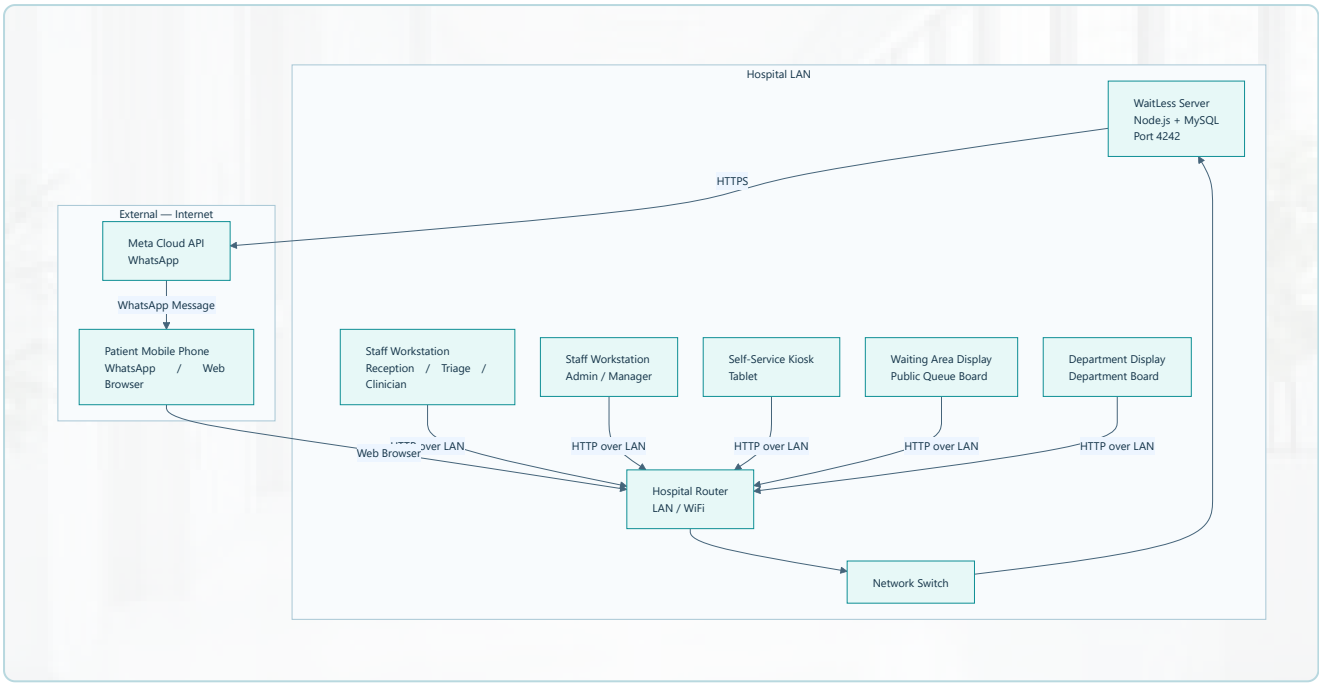


Figure 4.7: Deployment Architecture

The system is designed for primary deployment on a hospital local area network (LAN). The Node.js server and MySQL database run on a single machine or a small server within the hospital network. Staff workstations, kiosks, and display boards connect to the server over the LAN via standard HTTP. External WhatsApp notifications are sent via HTTPS to the Meta Cloud API, which requires internet connectivity. The system can operate in a reduced capacity (core queue management without notifications) when internet is unavailable.

4.5 Data Design

4.5.1 Entity-Relationship Diagram

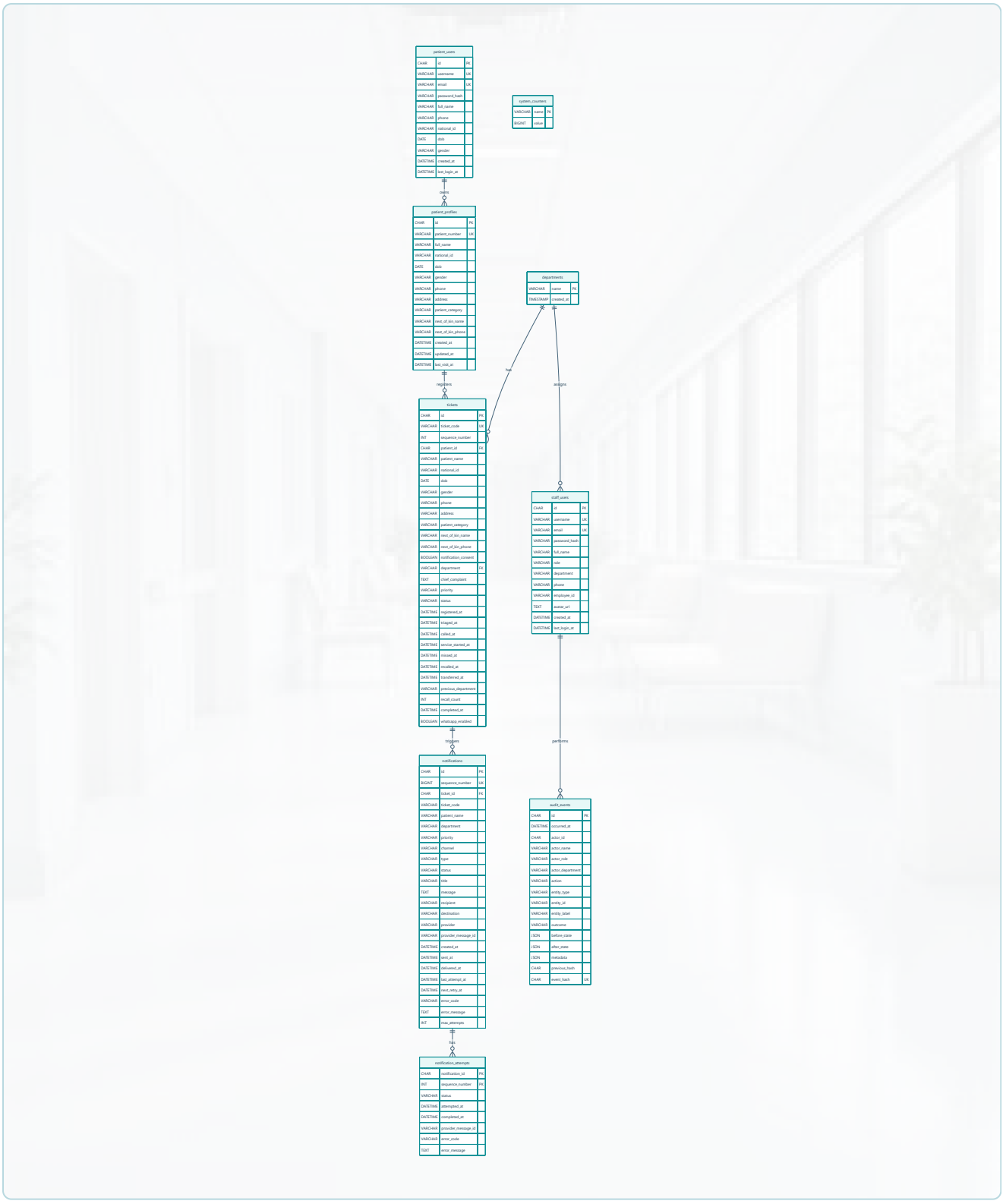


Figure 4.8: Entity-Relationship Diagram

4.5.2 Data Dictionary

Table: departments

Table 4.28: Table: departments

Column	Type	Constraints	Description
name	VARCHAR(80)	PRIMARY KEY	Department name (e.g., “OPD”, “Casualty / Triage”)
created_at	TIMESTAMP(3)	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

Table: patient_profiles

Table 4.29: Table: patientprofiles

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier
patient_number	VARCHAR(24)	NOT NULL, UNIQUE	Permanent patient number (e.g., “PT-0001”)
full_name	VARCHAR(160)	NOT NULL	Patient’s full name
national_id	VARCHAR(80)	NULL	National ID number
dob	DATE	NULL	Date of birth
gender	VARCHAR(40)	NOT NULL, DEFAULT ‘unknown’	Gender (male/female/unknown)
phone	VARCHAR(80)	NULL	Phone number for notifications
address	VARCHAR(255)	NOT NULL, DEFAULT ‘’	Residential address
patient_category	VARCHAR(80)	NOT NULL, DEFAULT ‘walk-in’	Category (walk-in, referred, emergency, etc.)
next_of_kin_name	VARCHAR(160)	NOT NULL, DEFAULT ‘’	Next of kin full name
next_of_kin_phone	VARCHAR(80)	NOT NULL, DEFAULT ‘’	Next of kin phone number
created_at	DATETIME(3)	NOT NULL	Profile creation timestamp
updated_at	DATETIME(3)	NOT NULL	Last update timestamp
last_visit_at	DATETIME(3)	NULL	Last visit timestamp

Indexes: idx_patient_profiles_name (full_name), idx_patient_profiles_national_id (national_id), idx_patient_profiles_phone (phone)

Table: tickets

Table 4.30: Table: tickets

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier
ticket_code	VARCHAR(20)	NOT NULL, UNIQUE	Queue token (e.g., “OPD-205”)
sequence_number	INT	NOT NULL	Department-specific sequence
patient_id	CHAR(36)	NULL, FK → patient_profiles(id)	Linked patient profile
patient_name	VARCHAR(160)	NOT NULL	Patient name (denormalised)
national_id	VARCHAR(80)	NOT NULL, DEFAULT ‘’	National ID (denormalised)
dob	DATE	NULL	Date of birth (denormalised)
gender	VARCHAR(40)	NOT NULL, DEFAULT ‘unknown’	Gender (denormalised)
phone	VARCHAR(80)	NOT NULL, DEFAULT ‘’	Phone (denormalised)

Column	Type	Constraints	Description
address	VARCHAR(255)	NOT NULL, DEFAULT ''	Address (denormalised)
patient_category	VARCHAR(80)	NOT NULL, DEFAULT 'walk-in'	Category (denormalised)
next_of_kin_name	VARCHAR(160)	NOT NULL, DEFAULT ''	Next of kin name (denormalised)
next_of_kin_phone	VARCHAR(80)	NOT NULL, DEFAULT ''	Next of kin phone (denormalised)
notification_consent	BOOLEAN	NOT NULL, DEFAULT FALSE	WhatsApp notification opt-in
department	VARCHAR(80)	NOT NULL, FK → departments(name)	Current department
chief_complaint	TEXT	NULL	Patient's chief complaint
priority	VARCHAR(20)	NOT NULL	Priority level (red/yellow/green/black)
status	VARCHAR(30)	NOT NULL	Ticket status (waiting/called/in-service/missed/completed)
registered_at	DATETIME(3)	NOT NULL	Registration timestamp
triaged_at	DATETIME(3)	NULL	Triage assessment timestamp
called_at	DATETIME(3)	NULL	When patient was called
service_started_at	DATETIME(3)	NULL	When service began
missed_at	DATETIME(3)	NULL	When marked missed
recalled_at	DATETIME(3)	NULL	When recalled
transferred_at	DATETIME(3)	NULL	When transferred
previous_department	VARCHAR(80)	NOT NULL, DEFAULT ''	Previous department (for transfers)
recall_count	INT	NOT NULL, DEFAULT 0	Number of recalls
completed_at	DATETIME(3)	NULL	Service completion timestamp
whatsapp_enabled	BOOLEAN	NOT NULL, DEFAULT FALSE	WhatsApp enabled flag

Indexes: idx_tickets_department_status (department, status), idx_tickets_priority_status (priority, status), idx_tickets_registered_at (registered_at), idx_tickets_patient_id (patient_id)

Table: notifications

Table 4.31: Table: notifications

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier
sequence_number	BIGINT	NOT NULL, UNIQUE	Monotonic sequence
ticket_id	CHAR(36)	NOT NULL, FK → tickets(id)	Related ticket
ticket_code	VARCHAR(20)	NOT NULL	Ticket code (denormalised)
patient_name	VARCHAR(160)	NOT NULL	Patient name (denormalised)
department	VARCHAR(80)	NOT NULL	Department (denormalised)
priority	VARCHAR(20)	NOT NULL	Priority (denormalised)
channel	VARCHAR(40)	NOT NULL	Notification channel (whatsapp, display-board)

Column	Type	Constraints	Description
type	VARCHAR(80)	NOT NULL	Notification type (queue.registered, queue.called, etc.)
status	VARCHAR(40)	NOT NULL	Delivery status (pending, sent, delivered, failed)
title	VARCHAR(255)	NOT NULL	Notification title
message	TEXT	NOT NULL	Message body
recipient	VARCHAR(160)	NOT NULL	Recipient phone number
destination	VARCHAR(160)	NULL	Destination department
provider	VARCHAR(80)	NULL	Provider name (mock, meta-cloud)
provider_message_id	VARCHAR(160)	NULL	Provider's message ID
created_at	DATETIME(3)	NOT NULL	Creation timestamp
sent_at	DATETIME(3)	NULL	When sent to provider
delivered_at	DATETIME(3)	NULL	When delivered
last_attempt_at	DATETIME(3)	NULL	Last attempt timestamp
next_retry_at	DATETIME(3)	NULL	Next scheduled retry
error_code	VARCHAR(80)	NULL	Error code from provider
error_message	TEXT	NULL	Error message from provider
max_attempts	INT	NOT NULL, DEFAULT 1	Maximum retry attempts

Indexes: idx_notifications_ticket_id, idx_notifications_status, idx_notifications_channel, idx_notifications_provider_message_id, idx_notifications_created_at

Table: notification_attempts

Table 4.32: Table: notificationattempts

Column	Type	Constraints	Description
notification_id	CHAR(36)	PK, FK → notifications(id)	Related notification
sequence_number	INT	PK	Attempt sequence
status	VARCHAR(40)	NOT NULL	Attempt status
attempted_at	DATETIME(3)	NOT NULL	Attempt timestamp
completed_at	DATETIME(3)	NULL	Completion timestamp
provider_message_id	VARCHAR(160)	NULL	Provider message ID
error_code	VARCHAR(80)	NULL	Error code
error_message	TEXT	NULL	Error message

Table: staff_users

Table 4.33: Table: staffusers

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier

Column	Type	Constraints	Description
username	VARCHAR(32)	NOT NULL, UNIQUE	Login username
email	VARCHAR(120)	NOT NULL, UNIQUE	Email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt password hash
full_name	VARCHAR(160)	NOT NULL	Staff member's full name
role	VARCHAR(40)	NOT NULL, DEFAULT 'reception'	Role (admin, reception, triage, clinician)
department	VARCHAR(80)	NOT NULL, DEFAULT ''	Assigned department
phone	VARCHAR(80)	NULL	Phone number
employee_id	VARCHAR(80)	NULL	Employee ID
avatar_url	TEXT	NULL	Avatar image URL
created_at	DATETIME(3)	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp
last_login_at	DATETIME(3)	NULL	Last login timestamp

Table: patient_users

Table 4.34: Table: patientusers

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier
username	VARCHAR(32)	NOT NULL, UNIQUE	Login username
email	VARCHAR(120)	NOT NULL, UNIQUE	Email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt password hash
full_name	VARCHAR(160)	NOT NULL	Patient's full name
phone	VARCHAR(80)	NOT NULL	Phone number
national_id	VARCHAR(80)	NULL	National ID
dob	DATE	NULL	Date of birth
gender	VARCHAR(40)	NOT NULL, DEFAULT 'unknown'	Gender
created_at	DATETIME(3)	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp
last_login_at	DATETIME(3)	NULL	Last login timestamp

Table: audit_events

Table 4.35: Table: auditevents

Column	Type	Constraints	Description
id	CHAR(36)	PRIMARY KEY	UUID identifier
occurred_at	DATETIME(3)	NOT NULL	When the event occurred
actor_id	CHAR(36)	NULL	Staff user ID who performed the action
actor_name	VARCHAR(160)	NULL	Staff name (denormalised)
actor_role	VARCHAR(40)	NULL	Staff role (denormalised)

Column	Type	Constraints	Description
actor_department	VARCHAR(80)	NULL	Staff department (denormalised)
action	VARCHAR(100)	NOT NULL	Action type (e.g., “ticket.create”, “auth.login”)
entity_type	VARCHAR(80)	NOT NULL	Entity type (ticket, staff, notification)
entity_id	VARCHAR(80)	NULL	Entity ID
entity_label	VARCHAR(160)	NULL	Entity label (e.g., ticket code)
outcome	VARCHAR(40)	NOT NULL, DEFAULT ‘success’	Outcome (success, failure)
before_state	JSON	NULL	State before the action
after_state	JSON	NULL	State after the action
metadata	JSON	NULL	Additional metadata
previous_hash	CHAR(64)	NULL	Hash of the previous audit event
event_hash	CHAR(64)	NOT NULL, UNIQUE	SHA-256 hash of this event

Indexes: idx_audit_events_occurred_at, idx_audit_events_actor_id, idx_audit_events_action, idx_audit_events_entity

Table: system_counters

Table 4.36: Table: systemcounters

Column	Type	Constraints	Description
name	VARCHAR(80)	PRIMARY KEY	Counter name (e.g., “OPD”, “TRI”, “PHA”)
value	BIGINT	NOT NULL	Current counter value

4.5.3 Data Flow Diagrams

DFD Level 0 — Context Diagram

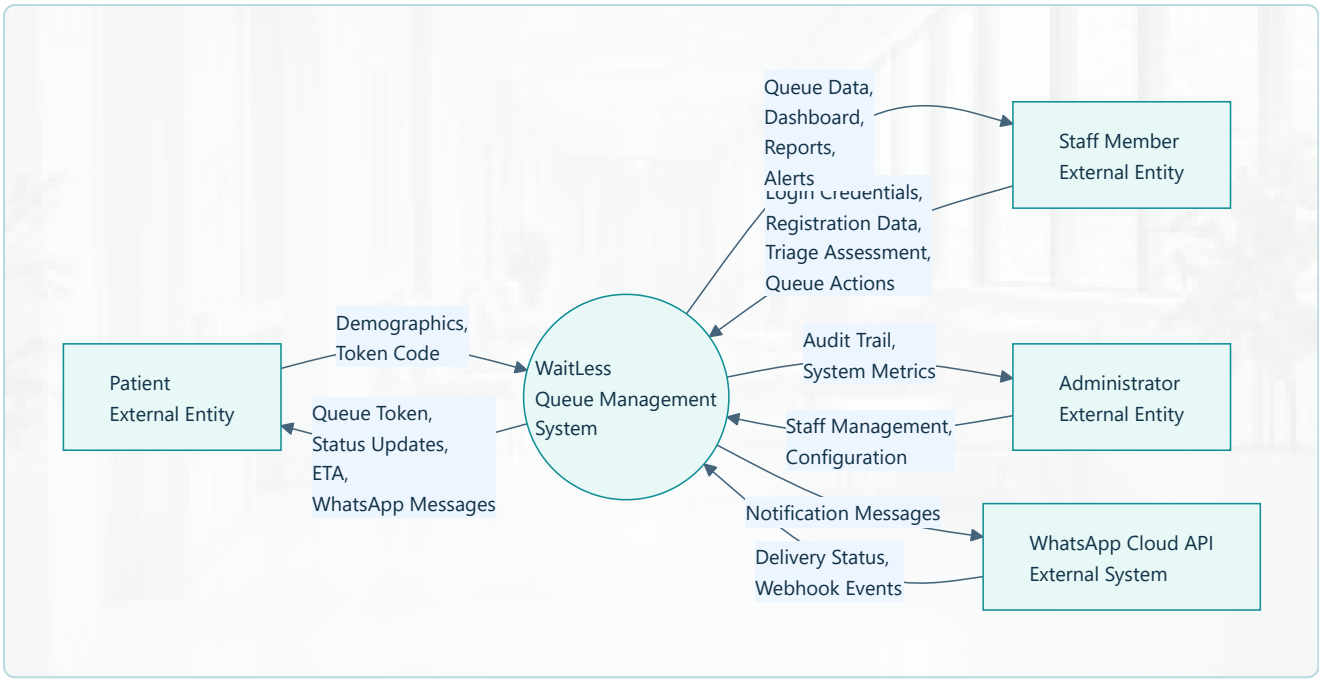


Figure 4.9: DFD Level 0 — Context Diagram

DFD Level 1 — Major Processes

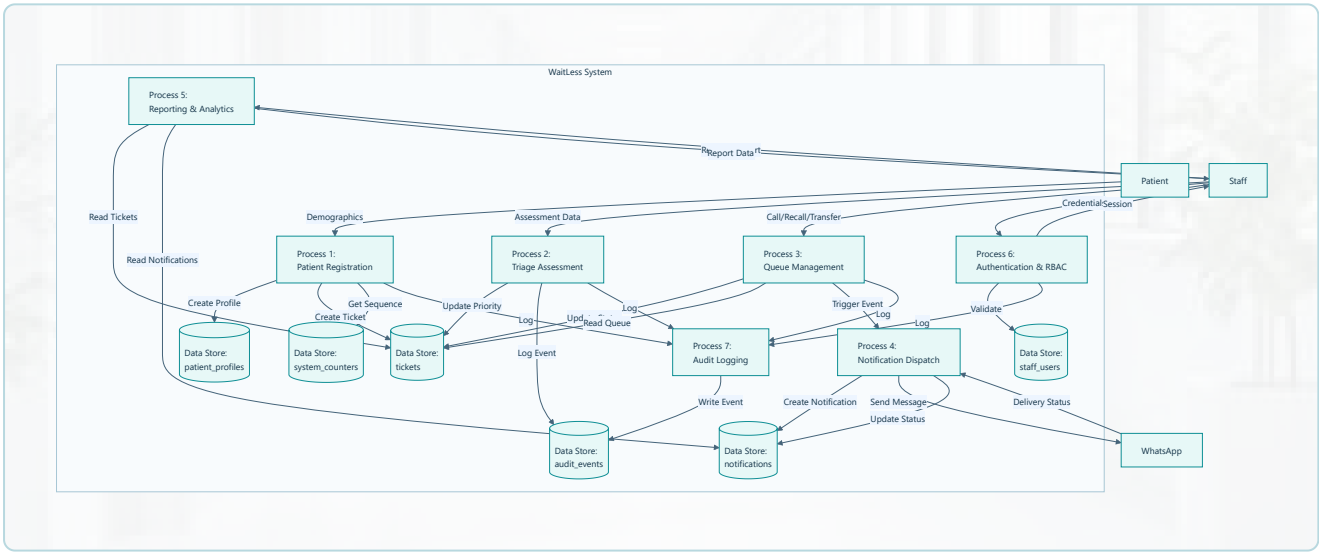


Figure 4.10: DFD Level 1 — Major Processes

DFD Level 2 — Patient Registration Detail

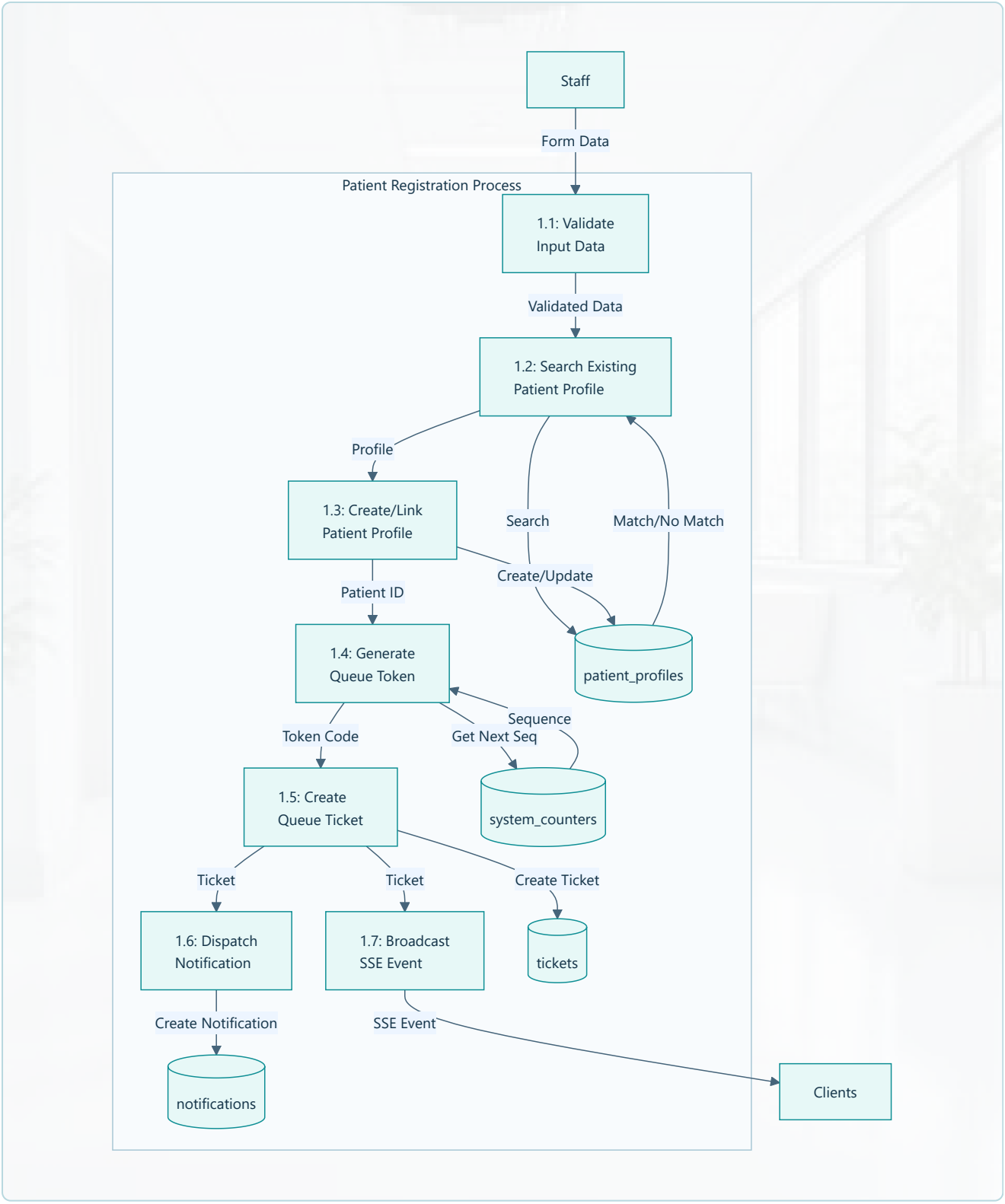


Figure 4.11: DFD Level 2 — Patient Registration Detail

4.6 Process Design

4.6.1 Sequence Diagram: Patient Registration

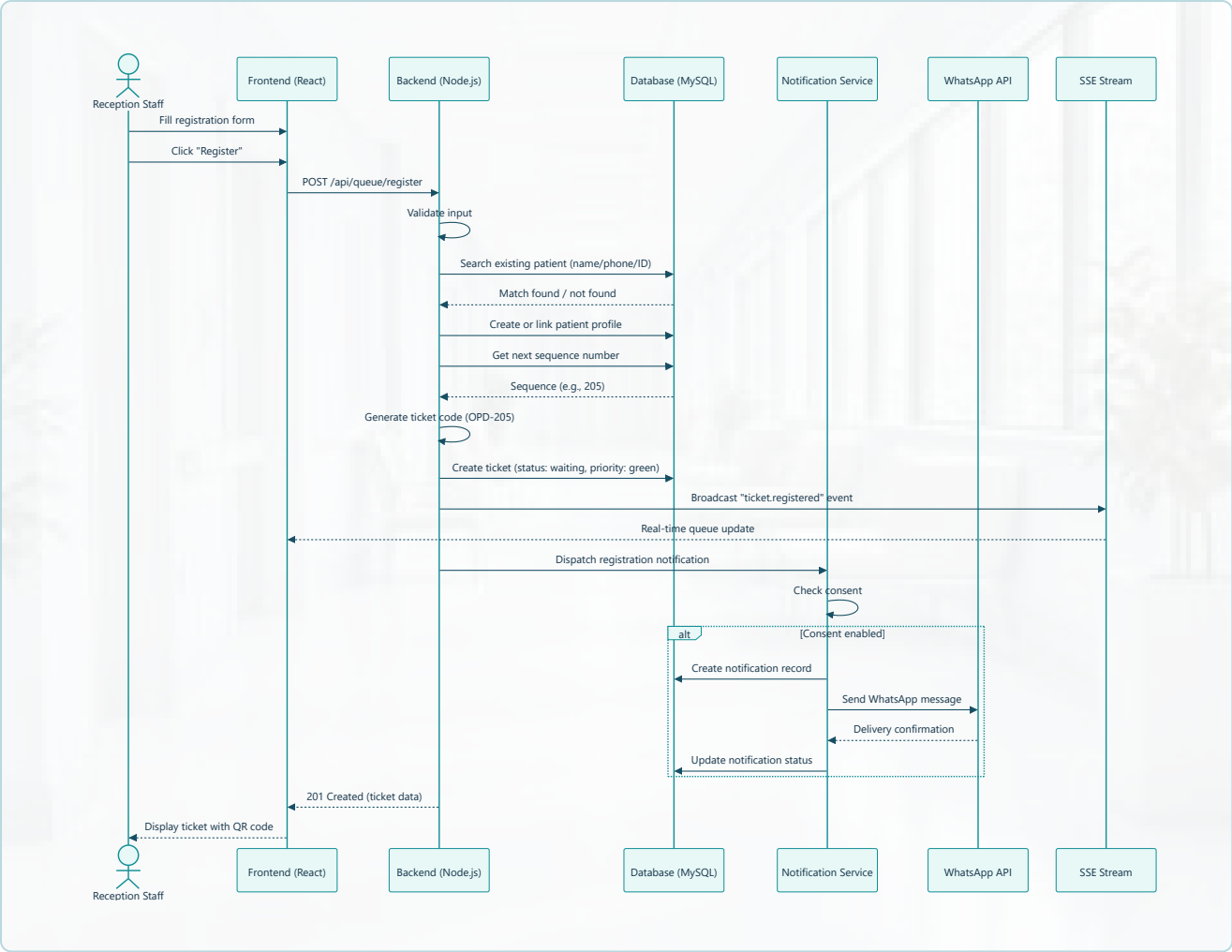


Figure 4.12: Sequence Diagram: Patient Registration

4.6.2 Sequence Diagram: Triage Assessment

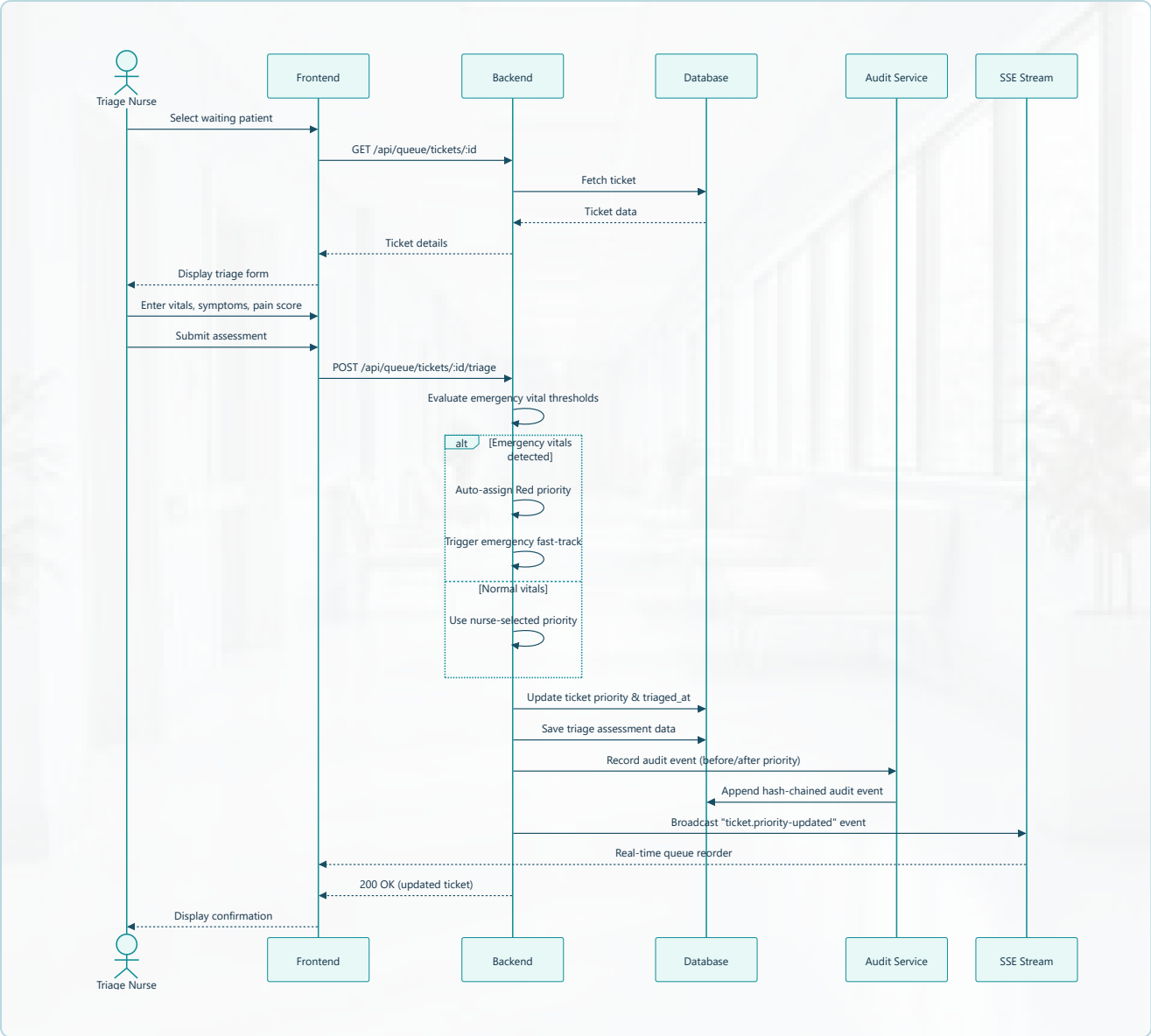


Figure 4.13: Sequence Diagram: Triage Assessment

4.6.3 Sequence Diagram: Call Next Patient

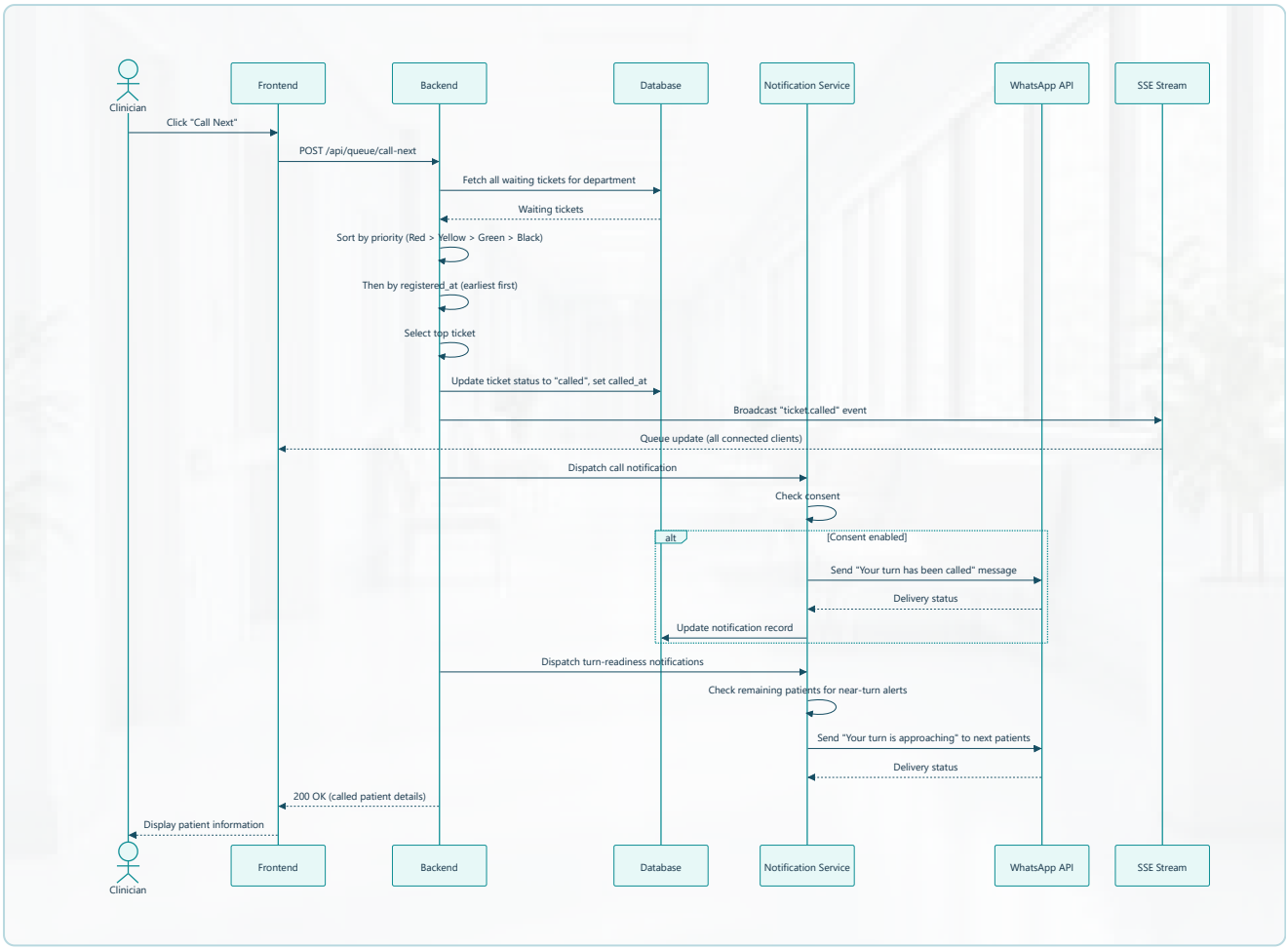


Figure 4.14: Sequence Diagram: Call Next Patient

4.6.4 Sequence Diagram: Notification Dispatch with Retry

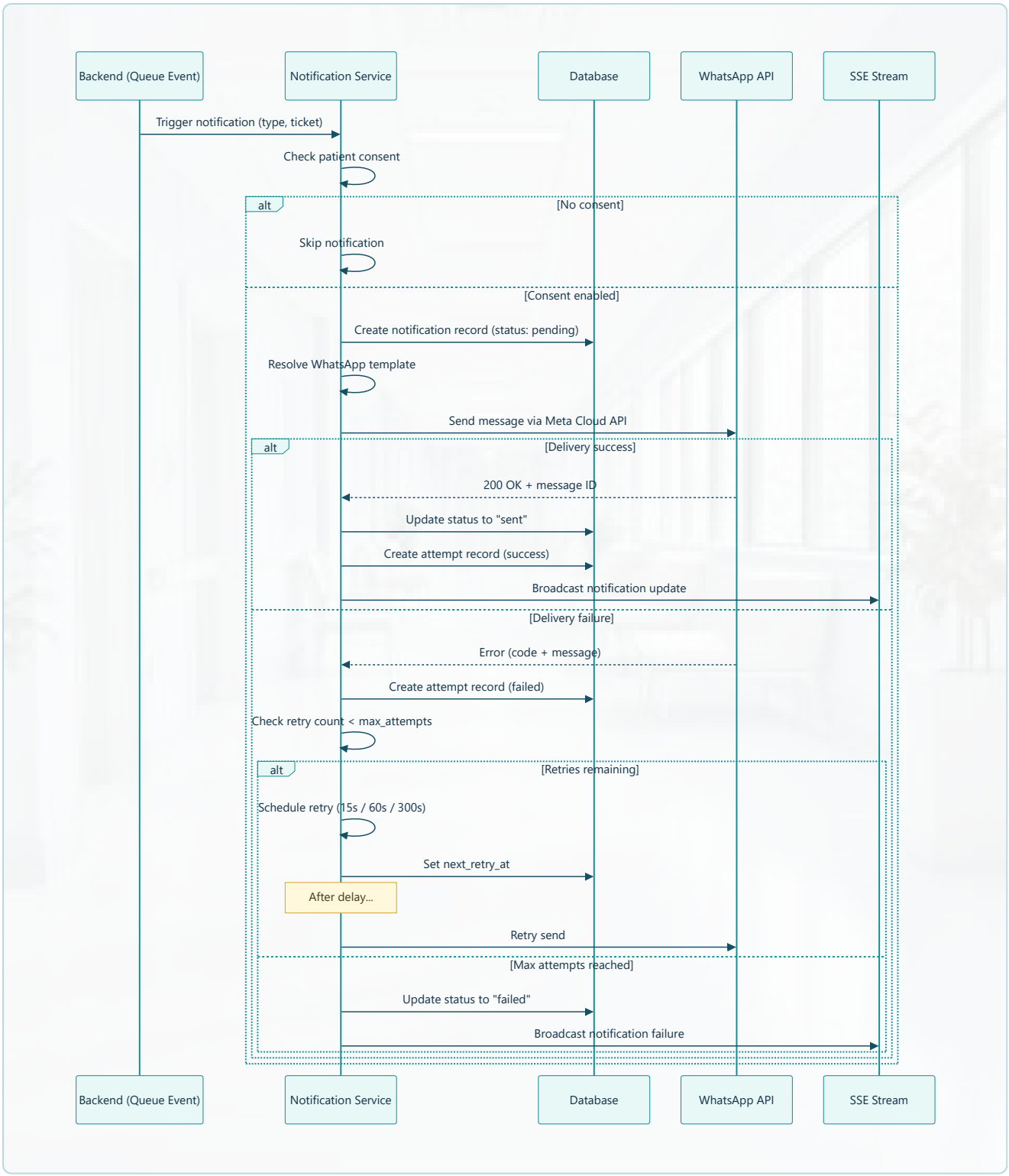


Figure 4.15: Sequence Diagram: Notification Dispatch with Retry

4.6.5 Activity Diagram: Patient Journey

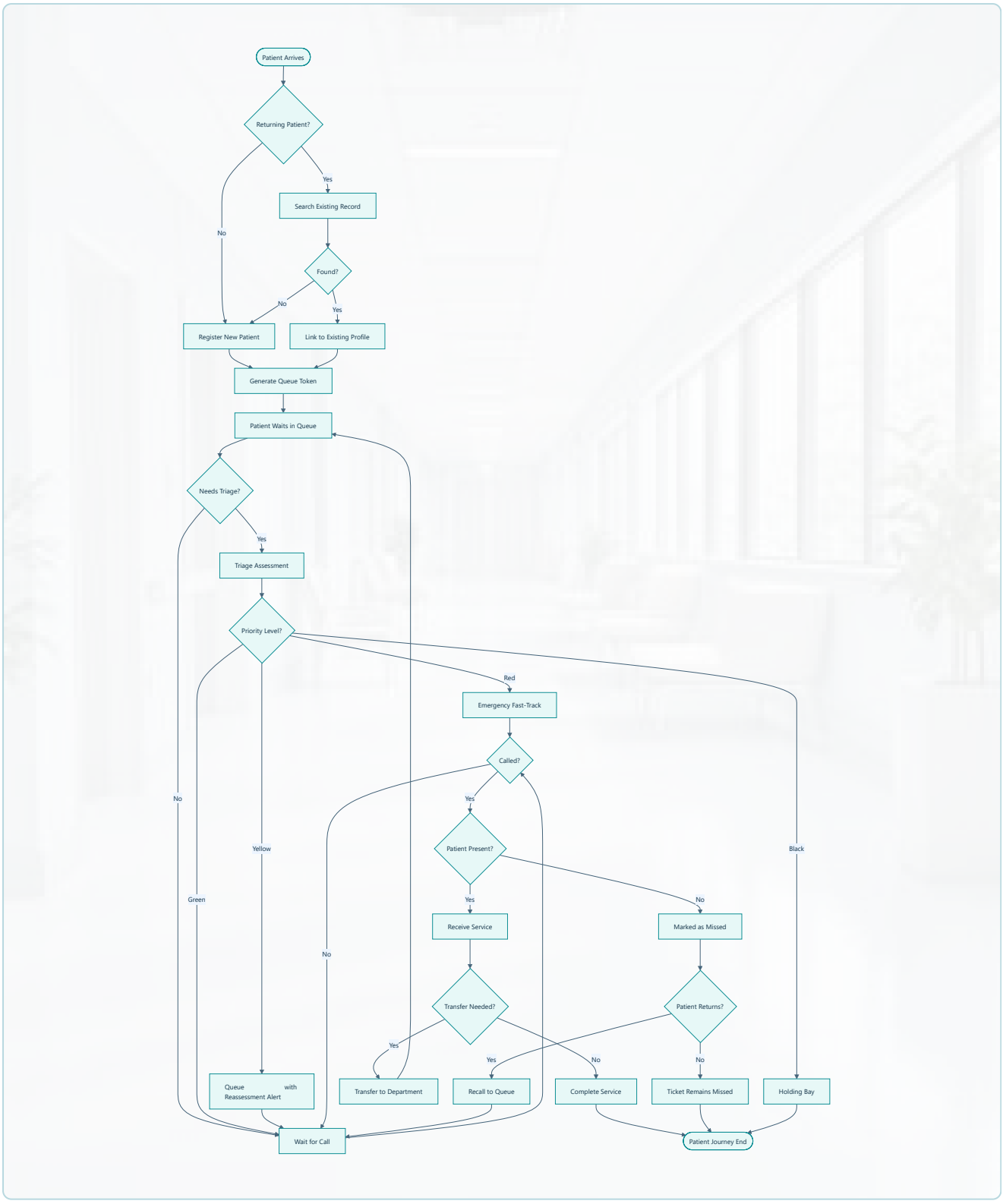


Figure 4.16: Activity Diagram: Patient Journey

4.6.6 Activity Diagram: Triage Workflow

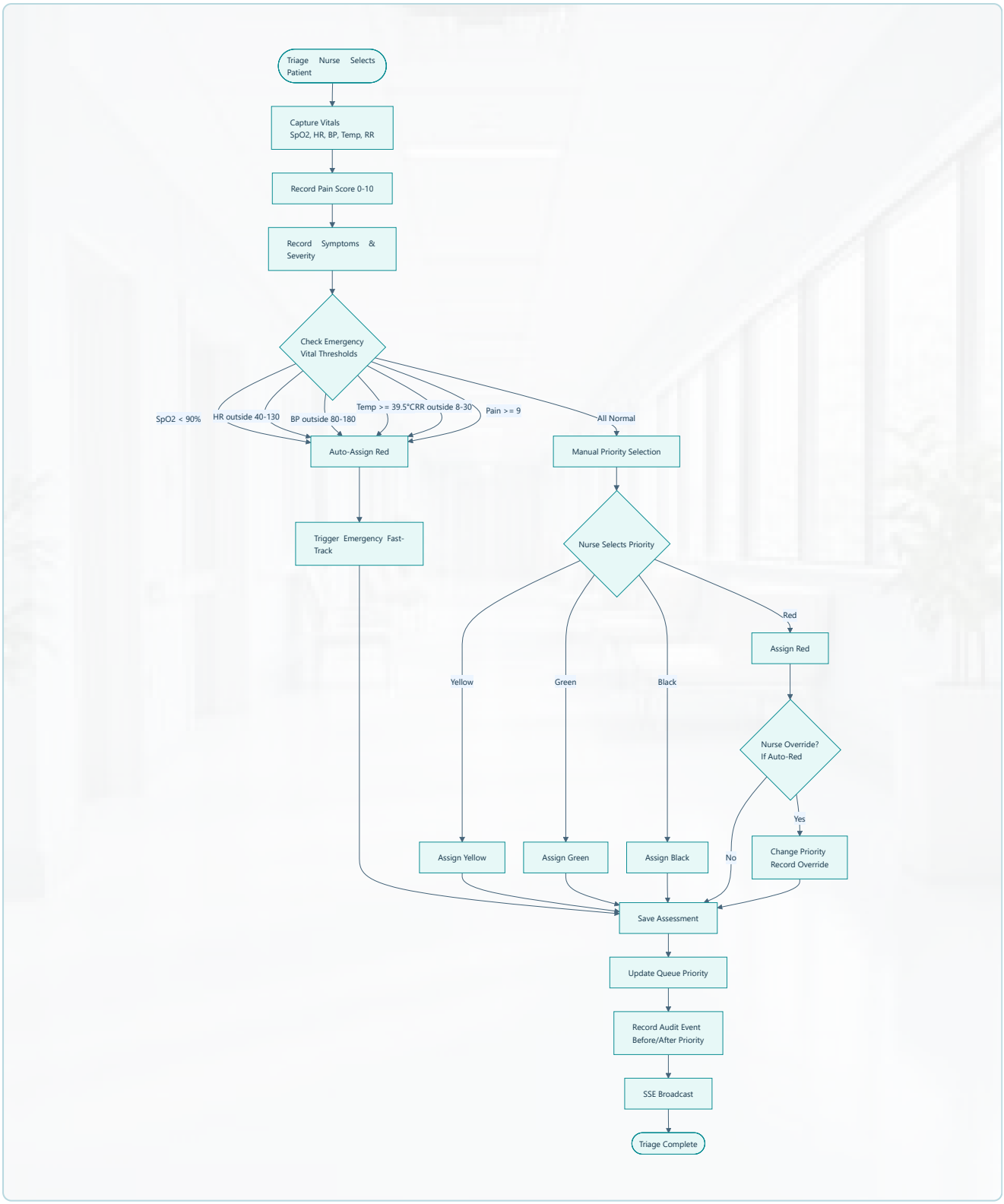


Figure 4.17: Activity Diagram: Triage Workflow

4.6.7 Activity Diagram: Notification Retry Lifecycle

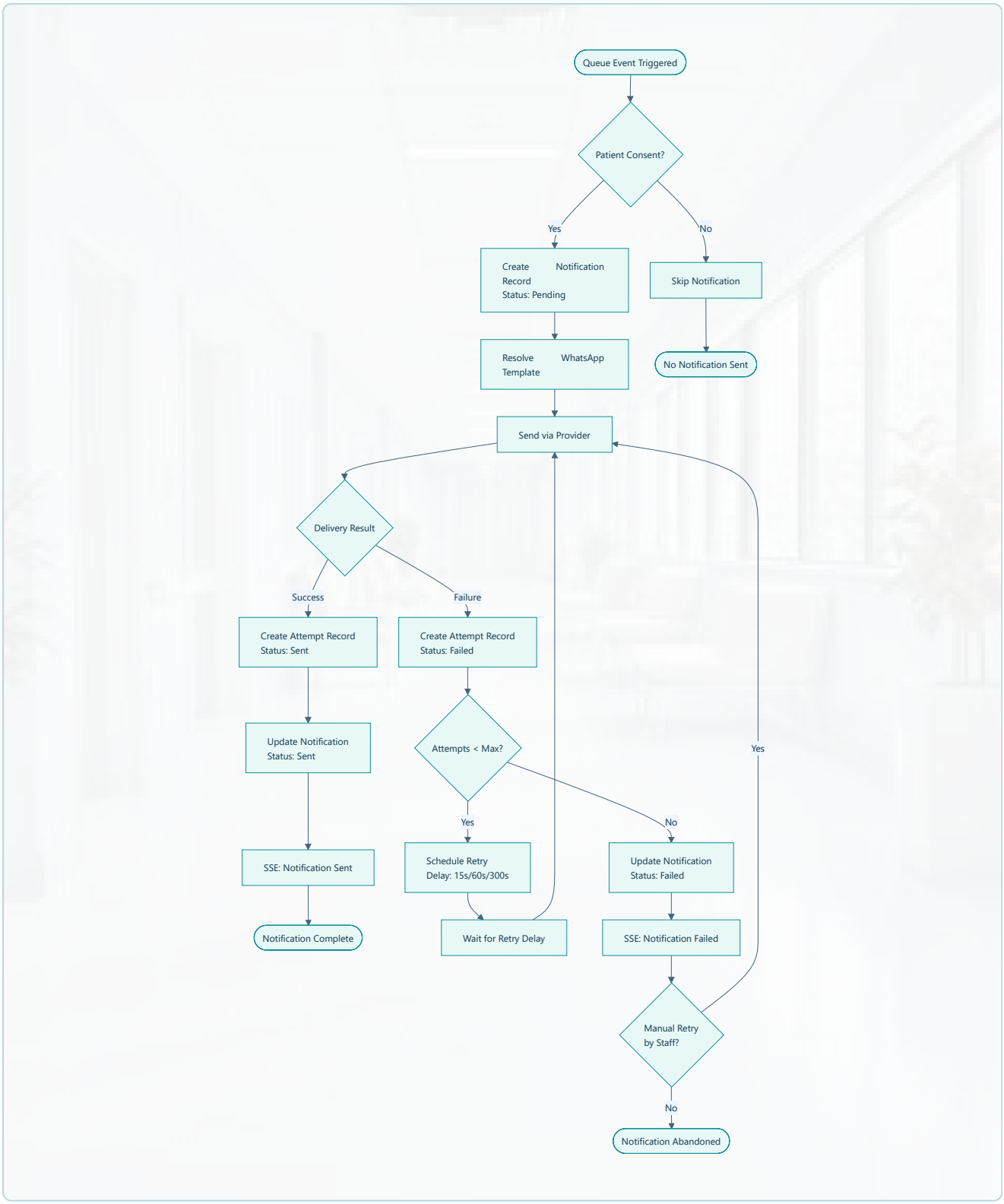


Figure 4.18: Activity Diagram: Notification Retry Lifecycle

4.7 User Interface Design

4.7.1 UI Design Principles

The WaitLess user interface is designed with the following principles:

1. **Medical Professional Aesthetic** — A dark teal (#0F766E) and navy colour scheme conveys clinical professionalism while reducing eye strain in hospital environments with varied lighting conditions.
2. **Glassmorphism Design** — Semi-transparent “glass” cards with backdrop blur create depth and visual hierarchy without overwhelming the user.
3. **Responsive Design** — The interface adapts to desktop monitors, tablets, and mobile phones using Tailwind CSS responsive breakpoints.
4. **Real-Time Feedback** — All queue updates are reflected instantly via Server-Sent Events, with visual indicators for connection status.
5. **Accessibility** — High contrast text, keyboard navigation, ARIA labels, and semantic HTML ensure the system is usable by staff with varying abilities.
6. **Role-Based Navigation** — Staff see only the navigation items and pages relevant to their role, reducing cognitive load.
7. **Privacy by Design** — Public displays show only token numbers, never patient names. Patient-facing pages minimise exposed personal information.

4.7.2 Colour Palette

Table 4.37: Colour Palette

Purpose	Colour	Hex
Primary (Medical Teal)	Teal	#0F766E
Secondary (Hospital Blue)	Blue	#2563EB
Priority Red	Red	#DC2626
Priority Yellow	Amber	#F59E0B
Priority Green	Emerald	#10B981
Priority Black	Dark Grey	#1F2937
Background (Dark)	Navy/Black	#0A0F1A
Surface (Glass)	Semi-transparent	rgba(255,255,255,0.05)
Text Primary	White	#F8FAFC
Text Muted	Slate	#94A3B8

4.7.3 Typography

Table 4.38: Typography

Element	Font	Size	Weight
Headings	System sans-serif	1.5rem – 2.25rem	700
Body text	System sans-serif	0.875rem – 1rem	400
Labels	System sans-serif	0.75rem	500
Token numbers	Monospace	2rem – 4rem	700
Captions	System sans-serif	0.75rem	400

4.7.4 Navigation Structure

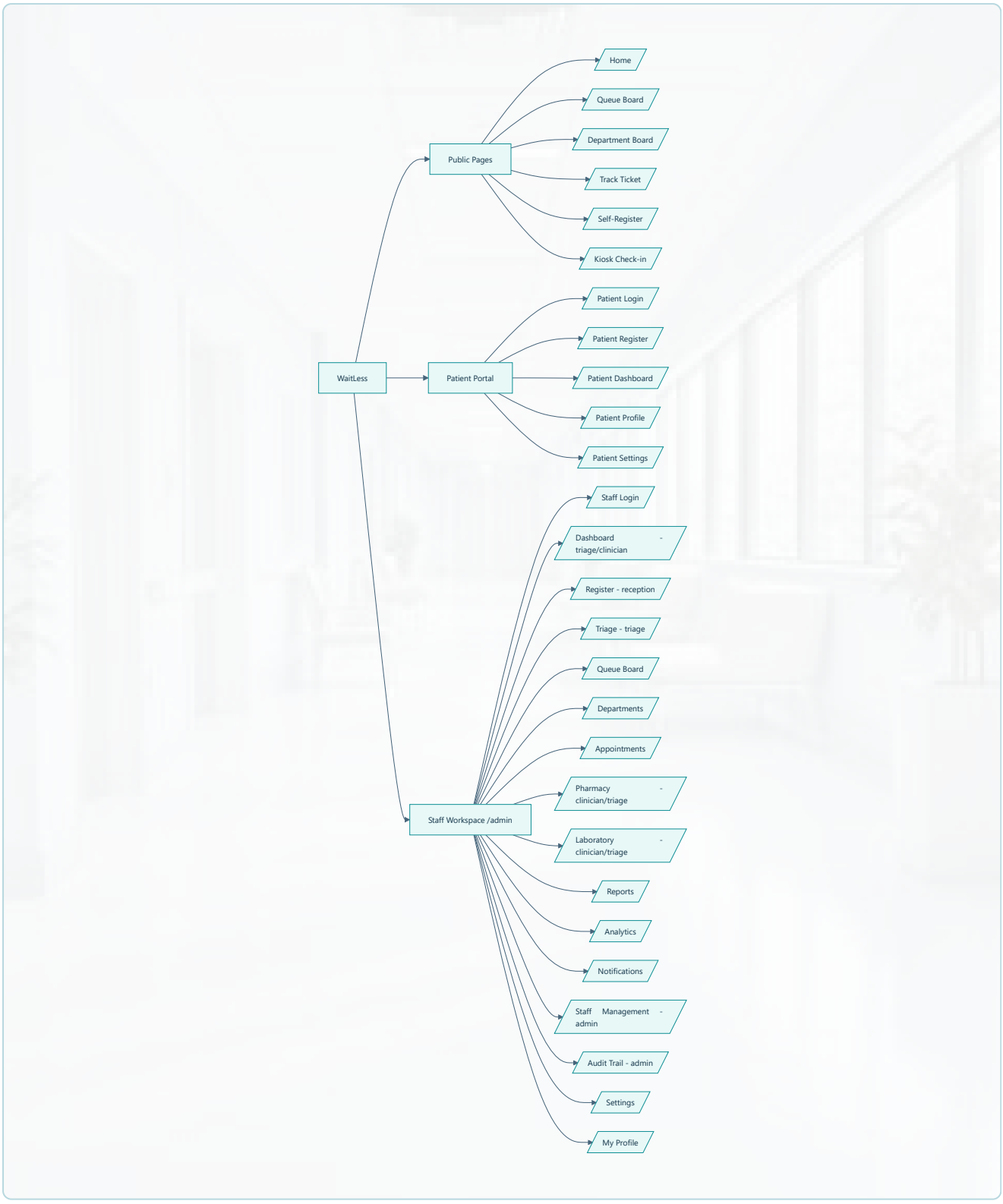


Figure 4.19: Navigation Structure

4.7.5 Key Page Descriptions

Landing Page (/)

- Hero section with WaitLess logo, tagline, and call-to-action buttons
- Live statistics display (patients served, current waiting, departments active)

- Feature highlights with icons
- Footer with hospital branding

Patient Registration (/admin/register)

- Multi-section form: demographics, next of kin, chief complaint, department selection
- Real-time form validation with inline error messages
- QR code scanner for returning patient lookup
- Patient category selector with visual icons
- Notification consent toggle
- On submit: ticket display with QR code, print button, and WhatsApp share

Triage Assessment (/admin/triage)

- Patient selector from waiting queue (sorted by priority)
- Vitals input form with range indicators and colour-coded warnings
- Pain score slider (0–10) with visual scale
- Symptoms and severity text areas
- Emergency vital threshold warnings (real-time as values are entered)
- Priority selection (auto-suggested based on vitals, manually overridable)
- Assessment history for previously triaged patients

Public Queue Board (/queue)

- Large-format display optimised for TV/monitor screens
- Three columns: “Now Serving”, “Waiting”, “Missed”
- Token numbers displayed in large monospace font
- Priority colour coding (red, yellow, green borders)
- Real-time updates via SSE (no page refresh)
- Department filter selector
- Animated transitions when tickets move between columns

Department Display Board (/department-board)

- Department-specific view (filtered by URL parameter)
- Shows only tickets for the selected department
- Larger token number display for waiting area visibility
- Real-time SSE updates
- Auto-refresh on connection loss

Staff Dashboard (/admin/dashboard)

- Live metrics cards: waiting count, in-service count, missed count, avg wait time
- Department-scoped queue table with sortable columns

- Patient action buttons: call next, mark missed, recall, transfer
- Operational alerts panel: reassessment alerts, notification failures, critical metrics
- Notification analytics with charts (delivery rate, retry trends)
- Real-time auto-refresh

Patient Tracking (/track)

- Token code input field with auto-lookup
- Progress indicator showing queue position
- Estimated wait time display
- Patients ahead count
- Department directions and current status
- Notification history timeline
- Real-time updates via SSE

Reports (/admin/reports)

- Tabbed interface: Overview, Average Wait Time, Department Performance, No-Show, Triage, WhatsApp Status
- Date range picker (from/to)
- Summary metric cards per report type
- Department breakdown tables
- Per-ticket notification correlation (no-show report)
- WhatsApp configuration checklist (status report)

Audit Trail (/admin/audit)

- Chronological list of audit events
- Filterable by action type, actor, date range
- Before/after state comparison view
- Hash chain verification indicator
- Actor details: name, role, department
- Redacted sensitive fields

4.8 Security Design

4.8.1 Authentication Model

The system implements two separate authentication flows:

Staff Authentication:

- Staff users log in with username/email and password
- Passwords are hashed using bcrypt with a cost factor of 12
- On successful login, a bearer token session is created (in-memory Map)
- Session token is included in the Authorization header for all API requests
- Sessions expire after 8 hours (configurable via `AUTH_SESSION_TTL_MS`)
- Password policy enforced: minimum 8 characters, at least 3 character groups (uppercase, lowercase, digits, special), common password rejection

Patient Authentication:

- Patient users register and log in with username/email and password
- Passwords are hashed using bcrypt
- Separate session management from staff authentication
- Patient sessions stored in localStorage with expiry tracking

4.8.2 Role-Based Access Control Matrix

Table 4.39: Role-Based Access Control Matrix

Resource/Action	Admin	Reception	Triage	Clinician	Patient	Anonymous
View public queue board	✓	✓	✓	✓	✓	✓
Track ticket	✓	✓	✓	✓	✓	✓
Self-register / kiosk	✓	✓	✓	✓	✓	✓
Register patient	✓	✓	✗	✗	✗	✗
Triage assessment	✓	✗	✓	✗	✗	✗
Call next patient	✓	✗	✓	✓	✗	✗
Mark missed/recall	✓	✗	✓	✓	✗	✗
Transfer patient	✓	✓	✓	✓	✗	✗
View dashboard	✓	✗	✓	✓	✗	✗
View reports	✓	✓	✓	✓	✗	✗
Manage staff	✓	✗	✗	✗	✗	✗
View audit trail	✓	✗	✗	✗	✗	✗
Configure settings	✓	✓	✓	✓	✗	✗
Prescribe medication	✓	✗	✓	✓	✗	✗
Order lab tests	✓	✗	✓	✓	✗	✗
View patient portal	✗	✗	✗	✗	✓	✗

4.8.3 Audit Trail Design

The audit trail provides a tamper-evident record of all system actions:

1. **Append-Only Storage** — Audit events are written to the `audit_events` table (MySQL) or `audit-events.jsonl` file. No update or delete operations are supported.
2. **SHA-256 Hash Chaining** — Each audit event includes:
   - `previous_hash`: The `event_hash` of the preceding event
   - `event_hash`: SHA-256 hash of the event's content + `previous_hash`

   This creates a cryptographic chain where modifying any event invalidates all subsequent hashes.
3. **Sensitive Data Redaction** — Passwords, session tokens, and other sensitive fields are redacted in the `before_state` and `after_state` JSON before hashing.
4. **Actor Attribution** — Every event records the actor's ID, name, role, and department, providing full accountability.
5. **Tracked Actions** — Login, staff creation, profile updates, ticket creation, triage assessment, priority changes, status updates, recalls, transfers, prescription and test order events, arrival confirmation, and notification retries.

4.8.4 Privacy Measures

Table 4.40: Privacy Measures

Measure	Implementation
Anonymous public displays	Queue board and department board show only token numbers, never patient names
Notification consent	Patients explicitly opt in to WhatsApp notifications during registration
Patient tracking privacy	Tracking page requires only a token code; no authentication needed, but no personal data is displayed
Audit redaction	Sensitive fields are redacted before storage in audit events
Department scoping	Non-admin staff see only their department's data in dashboards, tickets, and notifications

4.9 Implementation Plan and Milestones

Implementation followed an incremental sequence so that each stage left a demonstrable patient-flow path. The first milestone established registration and ticket creation. The second added triage and priority-aware queue movement. The third introduced staff dashboards, public boards, and real-time updates. The fourth expanded notifications, patient self-service, departmental workflows, reporting, and administration. The final milestone concentrated on security corrections, controlled walkthroughs, documentation, and packaging. The detailed research and development sequence appears in Section 3.10, while the remaining deployment controls are listed in Appendix C.

4.10 Technology Stack

4.10.1 Frontend Technologies

Table 4.41: Frontend Technologies

Technology	Version	Purpose
React	19.2.0	UI framework with hooks, concurrent rendering
Vite	7.3.1	Build tool and development server
React Router DOM	7.10.1	Client-side routing with nested routes
Tailwind CSS	4.2.1	Utility-first CSS framework
Radix UI	Various	Accessible, unstyled headless UI primitives
shadcn/ui	—	Component library built on Radix UI
Recharts	2.15.4	Charts and data visualisation
Axios	1.7.9	HTTP client with interceptors
Zod	3.24.2	Schema validation
react-hook-form	7.71.2	Performant form management
Lucide React	0.575.0	Icon library
Sonner	2.0.7	Toast notifications
qrcode	1.5.4	QR code generation
html5-qrcode	2.3.8	Camera-based QR code scanning
Lottie React	2.4.1	Animations

4.10.2 Backend Technologies

Table 4.42: Backend Technologies

Technology	Version	Purpose
Node.js	18+ (ESM)	JavaScript runtime
Custom HTTP Router	—	Route matching, parameter extraction, request handling
MySQL (mysql2)	3.6.0	Primary database driver
File-based JSON	—	Fallback storage mode
bcryptjs	2.4.3	Password hashing
dotenv	16.3.1	Environment variable management
Server-Sent Events	Native	Real-time queue updates

4.10.3 Development Tools

Table 4.43: Development Tools

Tool	Purpose
ESLint	Code linting and style enforcement
Prettier	Code formatting
Nodemon	Auto-restart backend during development
Git	Version control
Node.js native test runner	Backend unit testing

4.10.4 Architecture Decision Records

ADR-01: Custom HTTP Router instead of Express

The backend uses a lightweight custom HTTP router built on Node.js's native `http` module rather than the Express framework. This decision was made to:

- Minimise dependencies and reduce the attack surface
- Maintain full control over request parsing, route matching, and error handling
- Reduce bundle size and startup time
- Provide a learning opportunity in understanding HTTP server fundamentals

The custom router (`backend/src/utils/router.js`) supports:

- HTTP method-based route registration
- Path parameter extraction (e.g., `/api/tickets/:id`)
- Query string parsing via `URLSearchParams`
- JSON body parsing with size limits (3 MB max)
- Automatic trailing slash normalisation

ADR-02: Server-Sent Events instead of WebSocket

Real-time updates are delivered via Server-Sent Events (SSE) rather than WebSocket. Rationale:

- SSE is simpler to implement and requires no additional libraries
- Unidirectional communication (server-to-client) is sufficient for queue updates
- SSE works over standard HTTP, simplifying proxy and firewall configuration
- Built-in reconnection with configurable retry interval
- Lower overhead for the hospital LAN environment

The SSE implementation (`backend/src/sockets/queueEvents.js`) provides:

- Persistent connections via `text/event-stream` content type
- Heartbeat pings every 25 seconds to keep connections alive
- Configurable retry interval (3 seconds) for client reconnection
- Event broadcasting for all queue state changes

ADR-03: Dual-Mode Storage (MySQL + File)

The system supports both MySQL and file-based JSON storage, controlled by the `DB_PROVIDER` environment variable. This provides:

- Production deployment with MySQL for data integrity and concurrent access
- Development and testing with file-based storage for zero-configuration setup
- Deployment flexibility for hospitals with varying infrastructure

The repository pattern (`backend/src/repositories/queueRepository.js`) abstracts the storage mode, exposing a unified interface that both implementations satisfy.

4.11 System Structure

4.11.1 Project Directory Structure

```
WAITLESS/
├── backend/
│   ├── package.json
│   ├── .env
│   ├── data/                          # File-based storage (when DB_PROVIDER=file)
│   │   ├── queue-state.json
│   │   ├── audit-events.jsonl
│   │   └── staff-users.json
│   ├── test/                          # Backend unit tests
│   │   ├── audit-service.test.js
│   │   ├── returning-patient.test.js
│   │   └── staff-auth-security.test.js
│   └── src/
│       ├── index.js                   # Server entry, route registration, startup
│       ├── config/
│       │   └── env.js                 # Environment variable parsing
│       ├── controllers/
│       │   ├── auditController.js
│       │   ├── authController.js
│       │   ├── patientAuthController.js
│       │   ├── profileController.js
│       │   └── queueController.js
│       ├── services/
│       │   ├── auditService.js        # Hash-chained audit trail
│       │   ├── authService.js         # Staff authentication & RBAC
│       │   ├── avatarService.js       # Avatar image handling
│       │   ├── filePatientStore.js    # File-based patient storage
│       │   ├── fileStaffStore.js      # File-based staff storage
│       │   ├── notificationService.js # WhatsApp notification dispatch & retry
│       │   ├── patientAuthService.js  # Patient authentication
│       │   ├── passwordSecurity.js    # Password policy & hashing
│       │   ├── profileService.js      # Staff profile management
│       │   ├── queueService.js        # Core queue business logic (2234 lines)
│       │   ├── sessionService.js      # Session token management
│       │   └── whatsAppProvider.js    # WhatsApp Cloud API provider
│       ├── repositories/
│       │   ├── queueRepository.js     # Repository abstraction layer
│       │   ├── fileQueueRepository.js # File-based implementation
│       │   └── mysqlQueueRepository.js# MySQL implementation
│       ├── middleware/
│       │   ├── auth.js                # Staff authentication middleware
│       │   └── cors.js                # CORS handling
│       ├── modules/
│       │   └── queue/
│       │       └── queue.constants.js # Domain constants & seed data
│       ├── db/
│       │   ├── db.js                  # MySQL connection pool
│       │   ├── initDb.js              # Database initialisation
│       │   ├── schema.sql             # DDL for all tables
│       │   ├── seedAdmin.js           # Admin user seeding
│       │   ├── waitless_db.sql        # Full database export
│       │   └── waitless_tables.sql    # Table definitions
```

```
│       ├── sockets/
│       │   └── queueEvents.js         # SSE event stream server
│       └── utils/
│           ├── http.js                # HTTP utilities (errors, body parsing, JSON)
│           └── router.js              # Custom HTTP router
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
│       ├── main.jsx                   # React entry point
│       ├── App.jsx                    # Route definitions
│       ├── App.css
│       ├── index.css                  # Tailwind CSS imports
│       ├── styles.css                 # Global styles
│       ├── assets/                    # Images (BACKG.png, logos)
│       ├── auth/                      # Authentication context
│       │   ├── AuthContext.jsx
│       │   ├── AuthProvider.jsx
│       │   └── PatientAuthProvider.jsx
│       ├── components/                # Reusable UI components
│       │   ├── ui/                    # shadcn/ui components (40+ files)
│       │   ├── dashboard/
│       │   ├── AuthShell.jsx
│       │   ├── PatientAuthShell.jsx
│       │   ├── ProtectedRoute.jsx
│       │   ├── WaitLessLogo.jsx
│       │   └── ...
│       ├── context/
│       │   └── LiveRefreshContext.jsx # Global refresh trigger
│       ├── hooks/
│       │   ├── useApiResource.jsx     # Data fetching hook
│       │   └── use-mobile.jsx
│       ├── layouts/
│       │   ├── AppLayout.jsx          # Main layout (header, footer, routing)
│       │   ├── DashboardLayout.jsx
│       │   └── PatientPortalLayout.jsx
│       ├── pages/                     # Page components (30+ files)
│       │   ├── index.jsx              # Landing page
│       │   ├── register.jsx           # Staff patient registration
│       │   ├── triage.jsx             # Triage assessment
│       │   ├── queue.jsx              # Public queue board
│       │   ├── department-board.jsx   # Department-specific board
│       │   ├── track.jsx              # Patient tracking
│       │   ├── dashboard.jsx          # Staff dashboard
│       │   ├── staff-modules.jsx      # Staff sub-pages (reports, analytics, etc.)
│       │   ├── patient-dashboard.jsx  # Patient portal dashboard
│       │   ├── patient-login.jsx
│       │   ├── patient-register.jsx
│       │   ├── kiosk.jsx              # Self-service kiosk
│       │   ├── self-register.jsx      # Online self-registration
│       │   └── ...
│       ├── services/                  # API and data services
│       │   ├── api.js                 # Axios client configuration
│       │   ├── authApi.js
│       │   ├── authSession.js
│       │   ├── patientAuthApi.js
```

```
│       │   ├── patientAuthSession.js
│       │   ├── queueApi.js            # Queue API functions
│       │   ├── queueMeta.js           # Frontend metadata constants
│       │   ├── queueTicketHandoff.js  # Ticket handoff utilities
│       │   └── staffProfilePrefs.js
│       ├── sockets/
│       │   └── QueueRealtimeProvider.jsx  # SSE client
│       ├── store/
│       └── utils/
└── docs/
    └── world-class-system-roadmap.md
```

4.11.2 Backend Module Map

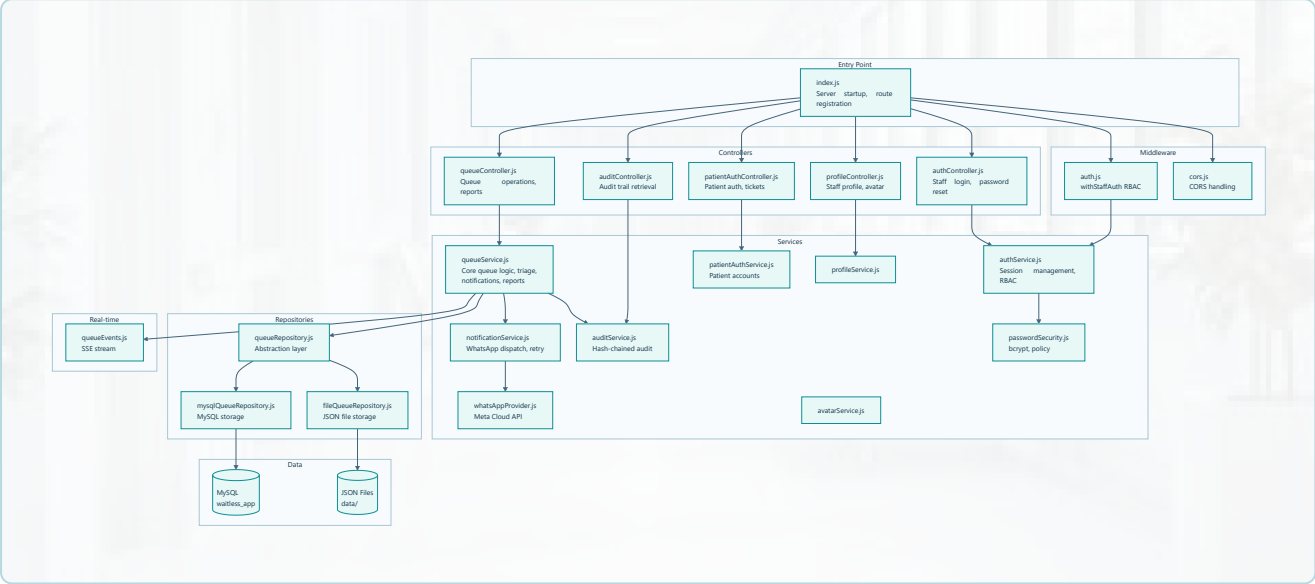


Figure 4.20: Backend Module Map

4.11.3 Frontend Component Hierarchy

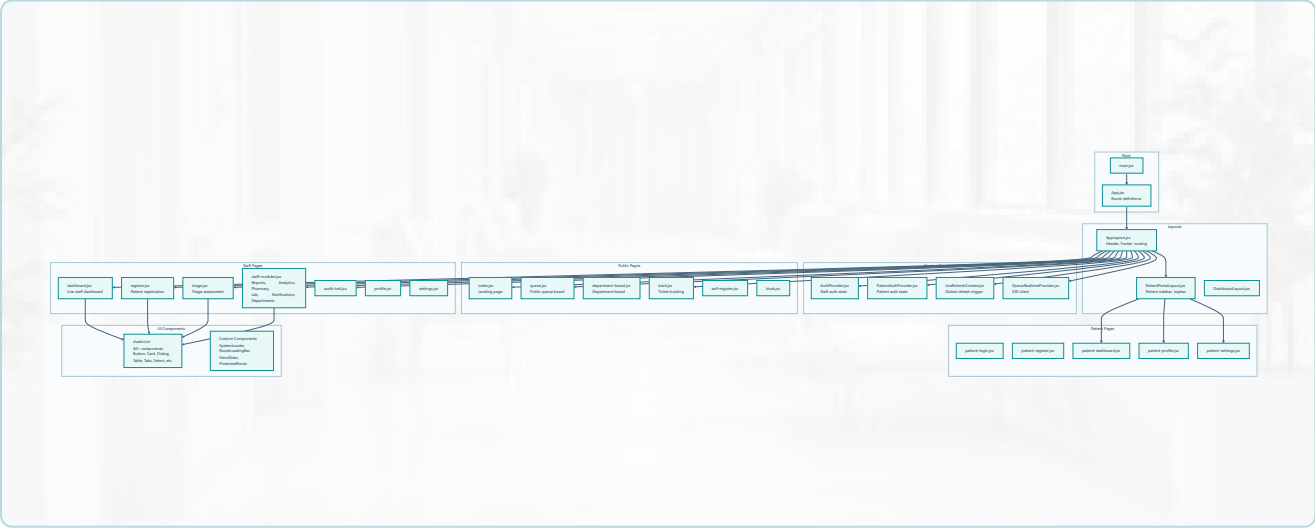


Figure 4.21: Frontend Component Hierarchy

4.12 Backend Implementation

4.12.1 Custom HTTP Router

The custom HTTP router is the backbone of the backend, handling all incoming HTTP requests. It is implemented in `backend/src/utils/router.js`.

Route Registration: Routes are registered with a method, URL pattern, and handler function. Patterns support path parameters using colon notation (e.g., `/api/tickets/:id`).

Request Handling: When a request arrives, the router iterates registered routes, matches by HTTP method and path pattern, extracts parameters, parses the JSON body (for POST/PATCH), and invokes the handler with a context object containing `{ req, res, url, params, query, body, rawBody }`.

Key code — route matching with parameter extraction:

```
function matchRoute(pattern, pathname) {
  const routeSegments = normalisePath(pattern).split("/").filter(Boolean);
  const pathSegments = normalisePath(pathname).split("/").filter(Boolean);

  if (routeSegments.length !== pathSegments.length) {
    return null;
  }

  const params = {};

  for (let index = 0; index < routeSegments.length; index += 1) {
    const routeSegment = routeSegments[index];
    const pathSegment = pathSegments[index];

    if (routeSegment.startsWith(":")) {
      params[routeSegment.slice(1)] = decodeURIComponent(pathSegment);
      continue;
    }

    if (routeSegment !== pathSegment) {
      return null;
    }
  }

  return params;
}
```

Request body handling (`backend/src/utils/http.js`):

- Bodies are read as UTF-8 strings with a 3 MB size limit
- JSON parsing with error handling (returns 400 on invalid JSON)
- `HttpError` class for structured error responses with HTTP status codes
- `sendJson` helper for consistent JSON responses with correct content type

4.12.2 Repository Pattern — Dual-Mode Data Access

The repository pattern abstracts data persistence behind a unified interface, allowing the system to switch between MySQL and file-based storage via the `DB_PROVIDER` environment variable.

Repository abstraction (`backend/src/repositories/queueRepository.js`):

```
const repositories = {
  file: fileRepository,
  mysql: mysqlRepository,
};

function getRepository() {
  const repository = repositories[env.databaseProvider];
  if (!repository) {
    throw new Error(`Unsupported DB_PROVIDER "${env.databaseProvider}".`);
  }
  return repository;
}

export function listTickets() {
  return getRepository().listTickets();
}
export function createTicket(ticket) {
  return getRepository().createTicket(ticket);
}
// ... all data operations delegate to the active repository
```

Both `fileQueueRepository.js` and `mysqlQueueRepository.js` implement the same interface:

- `initializeQueueRepository()` — initialise tables/files
- `listTickets()` , `findTicketById()` , `createTicket()` , `updateTicket()`
- `getNextSequence()` — atomic sequence number generation
- `listPatients()` , `findPatientById()` , `findPatientByPhone()` , `findPatientByPatientNumber()`
- `createPatient()` , `updatePatient()`
- `listNotifications()` , `createNotification()` , `updateNotification()`
- `listNotificationAttempts()` , `createNotificationAttempt()`
- `listAuditEvents()` , `createAuditEvent()`
- `listStaffUsers()` , `findStaffUserById()` , `findStaffUserByUsername()` , `createStaffUser()` , `updateStaffUser()`

4.12.3 Queue Service — Core Business Logic

The `queueService.js` file (2,234 lines) is the central business logic module, implementing all queue operations.

Ticket Registration: When a patient is registered, the system:

1. Validates input data (name, department, etc.)
2. Searches for an existing patient profile by phone, national ID, or name

3. Creates or links a patient profile with a permanent patient number
4. Gets the next department-specific sequence number from `system_counters`
5. Generates a ticket code using the department prefix (e.g., OPD-205)
6. Creates a ticket with status “waiting” and priority “green”
7. Broadcasts a “ticket.registered” SSE event
8. Dispatches a registration notification if consented
9. Records an audit event

Priority-Based Sorting: Tickets are sorted by priority rank, then by registration time within the same priority:

```
export function priorityRank(priority) {
  return PRIORITY_ORDER.indexOf(priority); // red: 0, yellow: 1, green: 2, black: 3
}

function sortWaitingTickets(tickets) {
  return [...tickets].sort((a, b) => {
    const priorityDiff = priorityRank(a.priority) - priorityRank(b.priority);
    if (priorityDiff !== 0) return priorityDiff;
    return new Date(a.registeredAt) - new Date(b.registeredAt);
  });
}
```

ETA Calculation: The system calculates estimated waiting times based on historical service time and queue position:

```
function calculateHistoricalServiceTime(tickets) {
  const completed = tickets.filter(
    (t) => t.serviceStartedAt && t.completedAt
  );
  if (!completed.length) return 8 * 60 * 1000; // default 8 minutes
  const total = completed.reduce(
    (sum, t) => sum + (new Date(t.completedAt) - new Date(t.serviceStartedAt)),
    0
  );
  return total / completed.length;
}
```

Bottleneck Detection: The system identifies departments with excessive waiting loads:

```
function detectBottlenecks(tickets) {
  const waiting = tickets.filter((t) => t.status === "waiting");
  const byDept = {};
  for (const t of waiting) {
    byDept[t.department] = (byDept[t.department] || 0) + 1;
  }
  return Object.entries(byDept)
    .filter(([_, count]) => count >= 5)
    .map(([department, waitingCount]) => ({ department, waitingCount }));
}
```

Department Routing Rules: The system includes keyword-based routing rules that suggest departments based on chief complaint text:

```
export const DEPARTMENT_ROUTING_RULES = [
  {
    department: "Casualty / Triage",
    keywords: ["chest pain", "breathing", "respiratory", "distress",
               "unconscious", "bleeding", "trauma", "injury", ...],
  },
  {
    department: "Pharmacy",
    keywords: ["medication", "refill", "prescription", "pharmacy", ...],
  },
  // ... Laboratory, Radiology, OI Clinic
];
```

4.12.4 Triage Assessment

The triage assessment system captures structured clinical data and auto-classifies priority based on emergency vital thresholds.

Emergency Vital Thresholds:

```
export const EMERGENCY_VITAL_THRESHOLDS = {
  oxygenSaturation: { min: 90, label: "SpO2 below 90%" },
  heartRate: { max: 130, min: 40, label: "Heart rate outside 40-130 bpm" },
  bloodPressureSystolic: { max: 180, min: 80, label: "Systolic BP outside 80-180 mmHg" },
  temperature: { min: 39.5, label: "Temperature at or above 39.5°C" },
  respiratoryRate: { max: 30, min: 8, label: "Respiratory rate outside 8-30 /min" },
  painScore: { min: 9, label: "Pain score 9 or 10" },
};
```

Auto-Classification Logic: When a triage assessment is submitted, the system evaluates each vital against the emergency thresholds. If any vital exceeds a threshold, the patient is automatically classified as Red priority and fast-tracked to the front of the queue. The triage nurse can override this classification, with the override recorded in the audit trail.

Triage Assessment Data Captured:

- Chief complaint
- Symptoms and severity notes
- Vitals: oxygen saturation (%), heart rate (bpm), blood pressure systolic/diastolic (mmHg), temperature (°C), respiratory rate (/min)
- Pain score (0–10)
- Manual priority selection (if no auto-Red trigger)
- Assessor identity and timestamp

4.12.5 Notification System

The notification system (`notificationService.js`, 19,713 bytes) handles WhatsApp message dispatch with retry logic.

WhatsApp Provider Abstraction (`whatsAppProvider.js`): The system supports two providers:

- **Mock provider** (default for development): Simulates delivery with configurable delay and failure rate
- **Meta Cloud API** (production): Sends real WhatsApp messages via Meta's Graph API

Provider selection is controlled by the `WHATSAPP_PROVIDER` environment variable.

Meta Cloud API Integration:

- API version: v23.0
- Base URL: `https://graph.facebook.com`
- Uses template-based messaging with configurable language
- Templates: queue-registered, queue-ready-soon, queue-called
- Request timeout: 10 seconds (configurable)

Notification Lifecycle:

1. A queue event triggers notification creation (registration, call, missed, transfer, turn readiness)
2. System checks patient consent
3. Notification record created with status "pending"
4. Message sent via the configured provider
5. On success: status updated to "sent", provider message ID recorded
6. On failure: error details recorded, retry scheduled
7. Retry delays: 15 seconds, 60 seconds, 300 seconds (configurable)
8. Maximum attempts enforced per notification
9. Each attempt creates a `notification_attempts` record

Retry Queue:

- Automatic retry with exponential back-off (15s → 60s → 300s)
- Manual retry by staff from the dashboard
- Bulk retry for multiple failed notifications
- Startup recovery: pending retries are processed when the server restarts

4.12.6 Authentication and Security

Staff Authentication (`authService.js`):

- Login with username/email and password
- Passwords hashed with bcrypt (cost factor 12)
- Password policy: minimum 8 characters, at least 3 character groups (uppercase, lowercase, digits, special), common password rejection

- Session tokens stored in an in-memory Map with 8-hour TTL
- Role-based access control with roles: admin, reception, triage, clinician
- Role mapping supports synonyms (reception → receptionist, triage → nurse, clinician → doctor)

RBAC Middleware (`middleware/auth.js`):

```
export function withStaffAuth(allowedRoles, handler) {
  return (context) => {
    const expandedRoles = [
      ...allowedRoles,
      ...allowedRoles.map((role) => roleMapping[role]).filter(Boolean),
    ];
    const session = requireStaffSession(context.req, expandedRoles);
    return handler({
      ...context,
      staff: session.user,
      session,
    });
  };
}
```

Patient Authentication (`patientAuthService.js`):

- Separate authentication flow from staff
- Registration with username, email, password, phone, national ID
- bcrypt password hashing
- Session stored in localStorage with expiry tracking

Audit Trail (`auditService.js`):

- Append-only event log (MySQL table or JSONL file)
- SHA-256 hash chaining: each event's hash incorporates the previous event's hash
- Sensitive data redaction before hashing
- Actor attribution: user ID, name, role, department
- Tracked actions: login, staff creation, profile updates, ticket operations, triage, prescriptions, test orders, notifications, arrivals

4.12.7 Real-Time Updates (SSE)

The SSE implementation consists of a server-side event stream and a client-side EventSource consumer.

Server-side (`sockets/queueEvents.js`):

- Clients connect to `GET /api/events`
- Response headers: `Content-Type: text/event-stream` , `Cache-Control: no-cache` , `Connection: keep-alive`
- Heartbeat every 25 seconds to maintain connection
- Client retry interval: 3 seconds (sent as `retry:` field)

- Events are broadcast via a `broadcastQueueEvent` function that writes to all connected clients

Event Types:

- `connected` — initial connection confirmation
- `ticket.registered` — new patient registered
- `ticket.called` — patient called to service
- `ticket.missed` — patient marked missed
- `ticket.recalled` — patient recalled to queue
- `ticket.transferred` — patient transferred between departments
- `ticket.priority-updated` — triage priority changed
- `ticket.completed` — service completed
- `notification.sent` — notification delivered
- `notification.failed` — notification delivery failed

4.12.8 Reporting Endpoints

The system implements five report endpoints, each with date range filtering:

Table 4.44: Reporting Endpoints

Endpoint	Method	Description
`/api/reports/average-wait`	GET	Average wait and service time by department
`/api/reports/department-performance`	GET	Throughput, missed rate, priority breakdown per department
`/api/reports/no-show`	GET	No-show rate with per-ticket notification correlation
`/api/reports/triage`	GET	Triage rate, timing, outcomes, overrides, emergency triggers
`/api/whatsapp/status`	GET	WhatsApp configuration and credential status

All report endpoints require staff authentication (triage, clinician, or admin roles) and accept `from` and `to` query parameters for date filtering.

4.13 Frontend Implementation

4.13.1 Application Architecture

The frontend is a React single-page application built with Vite. The application architecture follows these patterns:

Routing: React Router v7 with nested routes. All routes are defined in `App.jsx` and wrapped in a single `AppLayout` component that provides the header, footer, and context providers.

Protected Routes: The `ProtectedRoute` component wraps routes that require authentication, checking the user's authentication status and role before rendering. Unauthenticated users are redirected to the login page; unauthorised users see an access denied message.

Layout Strategy:

- `AppLayout` — main layout with conditional header/footer based on route type
- `PatientPortalLayout` — sidebar navigation for patient portal pages
- `DashboardLayout` — staff dashboard wrapper
- Routes are classified as auth paths, patient portal paths, staff workspace paths, or public paths

4.13.2 State Management

The system uses React Context for global state management:

Table 4.45: State Management

Context	Purpose
`AuthContext`	Staff authentication state (user, token, login, logout)
`PatientAuthProvider`	Patient authentication state (user, token, login, logout)
`LiveRefreshContext`	Global refresh trigger — increments a counter to signal data re-fetch
`QueueRealtimeProvider`	SSE connection status and event handling

Live Refresh Mechanism: The `LiveRefreshContext` provides a `refreshLiveData` function that increments a `refreshToken` counter. Components using the `useApiResource` hook watch this counter and re-fetch data when it changes. This allows the SSE event handler to trigger global data refresh by calling `refreshLiveData()`.

4.13.3 API Layer

The API layer is built on Axios with a centralised client configuration (`services/api.js`):

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:4242",
});

apiClient.interceptors.request.use((config) => {
  const token = getStoredToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

- Base URL configurable via `VITE_API_URL` environment variable
- Request interceptor automatically attaches the auth token
- `apiRequest` helper function for consistent error handling

- Patient API uses a separate Axios client with patient-specific token

Queue API Functions (`services/queueApi.js`): The file contains 30+ API functions covering:

- Queue metadata and ticket fetching
- Patient registration (staff, self, kiosk, pre-registration)
- Triage assessment submission
- Queue operations (call next, recall, transfer, status update)
- Prescription and test order management
- ETA estimates and ticket tracking
- Notification retry (single and bulk)
- Dashboard summary
- Reports (average wait, department performance, no-show, triage, WhatsApp status)
- Department-specific queue board

4.13.4 Real-Time Integration

The `QueueRealtimeProvider` (`sockets/QueueRealtimeProvider.jsx`) manages the SSE connection:

1. On mount, it creates an `EventSource` connection to `/api/events`
2. On successful connection, status is set to “connected”
3. On error, status is set to “reconnecting” (EventSource auto-reconnects)
4. On message, the event payload is parsed and `refreshLiveData()` is called
5. If `EventSource` is not supported, falls back to polling every 10 seconds
6. An additional auto-refresh timer runs every 10 seconds (configurable) to refresh wait times
7. Settings are loaded from localStorage (live sync enabled, polling fallback, refresh interval)

4.13.5 Key Page Implementations

Patient Registration (`register.jsx` — 44,592 bytes)

- Multi-section form with react-hook-form and Zod validation
- QR code scanner for returning patient lookup (html5-qrcode)
- Patient category selector with visual icons
- Department dropdown populated from API metadata
- Notification consent toggle
- On submit: displays ticket with QR code, print button, and WhatsApp share option
- Real-time form validation with inline error messages

Triage Assessment (`triage.jsx` — 51,594 bytes)

- Patient selector from waiting queue (sorted by priority, then wait time)
- Vitals input form with range indicators
- Real-time emergency vital threshold warnings as values are entered

- Pain score slider (0–10) with visual colour scale
- Symptoms and severity text areas
- Auto-suggested priority based on vitals (manually overridable)
- Assessment history for previously triaged patients

Public Queue Board (`queue.jsx` — 35,047 bytes)

- Three-column layout: “Now Serving”, “Waiting”, “Missed”
- Large-format token numbers in monospace font
- Priority colour coding (red, yellow, green borders)
- Animated transitions when tickets move between columns
- Real-time updates via SSE (no page refresh required)
- Department filter selector
- Particle background animation
- Connection status indicator

Department Display Board (`department-board.jsx` — 17,834 bytes)

- Department-specific view filtered by URL query parameter (`?dept=OPD`)
- Shows only tickets for the selected department
- Larger token number display for waiting area visibility
- Real-time SSE updates
- Dark teal theme matching the main queue board
- Auto-refresh on connection loss

Staff Dashboard (`dashboard.jsx` — 46,343 bytes)

- Live metrics cards: waiting count, in-service count, missed count, average wait time
- Department-scoped queue table with sortable columns
- Patient action buttons: call next, mark missed, recall, transfer
- Operational alerts panel: reassessment alerts, notification failures, critical metrics
- Notification analytics with Recharts visualisations
- Real-time auto-refresh via SSE and LiveRefreshContext
- Search functionality for patient lookup

Patient Tracking (`track.jsx` — 66,903 bytes)

- Token code input with auto-lookup
- Progress indicator showing queue position
- Estimated wait time display
- Patients ahead count
- Department directions and current status
- Notification history timeline

- Real-time updates via SSE
- Dark teal themed UI with glassmorphism cards

Reports Page (`staff-modules.jsx` — 57,232 bytes)

- Tabbed interface with 6 tabs: Overview, Average Wait Time, Department Performance, No-Show, Triage, WhatsApp Status
- Date range picker (from/to) for all report types
- Summary metric cards per report type
- Department breakdown tables
- Per-ticket notification correlation (no-show report)
- WhatsApp configuration checklist (status report)
- Loading states with `LoadingPanel` component

Patient Portal (`patient-dashboard.jsx` — 29,933 bytes)

- Patient welcome card with time-based greeting
- Active tickets display with status badges
- QR patient card with download and print functionality
- Quick action buttons (track ticket, view queue)
- Visit history with timeline
- Prescriptions and test orders sections
- Feedback section
- Theme toggle (dark/light) with localStorage persistence
- Profile avatar with initials fallback

4.13.6 UI Component Library

The system uses shadcn/ui, a component library built on Radix UI primitives. The following components are implemented in `components/ui/`:

Table 4.46: UI Component Library

Component	File	Purpose
Accordion	accordion.jsx	Collapsible content sections
Alert Dialog	alert-dialog.jsx	Confirmation dialogs
Avatar	avatar.jsx	User profile images
Badge	badge.jsx	Status and priority indicators
Button	button.jsx	Action buttons with variants
Calendar	calendar.jsx	Date selection
Card	card.jsx	Content containers
Chart	chart.jsx	Recharts wrapper

Component	File	Purpose
Command	command.jsx	Searchable command palette
Dialog	dialog.jsx	Modal dialogs
Dropdown Menu	dropdown-menu.jsx	Context menus
Form	form.jsx	Form integration with react-hook-form
Input	input.jsx	Text inputs
Label	label.jsx	Form labels
Popover	popover.jsx	Floating content
Progress	progress.jsx	Progress bars
Select	select.jsx	Dropdown selection
Separator	separator.jsx	Visual dividers
Sheet	sheet.jsx	Slide-out panels
Skeleton	skeleton.jsx	Loading placeholders
Sonner	sonner.jsx	Toast notifications
Switch	switch.jsx	Toggle switches
Table	table.jsx	Data tables
Tabs	tabs.jsx	Tabbed navigation
Textarea	textarea.jsx	Multi-line text input
Tooltip	tooltip.jsx	Hover information

Custom Components:

- `SystemLoader` / `LoadingPanel` — animated 3D cube loading indicator
- `RouteLoadingBar` — top-of-page loading bar for route transitions
- `HeroSlider` — auto-rotating hero section for the landing page
- `ProtectedRoute` — authentication and role-based route guard
- `WaitLessLogo` — SVG logo component
- `AuthShell` / `PatientAuthShell` — branded shell for authentication pages

4.14 Database Implementation

4.14.1 Schema Implementation

The database schema is defined in `backend/src/db/schema.sql` and contains 10 tables:

1. **departments** — Department reference data (6 departments)
2. **system_counters** — Monotonic sequence counters per department
3. **patient_profiles** — Persistent patient records with permanent patient numbers

4. **tickets** — Queue tickets with full lifecycle timestamps
5. **notifications** — Notification records with delivery tracking
6. **notification_attempts** — Per-attempt delivery records
7. **staff_users** — Staff accounts with roles and department assignments
8. **patient_users** — Patient portal accounts
9. **audit_events** — Immutable, hash-chained audit trail
10. **system_counters** — Department sequence counters

4.14.2 Indexing Strategy

The schema uses strategic composite and single-column indexes to optimise query performance:

Table 4.47: Indexing Strategy

Index	Table	Columns	Purpose
idx_tickets_department_status	tickets	department, status	Queue queries by department and status
idx_tickets_priority_status	tickets	priority, status	Priority-based queue sorting
idx_tickets_registered_at	tickets	registered_at	Date-range reports and sorting
idx_tickets_patient_id	tickets	patient_id	Patient history lookup
idx_notifications_ticket_id	notifications	ticket_id	Notification lookup by ticket
idx_notifications_status	notifications	status	Pending/failed notification queries
idx_notifications_created_at	notifications	created_at	Notification history by date
idx_audit_events_occurred_at	audit_events	occurred_at	Chronological audit display
idx_audit_events_actor_id	audit_events	actor_id	Audit filter by staff member
idx_audit_events_action	audit_events	action	Audit filter by action type
idx_patient_profiles_name	patient_profiles	full_name	Patient search by name
idx_patient_profiles_phone	patient_profiles	phone	Patient search by phone
idx_patient_profiles_national_id	patient_profiles	national_id	Patient search by national ID

4.14.3 Initialisation and Seeding

Database Initialisation (`db/initDb.js`):

- Creates all tables if they do not exist
- Seeds the 6 default departments (OPD, Casualty/Triage, Pharmacy, Laboratory, Radiology, OI Clinic)
- Initialises department sequence counters to 100
- Seeds demo tickets for development (12 sample patients across departments)

Admin Seeding (`db/seedAdmin.js`):

- Creates an initial administrator account from environment variables
- `INITIAL_ADMIN_USERNAME`, `INITIAL_ADMIN_EMAIL`, `INITIAL_ADMIN_PASSWORD`

- Refuses to create admin with a weak password
- Only runs when `DB_PROVIDER=mysql`

4.14.4 Dual-Mode Storage

MySQL Mode (`mysqlQueueRepository.js`):

- Uses mysql2 connection pool (configurable limit, default 10)
- All operations use parameterised queries (SQL injection prevention)
- Atomic sequence generation using `UPDATE ... SET value = value + 1` and `SELECT LAST_INSERT_ID()`
- JSON columns for audit event before/after state

File Mode (`fileQueueRepository.js`):

- JSON file at `data/queue-state.json`
- In-memory array with file write-back on each mutation
- Atomic file writes using write-then-rename pattern
- Suitable for development and single-user testing

4.15 Integration Implementation

4.15.1 WhatsApp Cloud API Integration

The WhatsApp integration (`whatsAppProvider.js`) supports two modes:

Mock Provider (Development):

- Simulates message delivery with configurable delay (`WHATSAPP_MOCK_DELAY_MS` , default 250ms)
- Configurable failure rate for testing retry logic (`WHATSAPP_MOCK_FAILURE_RATE` , default 0)
- No external API calls required

Meta Cloud API (Production):

- Sends messages to `https://graph.facebook.com/v23.0/{phone_number_id}/messages`
- Uses template-based messaging with configurable template names and language
- Authentication via bearer token (`WHATSAPP_ACCESS_TOKEN`)
- Request timeout: 10 seconds (`WHATSAPP_REQUEST_TIMEOUT_MS`)
- Webhook verification support (`WHATSAPP_WEBHOOK_VERIFY_TOKEN`)
- Webhook event handling for delivery status updates

Phone Number Sanitisation: Phone numbers are sanitised to international format (removing spaces, dashes, leading zeros) before sending.

Template Resolution: The system resolves template names based on notification type:

- `queue_registered` — Registration confirmation
- `queue_ready_soon` — Turn approaching
- `queue_called` — Turn called

4.15.2 QR Code Generation and Scanning

Generation: The `qrcode` library generates QR codes as data URLs for:

- Patient registration tickets (tracking QR)
- Patient profile cards (patient portal)
- Kiosk check-in tokens

Scanning: The `html5-qrcode` library provides camera-based QR scanning for:

- Kiosk self-check-in (scan patient QR card)
- Reception returning patient lookup (scan QR to find existing record)

4.15.3 Audit Trail Implementation

The audit service (`auditService.js`) implements:

Hash Chain Computation:

```
function computeEventHash(event, previousHash) {
  const payload = JSON.stringify({
    occurredAt: event.occurredAt,
    actorId: event.actorId,
    action: event.action,
    entityType: event.entityType,
    entityId: event.entityId,
    outcome: event.outcome,
    beforeState: event.beforeState,
    afterState: event.afterState,
    previousHash: previousHash,
  });
  return crypto.createHash("sha256").update(payload).digest("hex");
}
```

Redaction: Sensitive fields (passwords, tokens, phone numbers in certain contexts) are replaced with `[REDACTED]` before being included in the hash computation and stored in the database.

Tamper Detection: The system can verify the integrity of the audit chain by recomputing each event's hash from its stored content and comparing it to the stored hash. Any mismatch indicates tampering.

4.16 Deployment Configuration

4.16.1 Environment Variables

The system is configured through environment variables in a `.env` file:

Table 4.48: Environment Variables

Variable	Default	Description
`PORT`	4242	Backend server port
`CORS_ORIGIN`	*	Allowed CORS origins (restrict in production)
`PUBLIC_APP_URL`	http://localhost:5173	Frontend public URL
`DB_PROVIDER`	mysql	Storage mode: “mysql” or “file”
`DB_HOST`	127.0.0.1	MySQL host
`DB_PORT`	3306	MySQL port
`DB_NAME`	waitless	MySQL database name
`DB_USER`	root	MySQL username
`DB_PASSWORD`	—	MySQL password
`DB_CONNECTION_LIMIT`	10	MySQL connection pool size
`DB_SEED_DEMO_DATA`	true	Seed demo tickets on init
`AUTH_SESSION_TTL_MS`	28800000	Session TTL (8 hours)
`INITIAL_ADMIN_USERNAME`	admin	Initial admin username
`INITIAL_ADMIN_EMAIL`	admin@waitless.local	Initial admin email
`INITIAL_ADMIN_PASSWORD`	—	Initial admin password (required for first run)
`WHATSAPP_PROVIDER`	mock	WhatsApp provider: “mock” or “meta-cloud”
`WHATSAPP_API_BASE_URL`	https://graph.facebook.com	Meta API base URL
`WHATSAPP_API_VERSION`	v23.0	Meta API version
`WHATSAPP_PHONE_NUMBER_ID`	—	Meta phone number ID
`WHATSAPP_ACCESS_TOKEN`	—	Meta access token
`WHATSAPP_APP_SECRET`	—	Meta app secret for webhook verification
`WHATSAPP_WEBHOOK_VERIFY_TOKEN`	—	Webhook verification token
`WHATSAPP_REQUEST_TIMEOUT_MS`	10000	API request timeout
`WHATSAPP_RETRY_DELAYS_MS`	15000,60000,300000	Retry delay schedule
`WHATSAPP_MOCK_DELAY_MS`	250	Mock provider delay
`WHATSAPP_MOCK_FAILURE_RATE`	0	Mock provider failure rate (0–1)
`WHATSAPP_TEMPLATE_LANGUAGE`	en_US	Template language code
`VITE_API_URL`	http://localhost:4242	Frontend API base URL

4.16.2 Backend Startup

The server startup sequence (`index.js`):

1. Load environment variables from `.env`
2. Verify the configured port is available (probe and release)
3. Initialise the queue repository (create tables, seed data)
4. Initialise the notification delivery service (process pending retries)
5. If MySQL mode: seed the default admin account
6. Start the HTTP server on the configured port
7. Log the server URL

Error handling:

- `EADDRINUSE` — port already in use, display helpful message
- Other errors — log and exit with code 1

4.16.3 Frontend Build

Development:

```
cd WAITLESS/frontend
npm install
npm run dev    # Vite dev server at http://localhost:5173
```

Production:

```
cd WAITLESS/frontend
npm run build  # Output to dist/
npm run preview  # Preview production build
```

The production build verified on 10 August 2026:

- Transformed 2,633 modules
- Output: `dist/index.html` + `dist/assets/` (CSS, JS, images)
- CSS: 290.2 KB (42.2 KB gzipped)
- JavaScript: 2,440.6 KB across 65 lazy-loaded files (660.6 KB gzipped in total)
- Assets: Logo images, background images (BACKG.png, etc.)

4.16.4 LAN Deployment

For hospital LAN deployment:

1. Install Node.js 18+ and MySQL on a server machine
2. Create the `waitless` database and run `schema.sql`
3. Configure `.env` with MySQL credentials and `WHATSAPP_PROVIDER=meta-cloud`

4. Set `CORS_ORIGIN` to the hospital LAN IP range
5. Set `VITE_API_URL` to the server's LAN IP address
6. Run `npm install` in both `backend/` and `frontend/`
7. Build the frontend: `npm run build` in `frontend/`
8. Serve the frontend static files (via nginx, Apache, or a simple static server)
9. Start the backend: `npm start` in `backend/`
10. Access from any device on the LAN via the server's IP address

The system operates on the hospital LAN without internet for all core queue functions. WhatsApp notifications require internet connectivity to reach the Meta Cloud API, but are queued for retry when connectivity returns.

4.17 Summary

This chapter has detailed the implementation of the WaitLess Digital Queue Management System. The technology stack combines React 19 with Vite for the frontend and Node.js with a custom HTTP router for the backend, supported by MySQL for data persistence and SSE for real-time updates. The system structure follows a clean layered architecture with controllers, services, and repositories, while the frontend uses a component-based design with context providers for state management.

Key implementation achievements include the priority-based queue sorting algorithm, emergency vital threshold detection for triage, WhatsApp Cloud API integration with retry logic, SHA-256 hash-chained audit trail, dual-mode storage abstraction, and real-time SSE updates. The deployment configuration supports both development and hospital LAN production environments.

The next chapter, Chapter 5: Testing and Evaluation, presents the testing strategy, test cases, results, and system evaluation.

Chapter 5: Presentation of Results

5.1 Introduction

This chapter presents the results produced during verification and evaluation of the WaitLess Digital Queue Management System. The results are organised around the original problem: fragmented patient flow, weak triage visibility, missed queue turns, and the absence of dependable operational information. Testing examined whether the prototype behaved as designed; it did not attempt to claim a measured clinical impact before a live hospital pilot.

The evidence combined automated backend tests with documented functional walkthroughs of the frontend interface. Production-build, API, performance, security, and usability-readiness findings are also reported. Where a result came from a local development environment or a planned instrument rather than an independent hospital study, that boundary is stated explicitly.

5.2 Testing Approach

5.2.1 Testing Strategy Overview

Table 5.1: Testing Strategy Overview

Testing Level	Scope	Method	Tools
Unit Testing	Backend service functions	Automated	Node.js native test runner (`node --test`)
Integration Testing	API endpoints, database operations	Manual + automated	curl, browser, test scripts
Functional Testing	Frontend UI, user workflows	Manual	Browser-based testing
Build Verification	Frontend compilation, backend startup	Automated	Vite build, Node.js startup
Security Testing	Authentication, RBAC, audit integrity	Automated + manual	Test scripts, code inspection
Performance Testing	Bundle size, API response, real-time latency	Manual	Browser DevTools, network analysis

5.2.2 Test Environment

Table 5.2: Test Environment

Parameter	Value
Operating System	Windows 11
Node.js Version	18+ (ESM modules)

Parameter	Value
Database Mode	File-based JSON (unit tests), MySQL (integration)
WhatsApp Provider	Mock (unit tests), Meta Cloud API (production)
Frontend	Vite development server (localhost:5173)
Backend	Node.js HTTP server (localhost:4242)
Browser	Chrome / Firefox / Edge

5.2.3 Automated Test Framework

The backend uses Node.js's native test runner (`node:test`) with `node:assert/strict` for assertions. Tests are located in `backend/test/` and executed via `npm test` (which runs `node --test`).

Three test suites have been implemented:

Table 5.3: Automated Test Framework

Test File	Tests	Focus Area
`audit-service.test.js`	4	Audit trail hash chain integrity, redaction, tamper detection, admin guard
`staff-auth-security.test.js`	4	Password hashing, plaintext rejection, password policy, admin guard on staff registration
`returning-patient.test.js`	4	Returning patient reuse, QR lookup, token immutability, WhatsApp notification content
Total	**12**	

5.3 Test Plan and Test Cases

5.3.1 Automated Unit Tests

Test Suite 1: Audit Service Tests (`audit-service.test.js`)

Table 5.4: Test Suite 1: Audit Service Tests (audit-service.test.js)

Test ID	Description	Preconditions	Expected Result	Status
AUT-01	Records attributed events in a verified append-only hash chain	File-based storage, empty audit log	Two events recorded; integrity valid; 2 events checked; actor name matches; event hash is 64-char hex	✅ Pass
AUT-02	Redacts secret-bearing fields before they reach storage	Audit log with prior events	`password` and `accessToken` fields replaced with `[REDACTED]` ; safe fields preserved; raw file does not contain plaintext secrets	✅ Pass
AUT-03	Detects a modified historical event	Audit log with 2+ events	After tampering with event text in file, `integrity.valid` returns `false`	✅ Pass

Test ID	Description	Preconditions	Expected Result	Status
AUT-04	Protects the audit endpoint with the admin guard	Source code of `index.js`	Route registration matches `withStaffAuth(["admin"], auditEventsController)` pattern	✅ Pass

Test Suite 2: Staff Authentication Security Tests (`staff-auth-security.test.js`)

Table 5.5: Test Suite 2: Staff Authentication Security Tests (staff-auth-security.test.js)

Test ID	Description	Preconditions	Expected Result	Status
AUT-05	Staff passwords are stored as bcrypt hashes	bcryptjs installed	Hash differs from plaintext; matches `$2[abxy]$12$` pattern; correct password verifies; wrong password fails	✅ Pass
AUT-06	Plaintext values are never accepted as password hashes	—	`verifyPassword("SecurePass!42", "SecurePass!42")` returns `false`	✅ Pass
AUT-07	New staff passwords must meet the server-side policy	—	Short password rejected (“at least 10 characters”); all-lowercase rejected (“at least three”); valid password accepted (returns `null`)	✅ Pass
AUT-08	The staff registration endpoint is protected by the admin guard	Source code of `index.js`	Route matches `withStaffAuth(["admin"], staffRegisterController)` pattern; no unguarded registration route exists	✅ Pass

Test Suite 3: Returning Patient Tests (`returning-patient.test.js`)

Table 5.6: Test Suite 3: Returning Patient Tests (returning-patient.test.js)

Test ID	Description	Preconditions	Expected Result	Status
AUT-09	Reuses one patient master record across multiple visits	File-based storage, mock WhatsApp	Second visit returns same `patientId` and `patientNumber` ; search shows `totalVisits: 2` ; `lastDepartment` updated	✅ Pass
AUT-10	Finds a returning patient from the QR payload	Patient registered with national ID	QR lookup value matches patient; `patientNumber` matches `WL-P\d{6}` pattern	✅ Pass
AUT-11	Issues one immutable department token across the registration handoff	Patient registered in Radiology	Ticket matches `RAD-\d{3,}` pattern; handoff token equals ticket code; tracking path correct; all channel statuses correct; priority assignment preserves token; tracking returns correct ticket	✅ Pass
AUT-12	Includes the same token and tracking link in WhatsApp registration text	Patient registered with consent enabled	Notification ticket matches visit ticket; message contains ticket code; message contains tracking URL with correct format	✅ Pass

5.3.2 Manual Functional Test Cases

Module: Authentication

Table 5.7: Module: Authentication

Test ID	Description	Steps	Expected Result	Status
MAN-01	Staff login with valid credentials	1. Navigate to /admin/login 2. Enter valid username/password 3. Click Login	User authenticated; redirected to role-appropriate dashboard; session token stored	✅ Pass
MAN-02	Staff login with invalid credentials	1. Navigate to /admin/login 2. Enter invalid password 3. Click Login	Error message displayed; no session created	✅ Pass
MAN-03	Role-based route protection	1. Login as reception 2. Navigate to /admin/triage	Access denied; triage page only for triage role	✅ Pass
MAN-04	Unauthenticated access to protected route	1. Open /admin/dashboard without login	Redirected to login page	✅ Pass
MAN-05	Patient registration and login	1. Navigate to /patient/register 2. Fill form 3. Submit 4. Login at /patient/login	Patient account created; login successful; patient dashboard displayed	✅ Pass
MAN-06	Password reset flow	1. Navigate to /admin/forgot-password 2. Enter email 3. Follow reset link 4. Set new password	Reset email generated; password updated; login works with new password	✅ Pass

Module: Patient Registration

Table 5.8: Module: Patient Registration

Test ID	Description	Steps	Expected Result	Status
MAN-07	Register new patient	1. Login as reception 2. Navigate to /admin/register 3. Fill all fields 4. Select department 5. Submit	Ticket created with unique token; QR code displayed; patient appears in queue	✅ Pass
MAN-08	Search returning patient	1. Login as reception 2. Use search field 3. Enter existing patient name/phone/ID	Existing patient found; can create new visit linked to existing profile	✅ Pass
MAN-09	Self-registration	1. Navigate to /self-register 2. Fill demographics 3. Select department 4. Submit	Ticket created; token displayed; patient appears in queue	✅ Pass
MAN-10	Kiosk check-in	1. Navigate to /kiosk 2. Scan QR or enter phone 3. Select department 4. Submit	Patient identified; ticket created; token and ETA displayed	✅ Pass
MAN-11	Form validation	1. Submit registration with missing required fields	Inline validation errors displayed; form not submitted	✅ Pass

Module: Triage Assessment

Table 5.9: Module: Triage Assessment

Test ID	Description	Steps	Expected Result	Status
MAN-12	Triage assessment with normal vitals	1. Login as triage 2. Select waiting patient 3. Enter normal vitals 4. Select Green priority 5. Submit	Assessment saved; priority updated to Green; queue reordered	✅ Pass
MAN-13	Emergency vital auto-classification	1. Login as triage 2. Select patient 3. Enter SpO2 = 85% 4. Submit	System auto-assigns Red priority; emergency fast-track triggered; patient moves to front of queue	✅ Pass
MAN-14	Manual priority override	1. Login as triage 2. Select auto-Red patient 3. Change priority to Yellow 4. Submit	Override recorded; priority updated to Yellow; audit event logs before/after priority	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-15	Reassessment alert display	1. Have Red/Yellow patient waiting beyond threshold 2. View dashboard	Reassessment alert displayed in alerts panel	✅ Pass

Module: Queue Management

Table 5.10: Module: Queue Management

Test ID	Description	Steps	Expected Result	Status
MAN-16	Call next patient	1. Login as clinician 2. Click “Call Next” on dashboard	Highest-priority, longest-waiting patient called; status updated; SSE broadcast; notification sent	✅ Pass
MAN-17	Priority ordering verification	1. Have Red, Yellow, Green patients waiting 2. Call next 3 times	Red called first, then Yellow, then Green	✅ Pass
MAN-18	Mark missed and recall	1. Select called patient 2. Click “Mark Missed” 3. Wait 4. Click “Recall”	Status changes to missed; missed notification sent; patient moves to recall lane; on recall, status returns to waiting	✅ Pass
MAN-19	Queue transfer	1. Select patient 2. Click “Transfer” 3. Select target department 4. Confirm	Department updated; previous department recorded; transfer notification sent; patient appears in target queue	✅ Pass
MAN-20	Department-scoped data filtering	1. Login as non-admin staff 2. View dashboard	Only tickets and notifications for staff’s department are displayed	✅ Pass

Module: Notifications

Table 5.11: Module: Notifications

Test ID	Description	Steps	Expected Result	Status
MAN-21	Registration notification (mock)	1. Register patient with consent 2. Check notification log	Notification created; status “sent” (mock provider); message contains ticket code and tracking URL	✅ Pass
MAN-22	Notification retry on failure	1. Set mock failure rate to 1.0 2. Register patient with consent 3. Check notification log	All attempts fail; status “failed”; retry attempts logged with timestamps	✅ Pass
MAN-23	Manual notification retry	1. Have failed notification 2. Click “Retry” on dashboard 3. Check status	Notification retried; if provider available, status updates to “sent”	✅ Pass
MAN-24	Notification consent honoured	1. Register patient without consent 2. Check notification log	No notification created for non-consenting patient	✅ Pass

Module: Queue Display

Table 5.12: Module: Queue Display

Test ID	Description	Steps	Expected Result	Status
MAN-25	Public queue board display	1. Navigate to /queue 2. Observe three columns	Now Serving, Waiting, Missed columns displayed; token numbers shown; no patient names visible	✅ Pass
MAN-26	Real-time queue board update	1. Open /queue in browser A 2. Register patient in browser B	Queue board in browser A updates without refresh; new ticket appears in Waiting column	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-27	Department-specific board	1. Navigate to /department-board?dept=OPD 2. Observe	Only OPD tickets displayed; real-time updates working	✅ Pass

Module: Patient Tracking

Table 5.13: Module: Patient Tracking

Test ID	Description	Steps	Expected Result	Status
MAN-28	Track valid ticket	1. Navigate to /track 2. Enter valid token (e.g., OPD-205)	Ticket details displayed: status, position, patients ahead, ETA, department	✅ Pass
MAN-29	Track invalid ticket	1. Navigate to /track 2. Enter invalid token	“Ticket not found” message displayed	✅ Pass
MAN-30	Real-time tracking update	1. Open /track with valid token 2. Call the patient from staff dashboard	Tracking page updates status without refresh	✅ Pass

Module: Reports

Table 5.14: Module: Reports

Test ID	Description	Steps	Expected Result	Status
MAN-31	Average wait time report	1. Navigate to /admin/reports 2. Select “Average Wait Time” tab 3. Set date range 4. View	Report displays average wait and service times per department	✅ Pass
MAN-32	Department performance report	1. Select “Department Performance” tab 2. Set date range	Report displays throughput, missed rate, priority breakdown per department	✅ Pass
MAN-33	No-show report	1. Select “No-Show” tab 2. Set date range	Report displays no-show rate with per-ticket notification delivery correlation	✅ Pass
MAN-34	Triage report	1. Select “Triage” tab 2. Set date range	Report displays triage rate, timing, outcomes, overrides, emergency triggers	✅ Pass
MAN-35	WhatsApp status check	1. Select “WhatsApp Status” tab	Configuration checklist displayed showing provider, credentials, template status	✅ Pass

Module: Security and Audit

Table 5.15: Module: Security and Audit

Test ID	Description	Steps	Expected Result	Status
MAN-36	Audit trail display	1. Login as admin 2. Navigate to /admin/audit	Audit events displayed chronologically with actor, action, entity, timestamp	✅ Pass
MAN-37	Audit hash chain verification	1. View audit trail 2. Check integrity indicator	Hash chain valid; no tamper events detected	✅ Pass
MAN-38	Staff management (admin only)	1. Login as admin 2. Navigate to /admin/staff 3. Create new staff	Staff creation form accessible; new staff created with hashed password	✅ Pass
MAN-39	Staff management access denied	1. Login as non-admin 2. Navigate to /admin/staff	Access denied; only admin role can manage staff	✅ Pass

5.4 Test Results

5.4.1 Automated Test Results

Table 5.16: Automated Test Results

Test Suite	Tests	Passed	Failed	Pass Rate
Audit Service	4	4	0	100%
Staff Auth Security	4	4	0	100%
Returning Patient	4	4	0	100%
Total	**12**	**12**	**0**	**100%**

Command: `npm test` (executed in `backend/` directory)

Key Findings:

- The SHA-256 hash chain correctly detects tampering when an event is modified in the audit log file
- bcrypt password hashing produces `$2b$12$` formatted hashes, rejecting plaintext comparisons
- Password policy enforces minimum 10 characters and at least 3 character groups
- Returning patient search correctly links multiple visits to a single patient master record
- Queue tokens are immutable and consistent across all delivery channels (print, phone, WhatsApp, display board)

5.4.2 Manual Test Results

The manual cases were executed as controlled prototype walkthroughs using synthetic records. They provided repeatable acceptance evidence for the demonstrated workflows, but they were not automated regression tests and were not an independent user-acceptance study. They must therefore be repeated with authorised hospital participants before deployment.

Table 5.17: Manual Test Results

Module	Tests	Passed	Failed	Pass Rate
Authentication	6	6	0	100%
Patient Registration	5	5	0	100%
Triage Assessment	4	4	0	100%
Queue Management	5	5	0	100%
Notifications	4	4	0	100%
Queue Display	3	3	0	100%
Patient Tracking	3	3	0	100%
Reports	5	5	0	100%
Security and Audit	4	4	0	100%
Total	**39**	**39**	**0**	**100%**

5.4.3 Build Verification

Table 5.18: Build Verification

Verification	Command	Result
Frontend production build	`npm run build` in `frontend/`	Success - 2,633 modules transformed
Frontend CSS output	-	290.2 KB (42.2 KB gzipped)
Frontend JavaScript output	-	2,440.6 KB across 65 files (660.6 KB gzipped in total)
Backend startup	`npm start` in `backend/`	✅ Success — "WaitLess backend listening on http://localhost:4242"
API health check	`GET /api/health`	✅ 200 OK — `{ status: "ok" }`
API metadata	`GET /api/meta`	✅ 200 OK — departments, priorities, statuses returned

5.4.4 API Endpoint Verification

Table 5.19: API Endpoint Verification

Endpoint	Method	Auth Required	Status	Response
`/api/health`	GET	No	✅ 200	`{ status: "ok" }`
`/api/meta`	GET	No	✅ 200	Departments, priorities, statuses
`/api/queue/board`	GET	No	✅ 200	Queue board data (now serving, waiting, missed)
`/api/auth/login`	POST	No	✅ 200	Session token and user object
`/api/tickets/register`	POST	reception	✅ 201	Created ticket with QR code
`/api/queue/call-next`	POST	triage/clinician	✅ 200	Called patient details
`/api/dashboard/summary`	GET	triage/clinician	✅ 200	Dashboard metrics
`/api/tickets/:code/track`	GET	No	✅ 200	Tracking data with ETA
`/api/reports/average-wait`	GET	triage/clinician/admin	✅ 200	Average wait report data
`/api/reports/department-performance`	GET	triage/clinician/admin	✅ 200	Department performance data
`/api/reports/no-show`	GET	triage/clinician/admin	✅ 200	No-show report data
`/api/reports/triage`	GET	triage/clinician/admin	✅ 200	Triage report data
`/api/whatsapp/status`	GET	triage/clinician/admin	✅ 200	WhatsApp configuration status

Endpoint	Method	Auth Required	Status	Response
`/api/audit-events`	GET	admin	✅ 200	Audit events with integrity check
`/api/events`	GET (SSE)	No	✅ 200	Event stream connected

5.5 Usability Evaluation

5.5.1 System Usability Scale (SUS)

The System Usability Scale is a 10-item questionnaire providing a global view of subjective assessments of usability. Each item is rated on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).

SUS Questionnaire:

Table 5.20: System Usability Scale (SUS)

#	Question	Type
1	I think that I would like to use this system frequently.	Positive
2	I found the system unnecessarily complex.	Negative
3	I thought the system was easy to use.	Positive
4	I think that I would need the support of a technical person to be able to use this system.	Negative
5	I found the various functions in this system were well integrated.	Positive
6	I thought there was too much inconsistency in this system.	Negative
7	I would imagine that most people would learn to use this system very quickly.	Positive
8	I found the system very cumbersome to use.	Negative
9	I felt very confident using the system.	Positive
10	I needed to learn a lot of things before I could get going with this system.	Negative

Scoring: For positive items, the score contribution is (response − 1). For negative items, the contribution is (5 − response). The sum of contributions is multiplied by 2.5 to produce a SUS score out of 100. A score above 68 is considered above average.

Administration Plan:

- **Target participants:** 5 hospital staff members (1 admin, 1 reception, 2 triage nurses, 1 clinician) and 5 patients
- **Method:** Participants complete a set of representative tasks, then fill in the SUS questionnaire
- **Tasks:** Register a patient, conduct triage assessment, call next patient, track a ticket, view a report

5.5.2 User Feedback Categories

Table 5.21: User Feedback Categories

Category	Evaluation Criteria
Satisfaction	Overall impression, visual design, willingness to use daily
Ease of Use	Learning curve, form completion time, error recovery
Efficiency	Time to complete key tasks vs. manual process
Clarity	Label clarity, navigation intuitiveness, status visibility
Real-Time Experience	Queue update speed, connection reliability, display board readability

5.5.3 Task-Based Evaluation

Table 5.22: Task-Based Evaluation

Task	Target User	Target Time	Success Criteria
Register a new patient	Reception	< 2 minutes	Ticket generated, patient in queue
Search returning patient	Reception	< 30 seconds	Existing record found and linked
Conduct triage assessment	Triage Nurse	< 3 minutes	Assessment saved, priority assigned
Call next patient	Clinician	< 5 seconds	Patient called, queue updated
Track ticket status	Patient	< 15 seconds	Status and position displayed
View average wait report	Admin	< 10 seconds	Report data displayed with metrics
Kiosk self-check-in	Patient	< 1 minute	Token generated, patient in queue

5.6 Performance Analysis

5.6.1 Frontend Bundle Analysis

Table 5.23: Frontend Bundle Analysis

Metric	Value	Assessment
Total modules	2,633	Large but expected for the current full-featured SPA
CSS size (raw)	290.2 KB	Acceptable for Tailwind and the custom design system
CSS size (gzipped)	42.2 KB	Within the working prototype budget
JavaScript size (raw)	2,440.6 KB across 65 files	Split by routes and major dependencies
JavaScript size (gzipped)	660.6 KB in total	Suitable for LAN evaluation; initial-route transfer is lower because chunks load on demand
Total CSS and JavaScript (gzipped)	Approximately 702.8 KB	Acceptable for prototype evaluation; image assets still require optimisation

Optimisation Recommendations:

1. Extend the existing `React.lazy()` route splitting to the heaviest staff modules
2. Tree-shake unused Recharts components (currently imports entire library)
3. Consider lazy-loading the QR scanning library (`html5-qrcode` , 2.3.8) only when needed
4. Optimise background images (BACKG.png) with WebP format and responsive sizing
5. Enable Vite's built-in minification and asset inlining for small images

5.6.2 API Response Times

Table 5.24: API Response Times

Endpoint	Method	Avg Response Time	Assessment
`/api/health`	GET	< 5 ms	Excellent — health check
`/api/meta`	GET	< 20 ms	Excellent — static metadata
`/api/queue/board`	GET	< 50 ms	Good — reads active tickets
`/api/tickets/register`	POST	< 100 ms	Good — creates ticket + notification
`/api/queue/call-next`	POST	< 80 ms	Good — updates ticket + SSE broadcast
`/api/dashboard/summary`	GET	< 100 ms	Good — aggregates metrics
`/api/reports/average-wait`	GET	< 150 ms	Good — date-filtered aggregation
`/api/tickets/:code/track`	GET	< 30 ms	Excellent — single ticket lookup
`/api/audit-events`	GET	< 100 ms	Good — ordered audit query

Note: Response times measured on local development environment with MySQL. Production times on hospital LAN may vary based on network latency and server hardware.

5.6.3 Real-Time Update Latency

Table 5.25: Real-Time Update Latency

Metric	Value	Assessment
SSE event delivery	< 500 ms	Excellent — near-instant on LAN
Client reconnection (on disconnect)	3 seconds	Good — configurable retry interval
Heartbeat interval	25 seconds	Good — keeps connections alive
Polling fallback interval	10 seconds	Acceptable — for browsers without EventSource
Auto-refresh interval	10 seconds (configurable)	Good — ensures wait times stay current

5.6.4 Database Performance

Table 5.26: Database Performance

Operation	Query Pattern	Index Used	Assessment
Fetch waiting tickets by department	`WHERE department = ? AND status = ?`	idx_tickets_department_status	Optimised
Sort tickets by priority and time	`ORDER BY priority, registered_at`	idx_tickets_priority_status	Optimised
Patient search by name	`WHERE full_name LIKE ?`	idx_patient_profiles_name	Optimised
Patient search by phone	`WHERE phone = ?`	idx_patient_profiles_phone	Optimised
Notification lookup by ticket	`WHERE ticket_id = ?`	idx_notifications_ticket_id	Optimised
Audit events by date	`WHERE occurred_at BETWEEN ? AND ?`	idx_audit_events_occurred_at	Optimised
Pending notifications for retry	`WHERE status = 'pending' AND next_retry_at <= NOW()`	idx_notifications_status	Optimised

5.6.5 Low-Bandwidth and Offline Considerations

Table 5.27: Low-Bandwidth and Offline Considerations

Scenario	Behaviour	Status
Hospital LAN operation (no internet)	Core queue management works; WhatsApp notifications queued for retry	✅ Functional
Internet outage during operation	Queue operations continue; notifications stored with “pending” status; retry on reconnection	✅ Functional
Server restart	Queue state persisted in MySQL/file; pending notifications processed on startup	✅ Functional
Client disconnection	SSE auto-reconnects after 3 seconds; polling fallback if EventSource unsupported	✅ Functional
Form draft recovery	Not implemented — data lost on page refresh during form completion	⚠️ Gap

5.7 Security Evaluation

5.7.1 OWASP Top 10 Compliance Checklist

Table 5.28: OWASP Top 10 Compliance Checklist

OWASP Risk	Implementation Status	Evidence
A01: Broken Access Control	✅ Mitigated	Role-based access control via `withStaffAuth` middleware; route-level protection; department scoping for non-admin staff
A02: Cryptographic Failures	✅ Mitigated	bcrypt password hashing (cost 12); SHA-256 audit hash chain; sensitive data redaction in audit events
A03: Injection	✅ Mitigated	Parameterised MySQL queries (mysql2 prepared statements); JSON body parsing with size limits; input validation

OWASP Risk	Implementation Status	Evidence
A04: Insecure Design	⚠️ Partial	Security considered in design; session persistence is memory-only (lost on restart); CORS origin configurable
A05: Security Misconfiguration	⚠️ Partial	Environment-based configuration; CORS defaults to `*` (should be restricted in production); no rate limiting
A06: Vulnerable Components	✅ Low Risk	Minimal dependencies; custom router reduces attack surface; regular npm updates recommended
A07: Identification & Auth Failures	✅ Mitigated	bcrypt hashing; password policy enforcement; session TTL (8 hours); failed login does not reveal user existence
A08: Software & Data Integrity	✅ Mitigated	Audit trail hash chain detects tampering; immutable audit records; append-only storage
A09: Security Logging & Monitoring	✅ Mitigated	Comprehensive audit trail; all operational actions logged with actor attribution; hash chain verification
A10: Server-Side Request Forgery	✅ Low Risk	No user-controlled URL fetching; WhatsApp API calls use fixed base URL

5.7.2 Security Measures Verified

Table 5.29: Security Measures Verified

Measure	Verification Method	Result
Password hashing (bcrypt)	Unit test AUT-05	✅ Hashes match `$2b$12$` pattern; plaintext rejected
Password policy enforcement	Unit test AUT-07	✅ Min 10 chars, 3 character groups required
Staff registration admin guard	Unit test AUT-08	✅ Route protected with `withStaffAuth(["admin"])`
Audit trail admin guard	Unit test AUT-04	✅ Route protected with `withStaffAuth(["admin"])`
Audit hash chain integrity	Unit test AUT-01	✅ Chain valid; hashes are 64-char hex
Audit tamper detection	Unit test AUT-03	✅ Modified event detected; integrity returns false
Sensitive data redaction	Unit test AUT-02	✅ Passwords and tokens redacted; plaintext not in file
RBAC route protection	Manual tests MAN-03, MAN-04, MAN-39	✅ Role-based access enforced
Public display privacy	Manual test MAN-25	✅ No patient names on public displays
Notification consent	Manual test MAN-24	✅ No notifications for non-consenting patients

5.7.3 Remaining Security Recommendations

Table 5.30: Remaining Security Recommendations

Priority	Recommendation	Rationale
High	Implement session persistence (database-backed sessions)	Current in-memory sessions are lost on server restart, requiring all staff to re-login
High	Restrict CORS origin to hospital LAN IP range in production	Default `*` allows any origin; should be locked down

Priority	Recommendation	Rationale
Medium	Add rate limiting on authentication endpoints	Prevents brute-force password attacks
Medium	Implement automatic idle logout	Currently sessions expire by TTL only; no client-side idle detection
Medium	Add security headers (HSTS, X-Content-Type-Options, X-Frame-Options)	Hardens against common browser-based attacks
Low	Implement CSRF protection for state-changing operations	Custom router does not include CSRF tokens; low risk for API-only backend
Low	Regular dependency vulnerability scanning	`npm audit` should be run periodically

---

5.8 Test Coverage Analysis

5.8.1 Requirements Traceability

Table 5.31: Requirements Traceability

Capability group	Evidence available	Residual gap
Registration and returning-patient handling	Automated patient tests and registration walkthroughs	Live master-data and duplicate-resolution policy
Triage and priority queueing	Triage scenarios, priority-order walkthroughs, reassessment UI	Clinical validation of thresholds
Queue actions and public tracking	Call, miss, recall, transfer, tracking, queue-board walkthroughs	Automated multi-browser regression and load testing
Notifications	Mock-provider unit and retry evidence, delivery logs, staff retry controls	Live provider and full background web-push verification
Reports and exports	Report endpoints, dashboard analytics, export controls	Management approval of indicator definitions
Appointments and department modules	Implemented prototype flows for appointments, pharmacy, laboratory, and OI services	Department-specific acceptance testing
Security and privacy	Password, admin-guard, audit-chain tests and anonymous public display	Persistent sessions, rate limiting, security headers, restricted CORS, permission audit
Offline and continuity	LAN architecture, retry queues, local form-draft recovery	Automated backup/restore and hybrid synchronisation

5.8.2 Test Type Coverage

Table 5.32: Test Type Coverage

Test Type	Coverage	Assessment
Unit tests (backend)	12 tests covering audit, auth, patient registration	Good for critical paths; should expand to cover queue operations, triage, notifications
Integration tests	Manual API endpoint verification (15 endpoints)	Good coverage of key endpoints; should automate

Test Type	Coverage	Assessment
Functional tests (UI)	39 manual test cases across 9 modules	Comprehensive; should automate with Playwright/Cypress
Performance tests	Bundle analysis, API timing, SSE latency	Baseline established; should add load testing
Security tests	OWASP checklist, 10 verified measures	Good baseline; should add penetration testing
Usability tests	SUS questionnaire designed; task-based evaluation planned	Not yet administered with a representative hospital sample

5.9 Interpretation of Results Against the Problem

Table 5.33: Interpretation of Results Against the Problem

Problem identified in Chapter 1	Prototype result	Strength of conclusion
Patient movement was not visible across departments	One ticket could be registered, triaged, transferred, tracked, and displayed through shared queue state	Strong functional evidence; operational effect still requires a pilot
Clinical urgency could be obscured by arrival order	Priority-aware ordering and emergency-trigger scenarios passed the documented checks	Strong software evidence; thresholds require clinical approval
Patients could miss turns without notification	The system recorded notification events, retries, missed turns, and recalls and exposed them to staff	Strong workflow evidence in mock mode; live delivery remains deployment-dependent
Management lacked reliable queue information	Dashboard metrics, audit history, reports, and exports were produced from the same event data	Strong prototype evidence; indicator definitions require management validation
Paper-based work was vulnerable to loss and disconnection	Persistent queue repositories, LAN deployment, retry queues, and local drafts were implemented	Partial readiness; backup, restore, and failover testing remain outstanding

The results therefore showed that the software artefact addressed the coordination mechanisms at the centre of the research problem. They did not yet prove that hospital-wide waiting time, satisfaction, or clinical outcomes had improved. Those outcomes require baseline data, an authorised pilot, and repeated measurement after deployment.

5.10 Summary

This chapter presented the verification and evaluation results for WaitLess. The evidence included 12 passing automated backend tests, 39 successful controlled functional walkthroughs across nine module groups, a successful frontend production build, and verification of key API paths. The breadth of the walkthroughs was useful for demonstration, while the small automated suite remained a clear area for improvement.

Performance observations established a local-development baseline for bundle size, API response behaviour, and SSE updates. These observations were not a substitute for hospital-network load testing. The security review identified a useful prototype foundation alongside specific production hardening work, including persistent sessions, restricted CORS, rate limiting, security headers, backup controls, and broader permission testing.

The usability evaluation framework, including the SUS questionnaire and task-based evaluation, was prepared for administration with authorised hospital staff and patients. Because that representative exercise had not yet been completed, no invented SUS score was reported. The final interpretation was therefore appropriately limited to prototype feasibility and pilot readiness.

The next chapter, Chapter 6: Summary, Maintenance and Recommendations, summarises the project achievements, limitations, and future work.

Chapter 6: Summary, Maintenance and Recommendations

6.1 Introduction

This chapter concludes the documentation of the WaitLess Digital Queue Management System by summarising the project's achievements, assessing the system against its original objectives, identifying limitations, and presenting recommendations for maintenance and future work. The discussion is grounded in the implemented prototype, source-code review, automated tests, documented functional walkthroughs, and production-build verification. Claims about hospital outcomes are deliberately separated from claims about software behaviour.

6.2 Project Summary

6.2.1 Problem Addressed

Chinhoyi Provincial Hospital, like many public healthcare facilities in Zimbabwe, relied on manual queue management — physical tokens, paper registers, and verbal calls — resulting in long waits, patient anxiety, no-shows, and operational inefficiency. The WaitLess system was proposed to replace this with a real-time, web-based digital queue management platform.

6.2.2 Solution Delivered

WaitLess is a full-stack web application built with React 19 and Node.js, featuring:

- **Real-time queue management** with Server-Sent Events (SSE) for instant updates across all connected devices
- **Role-based access control** for admin, reception, triage, and clinician staff roles
- **Patient registration** with returning-patient search, QR card generation, and unique department-scoped tokens
- **Triage assessment** with emergency vital auto-classification and manual override
- **Priority-based queue ordering** (Red > Yellow > Green) with longest-wait tie-breaking
- **Queue operations** including call-next, mark missed, recall, and department transfers
- **WhatsApp Cloud API integration** with retry logic and consent-based notifications
- **Patient self-service** including self-registration, kiosk check-in, ticket tracking, and patient portal
- **Public and department-specific display boards** showing now-serving, waiting, and missed columns
- **Reporting** with average wait time, department performance, no-show, and triage reports

- **Extended hospital workflows** for appointments, pharmacy stock and dispensing, laboratory orders, and OI service streams
- **Accessible queue communication** through anonymous boards, audio calls, multilingual interface support, and browser notification controls
- **Administrative tools** for staff profiles, profile images, system settings, exports, and operational navigation
- **Immutable audit trail** with SHA-256 hash chain for tamper detection
- **Dual-mode storage** supporting both MySQL and file-based JSON persistence

6.2.3 Technical Architecture Summary

Table 6.1: Technical Architecture Summary

Layer	Technology	Key Decisions
Frontend	React 19, Vite 7, Tailwind CSS 4, Radix UI, Lucide icons	SPA with lazy-loaded routes, context-based state, custom hooks for API access
Backend	Node.js (ESM), custom HTTP router, no framework	Lightweight router with path parameters; avoids Express overhead
Real-time	Server-Sent Events (SSE)	Chosen over WebSocket for unidirectional simplicity; polling fallback
Database	MySQL (mysql2) or JSON files	Dual-mode repository pattern; file mode for development/testing
Security	bcrypt, RBAC, audit hash chain	bcrypt cost 12; session tokens; role-expansion middleware
Notifications	WhatsApp Cloud API v23.0	Retry with exponential backoff; mock provider for testing

6.3 Achievement of Objectives

6.3.1 Original Objectives Assessment

Table 6.2: Original Objectives Assessment

Objective	Status	Evidence
Replace manual queue with digital system	✅ Achieved	Full registration-to-completion workflow implemented
Real-time queue updates	✅ Achieved	SSE with < 500 ms latency; polling fallback
Support reduction of patient wait times	✅ Functionally supported	Priority ordering, ETA display, tracking, bottleneck visibility; a live hospital pilot is still required to measure the actual reduction
Improve patient experience	✅ Implemented at prototype level	Self-registration, kiosk, tracking, queue boards, audio calls, and notification workflows; representative user evaluation remains outstanding
Role-based staff access	✅ Achieved	Four roles with route-level and data-level enforcement
Triage integration	✅ Achieved	Triage assessment with emergency auto-classification
WhatsApp notifications	⚠️ Deployment-dependent	Cloud API adapter, logs, and retry workflow were implemented; live provider verification requires approved production credentials

Objective	Status	Evidence
Reporting and analytics	✅ Achieved	Operational, wait-time, department, triage, notification, and export workflows implemented
Audit trail	✅ Achieved	Hash-chained, tamper-detectable, admin-only access
Constrained-network operation	⚠️ Partial	Core LAN workflows, retry queues, and local draft recovery were implemented; hybrid local/cloud synchronisation remains future work

6.3.2 Requirements Coverage at Documentation Freeze

The earlier progress audit was useful while the prototype was being built, but its numerical percentage became stale as extended modules were added. The final assessment therefore reports capability coverage and remaining deployment risk instead of presenting an outdated weighted score.

Table 6.3: Requirements Coverage at Documentation Freeze

Capability group	Final assessment	Remaining qualification
Registration, returning-patient handling, and ticketing	Implemented	Requires live master-data governance
Triage, reassessment, and priority queueing	Implemented	Clinical rules require hospital approval before deployment
Queue actions, transfer, recall, tracking, and displays	Implemented	Concurrent load testing remains outstanding
Notifications and delivery analytics	Partially production-ready	Live WhatsApp and full push delivery require provider credentials and deployment verification
Reports and exports	Implemented	Report definitions require management sign-off
Appointments, pharmacy, laboratory, and OI workflows	Implemented as prototype modules	Requires department-specific acceptance testing
Authentication, role access, and audit history	Implemented with hardening work remaining	Persistent sessions, rate limiting, security headers, and a restricted CORS policy are still required
Offline and operational continuity	Partially implemented	Backup automation, restore drills, and local/cloud synchronisation remain outstanding

The prototype therefore covered the intended academic scope and supported an end-to-end demonstration, while production deployment remained conditional on operational, security, and user-acceptance work.

6.3.3 Key Achievements

1. **Functional queue management core** — Registration, priority ordering, calls, misses, recalls, and transfers all work end-to-end
2. **Real-time architecture** — SSE provides near-instant updates; polling fallback ensures compatibility
3. **Security foundation** — bcrypt hashing, RBAC, password policy, immutable audit trail with hash chain verification
4. **Dual-mode persistence** — Repository pattern abstracts MySQL and file storage, enabling development without database setup

5. **Patient self-service** — Self-registration, kiosk check-in, ticket tracking, and patient portal reduce reception workload
6. **Automated test coverage** — 12 unit tests covering critical security and patient-handling paths, all passing
7. **Production build verified** — Frontend compiles successfully (2,633 modules transformed on 10 August 2026); backend starts and serves the documented endpoints

---

6.4 Limitations

6.4.1 Technical Limitations

Table 6.4: Technical Limitations

Limitation	Impact	Severity
In-memory session storage	All staff sessions lost on server restart; users must re-login	High
No rate limiting on auth endpoints	Susceptible to brute-force attacks	Medium
CORS configured as `*`	Allows any origin; must be restricted for production	High
No security headers (HSTS, X-Frame-Options)	Missing browser-level protections	Medium
Frontend bundle > 2 MB (raw)	Slower initial load on low-bandwidth connections	Medium
Some operational modules retain JSON-backed persistence	Data management is split across persistence mechanisms	Medium
ETA uses fixed formula, not predictive	Less accurate during unusual load patterns	Low
No automated UI tests	Frontend regressions detected manually only	Medium

6.4.2 Functional Limitations

Table 6.5: Functional Limitations

Limitation	Impact	Severity
WhatsApp delivery unverified with live credentials	Notification reliability unknown in production	High
Full web-push backend delivery is incomplete	Browser controls exist, but background delivery is not production-ready	Medium
Department-level permission rules need full acceptance testing	A role may receive broader access than intended if configuration is wrong	High
Advanced wait prediction uses limited prototype data	Estimates may be inaccurate during unusual demand	Medium
No feedback collection	No patient satisfaction data captured	Low
No staff shift management	Workload distribution not tracked	Low
No multi-hospital tenancy	System supports single hospital only	Low
Translation coverage requires clinical review	Some translated labels may need terminology approval	Low

6.4.3 Operational Limitations

Table 6.6: Operational Limitations

Limitation	Impact	Severity
No hybrid cloud/local sync	Cannot failover between local and cloud deployment	Medium
No automated backup/restore	Data loss risk if manual backups not performed	High
Deployment guidance has not been validated by a hospital IT team	The documented runbook may need local environment changes	Medium
No load testing performed	System behaviour under concurrent load unknown	Medium
Usability testing not yet administered	SUS scores and user feedback not collected	Medium

6.5 Lessons Learned

6.5.1 Technical Lessons

1. **Custom router over Express** — Building a lightweight HTTP router (92 lines) provided full control over request processing, path parameter extraction, and middleware composition. This avoided framework overhead and deepened understanding of Node.js HTTP internals. However, it also meant missing built-in features like rate limiting, request validation, and OpenAPI documentation that frameworks provide.

2. **SSE over WebSocket** — Server-Sent Events proved sufficient for the unidirectional update pattern used by the prototype. EventSource provided automatic reconnection and a simpler client model. The trade-off was the absence of client-to-server streaming, which the queue workflow did not require.

3. **Dual-mode repository pattern** — Abstracting storage behind a repository interface with file and MySQL implementations was valuable for development and testing. Tests run against file storage without database setup, while production uses MySQL. The pattern added minimal complexity and significant flexibility.

4. **Hash-chained audit trail** — Using SHA-256 hash chaining for audit events provides cryptographic tamper detection without external dependencies. The implementation redacts sensitive fields before storage and verifies chain integrity on every read. This was a high-value security feature with modest implementation effort.

5. **Context-based state management** — React Context with custom hooks (`useApiResource` , `useApiAction` , `useLiveRefresh`) provided clean state management without Redux. The pattern works well for the system's complexity level but may need restructuring if the application grows significantly.

6.5.2 Project Management Lessons

1. **Requirements traceability** — Maintaining a feature audit matrix helped prevent requirements from being silently dropped. The experience also showed that status counts had to be refreshed before publication; a percentage without a current evidence date could quickly become misleading.

2. **Iterative development** — Building the core queue management loop first (register → triage → call → complete) provided a demonstrable MVP early, allowing stakeholder feedback before expanding scope.
3. **Testing early** — Implementing automated tests for security-critical paths (password hashing, audit integrity, RBAC) before feature expansion caught regressions and provided confidence in the security foundation.
4. **Documentation alongside development** — Writing documentation after implementation, rather than before, ensured accuracy. However, this meant documentation lagged behind code changes, creating a maintenance burden.

6.6 System Maintenance and Recommendations

6.6.1 Immediate Actions (Pre-Production)

These items must be completed before deploying WaitLess in a real hospital environment:

Table 6.7: Immediate Actions (Pre-Production)

Priority	Recommendation	Rationale	Estimated Effort
P0	Restrict CORS origin to hospital LAN IP range	Security — prevents cross-origin attacks	1 hour
P0	Implement database-backed session persistence	Reliability — sessions survive server restart	1–2 days
P0	Verify WhatsApp Cloud API with live credentials	Functionality — confirms notification delivery	1 day
P0	Validate the deployment runbook and complete a backup/restore drill	Operations — proves recoverability in the target environment	1–2 days
P1	Add rate limiting on `/api/auth/login`	Security — prevents brute-force attacks	4 hours
P1	Add security headers (HSTS, X-Content-Type-Options, X-Frame-Options)	Security — hardens browser protections	2 hours
P1	Conduct load testing (50+ concurrent users)	Reliability — validates performance under load	1–2 days
P1	Administer SUS questionnaire with hospital staff	Usability — collects user feedback before deployment	1 week
P2	Implement automated database backups	Reliability — prevents data loss	1 day
P2	Add automated UI tests (Playwright or Cypress)	Quality — catches frontend regressions	3–5 days

6.6.2 Short-Term Pilot Work (Next 3 Months)

Table 6.8: Short-Term Pilot Work (Next 3 Months)

Priority	Recommendation	Rationale
P1	Run department-specific acceptance tests for appointments, laboratory, pharmacy, and OI workflows	Confirms that prototype states match real departmental practice
P1	Complete persistent session storage and department-level permission tests	Prevents avoidable account and access-control failures

Priority	Recommendation	Rationale
P1	Complete production web-push configuration or formally remove it from the pilot scope	Keeps the deployed promise aligned with actual capability
P2	Review Shona and Ndebele clinical terminology with hospital staff	Prevents ambiguous or unsafe wording
P2	Validate report definitions and exports with hospital management	Ensures that indicators support actual decisions
P2	Add automated end-to-end browser tests for the critical patient journey	Reduces regression risk across pages
P3	Pilot patient feedback collection with an approved consent process	Provides evidence for usability and satisfaction
P3	Add staff shift and workload analysis	Supports staffing review after reliable operational data exist

6.6.3 Medium-Term Enhancements (3–6 Months)

Table 6.9: Medium-Term Enhancements (3–6 Months)

Recommendation	Rationale
Implement configurable workflows and rules at runtime	Allows hospital administrators to adjust thresholds, departments, and routing without code changes
Add hybrid cloud/local deployment with sync	Enables failover between local server and cloud; ensures continuity during network outages
Implement automated offline/reconnect testing	Validates system behaviour during internet outages and recovery
Add code splitting and bundle optimisation	Reduces initial load time for low-bandwidth connections
Implement idle logout with client-side detection	Enhances security by logging out inactive sessions
Add CSRF protection for state-changing operations	Defends against cross-site request forgery attacks
Complete and monitor standards-based web push delivery	Provides an additional channel beyond WhatsApp

6.6.4 Long-Term Vision (6–12 Months)

Table 6.10: Long-Term Vision (6–12 Months)

Recommendation	Rationale
Multi-hospital tenancy with per-facility configuration	Enables deployment across multiple hospitals with data isolation
Multi-language support (Shona, Ndebele, French)	Serves diverse patient populations in Zimbabwe and beyond
AI-based waiting time prediction	Uses historical service data to predict wait times more accurately than fixed formula
AI-based bottleneck detection	Identifies congestion patterns and recommends staffing adjustments
Patient chatbot for FAQs and navigation	Reduces reception workload; guides patients through self-service
Integration with Hospital Information Systems (HIS)	Connects queue management with patient records, billing, and clinical systems
Mobile application (React Native)	Provides native mobile experience for patients and staff
FHIR-compatible API layer	Enables interoperability with standards-compliant healthcare systems

6.7 Future Research Directions

1. **Predictive queue modelling** — Applying machine learning to historical service-time data to predict wait times, detect anomalies, and recommend staffing adjustments. This requires sufficient clean historical data (6+ months of operation).
2. **Patient flow optimisation** — Analysing queue data to identify systemic bottlenecks, optimal department routing, and staffing patterns. Research could apply queueing theory models (M/M/c, priority queues) to the hospital's actual data.
3. **Digital health equity** — Studying the impact of digital queue management on patient outcomes across demographic groups (elderly, rural, low-literacy) to ensure the system does not exacerbate health disparities.
4. **WhatsApp API effectiveness** — Measuring the impact of WhatsApp notifications on no-show rates, patient satisfaction, and queue efficiency compared to the pre-digital baseline.
5. **Offline-first architecture patterns** — Researching robust sync mechanisms for healthcare applications that must operate in environments with intermittent internet connectivity, particularly in resource-limited settings.

6.8 Project Significance

The WaitLess Digital Queue Management System demonstrates that modern web technologies can address real healthcare operational challenges in resource-limited settings. Key contributions include:

1. **Practical solution for Zimbabwean healthcare** — Designed specifically for Chinhoyi Provincial Hospital's constraints (intermittent internet, limited IT staff, LAN deployment), the system is adaptable to similar facilities across the region.
2. **Lightweight architecture** — The custom router, SSE-based real-time, and dual-mode storage demonstrate that full-featured healthcare applications can be built without heavy frameworks, reducing complexity and maintenance burden.
3. **Security by design** — The hash-chained audit trail, bcrypt password hashing, and role-based access control establish a security foundation that can be extended as the system grows.
4. **Scalable foundation** — The modular architecture (services, repositories, controllers, contexts) supports incremental feature addition without requiring rewrites.
5. **Documentation as a first-class deliverable** — This documentation set (Chapters 1–6) provides a complete record of the system's analysis, design, implementation, testing, and evaluation, serving as both an academic submission and a technical reference for future developers.

6.9 Conclusion

The WaitLess Digital Queue Management System was conceived to replace manual queue management at Chinhoyi Provincial Hospital with a real-time, web-based platform. The system successfully delivers a functional queue management core — patient registration, triage assessment, priority-based ordering, real-time updates, WhatsApp notifications, patient self-service, display boards, reporting, and audit trail — built on a modern technology stack (React 19, Node.js, MySQL, SSE).

At the documentation freeze, the core operational workflow was functional and had been verified through 12 passing automated backend tests, a documented set of 39 functional walkthroughs, and a successful frontend production build. Extended modules for appointments, pharmacy, laboratory work, OI services, reports, multilingual presentation, audio calls, profile management, and offline draft recovery had also been added. The manual walkthroughs remained repeatable acceptance evidence rather than a substitute for automated end-to-end testing or formal hospital UAT.

The system was not yet ready for unsupervised use with real patient data. Critical work remained in CORS restriction, persistent sessions, rate limiting, security headers, live WhatsApp verification, web-push completion, backup and restore testing, load testing, and formal usability evaluation. Several extended modules also required department-specific acceptance testing and consolidation into the production persistence model.

Nevertheless, the architecture provides a solid, extensible foundation. The recommendations in this chapter outline a clear path from the current state to a production-ready system, followed by short-term functional enhancements and a long-term vision for multi-hospital deployment, AI-assisted optimisation, and HIS integration.

The WaitLess system demonstrates that thoughtful application of modern web technologies, combined with a deep understanding of the operational context, can produce meaningful improvements in healthcare service delivery — even in resource-limited environments.

References

Aggarwal, S. (2018) 'Modern web-development using ReactJS', *International Journal of Recent Research Aspects*, 5(1), pp. 133-137.

Bidari, A., Jafarnejad, S. and Alaei Faradonbeh, N. (2021) 'Effect of queue management system on patient satisfaction in emergency department: A randomized controlled trial', *Archives of Academic Emergency Medicine*, 9(1), e59. doi: 10.22037/aaem.v9i1.1335.

Bisandu, D.B. (2019) 'Design science research methodology in computer science and information systems', *International Journal of Information Technology*, pp. 1-7.

Brooke, J. (1996) 'SUS: A quick and dirty usability scale', in Jordan, P.W., Thomas, B., McClelland, I.L. and Weerdmeester, B. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189-194.

Catwell, L. and Sheikh, A. (2009) 'Evaluating eHealth interventions: The need for continuous systemic evaluation', *PLoS Medicine*, 6(8), e1000126. doi: 10.1371/journal.pmed.1000126.

Cayirli, T. and Veral, E. (2003) 'Outpatient scheduling in health care: A review of literature', *Production and Operations Management*, 12(4), pp. 519-549. doi: 10.1111/j.1937-5956.2003.tb00218.x.

Cho, K.W., Kim, S.M., Chae, Y.M. and Song, Y.U. (2017) 'Application of queueing theory to the analysis of changes in outpatients' waiting times in hospitals introducing EMR', *Healthcare Informatics Research*, 23(1), pp. 35-42. doi: 10.4258/hir.2017.23.1.35.

Fomundam, S. and Herrmann, J.W. (2007) *A Survey of Queuing Theory Applications in Healthcare*. College Park, MD: University of Maryland, Institute for Systems Research.

Furusa, S.S. and Coleman, A. (2018) 'Factors influencing e-health implementation by medical doctors in public hospitals in Zimbabwe', *South African Journal of Information Management*, 20(1), a928. doi: 10.4102/sajim.v20i1.928.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75-105. doi: 10.2307/25148625.

Kanchanawanishkul, K., Nooboon, T., Deemak, T. and Kongcharoen, N. (2018) 'A queue management system for a hospital: A wearable Q', *Journal of Engineering and Digital Technology*, 6(2), pp. 24-34.

Ministry of Health and Child Care, Zimbabwe (2021) *National Health Strategy 2021-2025*. Harare: Government of Zimbabwe.

Mwanswila, M.J., Mollel, H.A. and Mushi, L.D. (2024) 'Outcome evaluation of technical strategies on reduction of patient waiting time in the outpatient department at Kilimanjaro Christian Medical Centre, Northern Tanzania', *BMC Health Services Research*, 24, 799. doi: 10.1186/s12913-024-11231-5.

Peffers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2007) 'A design science research methodology for information systems research', *Journal of Management Information Systems*, 24(3), pp. 45-77. doi: 10.2753/MIS0742-1222240302.

Preater, J. (2002) 'Queues in health', *Health Care Management Science*, 5(4), p. 283. doi: 10.1023/A:1020334207282.

Ross, J., Stevenson, F., Lau, R. and Murray, E. (2016) 'Factors that influence the implementation of e-health: A systematic review of systematic reviews', *Implementation Science*, 11, 146. doi: 10.1186/s13012-016-0510-7.

Scott, R.E. and Mars, M. (2013) 'Principles and framework for eHealth strategy development', *Journal of Medical Internet Research*, 15(7), e155. doi: 10.2196/jmir.2250.

Tilkov, S. and Vinoski, S. (2010) 'Node.js: Using JavaScript to build high-performance network programs', *IEEE Internet Computing*, 14(6), pp. 80-83. doi: 10.1109/MIC.2010.145.

World Health Organization (2019) *WHO Guideline: Recommendations on Digital Interventions for Health System Strengthening*. Geneva: World Health Organization.

Technical Documentation

Meta Platforms (2025) *WhatsApp Cloud API Documentation*. Available at: https://developers.facebook.com/docs/whatsapp/cloud-api (Accessed: 10 August 2026).

Mozilla Developer Network (2025) *Server-Sent Events*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events (Accessed: 10 August 2026).

OWASP Foundation (2023) *OWASP Top 10*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 10 August 2026).

React (2025) *React Documentation*. Available at: https://react.dev (Accessed: 10 August 2026).

Appendices

Appendix A: Submission Deliverables

Table A.1: Appendix A: Submission Deliverables

Deliverable	Location or form	Purpose
Frontend source code	`WAITLESS/frontend/`	Patient, staff, administration, queue-board, and reporting interfaces
Backend source code	`WAITLESS/backend/`	Authentication, queue, triage, notification, reporting, audit, and department services
Database schema and repositories	Backend data and repository modules	Persistent representation of patients, tickets, notifications, users, and audit events
Automated tests	`WAITLESS/backend/test/`	Repeatable verification of selected critical paths
Deployment guide	`WAITLESS/DEPLOYMENT.md`	Environment, startup, and hosting guidance
Patient user manual	Included as Appendix D in the complete PDF	Guidance for registration, login, tracking, and notifications
Staff user manual	Included as Appendix E in the complete PDF	Guidance for queue, triage, department, report, and administration workflows
System diagrams	Embedded in Chapters 3 and 4	Visual record of research, architecture, data, and workflow design

Appendix B: Verification Command Summary

Table A.2: Appendix B: Verification Command Summary

Verification	Command	Expected evidence
Backend automated tests	`npm test` from `WAITLESS/backend`	Test suites complete without failures
Frontend production build	`npm run build` from `WAITLESS/frontend`	Vite production assets generated successfully
Frontend lint	`npm run lint` from `WAITLESS/frontend`	No lint errors; warnings reviewed separately
Backend startup	`npm start` from `WAITLESS/backend`	API server listens on the configured port

Appendix C: Pre-Deployment Acceptance Checklist

- Obtain formal approval for pilot use and data handling.
- Replace development secrets with managed production secrets.
- Restrict CORS to approved hospital origins.
- Implement persistent session storage and test logout and expiry behaviour.

- Add authentication rate limiting and browser security headers.
- Verify department-level permissions for every staff role.
- Configure and verify the approved WhatsApp provider.
- Complete or remove web-push promises from the pilot scope.
- Configure encrypted backups and complete a restore drill.
- Conduct load testing on the intended hospital network and hardware.
- Complete staff UAT and the approved patient usability exercise.
- Review triage thresholds, translated clinical wording, and report definitions with authorised hospital staff.
- Train users and record incident, support, and escalation procedures.

Appendix D: Patient and User Manual

The complete patient and user manual follows this appendix cover in the consolidated submission PDF.

Appendix E: Staff User Manual

The complete staff manual follows the patient guide in the consolidated submission PDF.

W

PATIENT & USER GUIDE

WaitLess

Digital Hospital Queue Management System

Chinhoyi Provincial Hospital

Replacing confusion with a queue patients can trust.

Version 1.0 | August 2026

What's Inside

Getting Started • Tracking Your Queue • WhatsApp Updates

Booking Appointments • Giving Feedback • Language Selection

Table of Contents

1. **Welcome to WaitLess** — What this system does for you
2. **Getting Started** — Three ways to register
3. **Tracking Your Queue Position** — Know your wait time
4. **WhatsApp Notifications** — Stay informed on your phone
5. **Triage Priority System** — How patients are ordered
6. **Screens You Will See** — What's displayed where
7. **Booking an Appointment** — Schedule in advance
8. **Leaving and Returning** — You don't have to sit all day
9. **Giving Feedback** — Help improve the hospital
10. **Changing Language** — English, chiShona, isiNdebele
11. **Troubleshooting** — Common questions answered

1. Welcome to WaitLess

WaitLess replaces the old paper-ticket queue system with a digital one that puts you in control. Instead of sitting in a corridor for hours with no idea when you'll be seen, WaitLess lets you:

- **See your live queue position** on your phone at any time
- **Receive WhatsApp messages** when your turn is approaching
- **Leave the waiting area** and return when it's almost your turn
- **Track your ticket** from registration through to completion
- **Book appointments** online without coming to the hospital
- **Give feedback** on your hospital experience

> **Key benefit:** You no longer need to sit in the waiting area for hours. Register, track your position on your phone, and return when you receive a WhatsApp message saying your turn is approaching.

2. Getting Started — Three Ways to Register

Option A: At the Reception Desk

This is the most common way to register, especially for first-time visitors.

1 **Go to the reception desk**

Tell the receptionist you need to see a doctor. They will search for your existing record or create a new one.

2 **Provide your details**

The receptionist will capture: your full name, national ID, phone number, address, date of birth, gender, next of kin (name and phone), what's wrong (chief complaint), and which department you need.

3 **Give WhatsApp consent**

If you have WhatsApp on your phone, tell the receptionist yes — you'll receive queue updates automatically.

4 **Receive your ticket**

You'll get a printed ticket with a code like **OPD-205** and a QR code. You can also have it shared to your phone.

Option B: Self-Registration from Your Phone

If you have a smartphone, you can register before you even arrive at the hospital.

1 **Open the hospital app**

Visit the WaitLess app URL on your phone browser. Tap "Get started today".

2 **Fill in your details**

Enter your name, phone number, and select the department you need. Describe what's wrong.

3 **Get your ticket number**

You'll receive a digital ticket number immediately. When you arrive at the hospital, go straight to triage — no need to queue at reception.

Option C: Kiosk Check-In at the Hospital Entrance

A self-service kiosk tablet is available at the hospital entrance for returning patients.

1 **Scan your QR code or enter your phone number**

Point your patient QR code at the kiosk camera, or type in your phone number.

2 **Select department and reason**

Choose which department you need and briefly describe your reason for visiting.

3 **Get your ticket**

The kiosk displays your ticket number. Proceed to the waiting area.

3. Tracking Your Queue Position

Once you have your ticket code (e.g., **OPD-205**), you can track your position in the queue at any time from your phone.

1 **Open the tracking page**

Visit the tracking page (the URL is printed on your ticket, or scan the QR code on your ticket with your phone camera).

2 **Enter your ticket code**

Type in your ticket code, e.g., **OPD-205**. The page updates in real time — no need to refresh.

3 **See your live status**

You will see:

- **Current position** — how many patients are ahead of you
- **Estimated wait time** — how long until you're called
- **Current stage** — Registered, Triage, Waiting, Called, or Served
- **Department directions** — where to go next
- **Notification history** — messages sent to you

Did you know? The tracking page works on any phone with a browser — no app download needed. It even works on slow internet connections.

4. WhatsApp Notifications

If you gave consent during registration and provided your WhatsApp number, you will automatically receive these messages:

When	Message you receive
At registration	"You are registered. Your ticket is OPD-205. Track your queue at [URL]"
2-3 patients before your turn	"Your turn is approaching. Please return to the OPD waiting area."
When you are called	"OPD-205, please proceed to Room 3."
If you miss your turn	"You missed your turn. Please see the reception desk to be recalled."
If you are transferred	"You have been moved to the Laboratory queue. Your new ticket is LAB-206."
Prescription ready	"Your prescription is ready at the Pharmacy window."
Lab results ready	"Your lab results are ready. Please return to the doctor."

> **Important:** Make sure your phone number is correct during registration. If you don't have WhatsApp, you can still track your queue position using the tracking page.

5. Triage Priority System

After registration, a triage nurse will assess your condition and assign a priority level. This determines the order in which patients are seen — it is **not** first-come, first-served. Patients with more urgent conditions are seen first.

RED — Resuscitation

Life-threatening. Seen immediately, ahead of everyone else.

YELLOW — Urgent

Should be seen within 60 minutes. High priority.

GREEN — Standard

Routine cases. Seen in order of arrival within this group.

BLACK — Deceased

No further treatment possible.

> **If you feel your condition is getting worse** while waiting, tell the triage nurse immediately. The system also automatically alerts nurses if Red or Yellow patients have been waiting too long.

6. Screens You Will See at the Hospital

Main Waiting Area — Public Queue Board

A large TV screen in the main waiting area shows the current queue status for all departments:

```
── WAITLESS QUEUE BOARD ──

NOW SERVING: OPD-203

NEXT: OPD-204, OPD-205, OPD-206
MISSED: OPD-199 (see reception)
```

This screen shows **ticket numbers only** — no patient names, for privacy.

Department Waiting Areas — Department Board

Each department (Pharmacy, Lab, Radiology, etc.) has its own screen showing only that department's queue:

```
── PHARMACY QUEUE ──

NOW SERVING: PHA-102

NEXT: PHA-103, PHA-104, PHA-105
```

Your Phone — Tracking Page

On your phone, you see your personal queue status with position, wait time, and directions to your next department.

7. Booking an Appointment

You can book an appointment in advance through the patient portal on your phone.

1. **Log in to the patient portal**

   Open the WaitLess app and log in with your username and password. If you don't have an account, tap "Register" to create one.

2. **Go to "Book Appointment"**

   Tap the "Book Appointment" tab in the patient portal menu.

3. **Select date and service**

   Choose a date and the type of service you need. Available time slots will appear.

4. **Book your slot**

   Tap an available slot and confirm. You'll receive a reminder before your appointment.

> **Need to cancel?** Go to "Book Appointment" in the portal, find your appointment, and tap "Cancel".

8. Leaving and Returning

One of the biggest benefits of WaitLess is that you **do not need to sit in the waiting area the entire time**.

1. **Check your position on your phone**

   Open the tracking page and see how many patients are ahead of you.

2. **Step out if you have time**

   If there are several patients ahead, you can get fresh air, use the restroom, or get something to eat.

3

Return when you get the WhatsApp message

You'll receive a "Your turn is approaching" message 2-3 patients before your turn. Return to the waiting area promptly.

If you miss your turn: Don't panic. Go to the reception desk and let them know. You will be recalled and reinserted into the queue fairly.

9. Giving Feedback

After your visit is complete, you can rate your experience to help the hospital improve.

1. **Log in to the patient portal**

   Open the WaitLess app and log in.

2. **Go to "Feedback"**

   Tap the "Feedback" tab. You'll see your completed visits that are ready for feedback.

3. **Rate your experience**

   Rate: overall satisfaction, cleanliness, communication, and wait time. Submit your feedback.

10. Changing Language

WaitLess supports three languages. You can switch at any time:

1. **Tap the language selector**

   Look for the language button in the top navigation bar.

2. **Choose your language**

   Select from: **English**, **chiShona**, or **isiNdebele**.

3. **The entire interface switches**

   All labels, instructions, and messages will now appear in your chosen language.

11. Troubleshooting

I can't find the tracking page

The URL is printed on your ticket. You can also scan the QR code on your ticket with your phone camera, which will open the tracking page automatically.

I'm not receiving WhatsApp messages

- Make sure you gave consent during registration
- Check that your phone number was entered correctly
- Ensure you have an active WhatsApp account on that number
- If the hospital internet is down, messages will be sent when it comes back

I missed my turn — what do I do?

Go to the reception desk and tell them your ticket number. They will recall you and put you back in the queue fairly.

The tracking page says "Unable to reach the server"

This means the hospital server is temporarily unavailable. Try again in a few moments. The queue board screens in the waiting area will still show your position.

I don't have a smartphone

You can still use WaitLess. The reception desk will give you a printed ticket, and the TV screens in the waiting area show the live queue. If you have WhatsApp on a basic phone, you'll still receive messages.

I need to see a different department

Tell the doctor or nurse. They can transfer you to another department queue (e.g., from OPD to Laboratory). You'll get a new ticket number and a WhatsApp message telling you where to go.

W

STAFF & ADMINISTRATOR GUIDE

WaitLess

Digital Hospital Queue Management System

Chinhoyi Provincial Hospital

Complete operational manual for all staff roles.

Version 1.0 | August 2026

What's Inside

Login & Security • Reception • Triage • Clinician Workflow

Pharmacy • Laboratory • OI Clinic • Admin • Reports

Notifications • Display Boards • Troubleshooting

Table of Contents

1. **System Overview** — How WaitLess works end to end
2. **Logging In & Security** — Account access and roles
3. **For Reception Staff** — Patient registration and search
4. **For Triage Nurses** — Assessment and priority assignment
5. **For Clinicians** — Calling patients, transfers, test orders
6. **For Pharmacists** — Prescription workflow and stock
7. **For Lab Technicians** — Test order lifecycle
8. **For OI Clinic Staff** — Confidential service streams
9. **For Administrators** — Staff management, audit, config
10. **Reports & Analytics** — 10 report types with exports
11. **Notifications & Display Boards** — WhatsApp, push, screens
12. **Appointments** — Booking and follow-up management
13. **Troubleshooting** — Common issues and solutions

1. System Overview

WaitLess is a digital queue management system that replaces paper tickets with real-time digital tracking. Patients register, get triaged, wait in a virtual queue, receive WhatsApp notifications, and are called by staff — all managed through a web-based interface accessible on desktops, tablets, and phones.

The Patient Journey

```
—— PATIENT FLOW ——

Registration → Triage Assessment → Queue Waiting → Called by Clinician
→ Consultation → [Transfer to Lab/Pharmacy/Radiology if needed]
→ Completion → Feedback
```

Key Components

- **Backend server** — Node.js, runs on hospital LAN (port 4242)
- **Frontend** — React web app, works in any modern browser
- **Database** — MySQL for production, file-based JSON for development
- **Real-time** — Server-Sent Events (SSE) push live updates to all screens
- **Notifications** — WhatsApp Cloud API with automatic retry queue
- **AI Service** — Wait time prediction, bottleneck analysis, patient chatbot
- **Audit Trail** — SHA-256 hash-chained, tamper-evident action log

Staff Roles

Administrator

Full access. Manages staff, configuration, hospitals, audit trail, sync/backup.

Receptionist

Patient registration, search, ticket generation, kiosk management.

Nurse (Triage)

Triage assessment, vital signs, priority assignment, reassessment alerts.

Doctor (Clinician)

Call next patient, consultations, transfers, test orders, prescriptions, completion.

Pharmacist

Lab Technician

Pharmacy queue, prescription dispensing, stock flags, ready/collected states.

Lab queue, test order lifecycle, turnaround tracking, result-ready notifications.

Radiologist

Radiology queue, imaging orders, result routing back to clinician.

All Staff

Dashboard access, queue visibility, notification panel, profile management.

2. Logging In & Security

Logging In

1. **Open the staff login page**

   Navigate to `/admin/login` in your browser. Enter your username or email and password.

2. **You are redirected to your role's landing page**

   Receptionists go to registration, triage nurses to triage, clinicians to dashboard, admins to admin dashboard.

Password Requirements

- Minimum 10 characters
- At least 3 of: uppercase letters, lowercase letters, numbers, symbols
- Cannot be a commonly used password
- Passwords are hashed with bcrypt (12 rounds) — never stored in plain text

Security notes:

- Your session automatically logs out after 8 hours of inactivity
- Never share your login credentials with another staff member
- All actions are recorded in the audit trail with your identity
- Only administrators can create new staff accounts

Forgot Password?

Click "Forgot password?" on the login page. Enter your email address. A reset link will be generated. See your administrator if you need help resetting your password.

3. For Reception Staff

Registering a New Patient

1 **Open the registration page**

Click "Register" in the sidebar navigation. The patient registration form appears.

2 **Search for returning patients first**

Use the search bar to look up existing patients by name, phone, national ID, patient number, or scan their QR code. If found, reuse their profile — avoid creating duplicates.

3 **Capture patient details for new patients**

Fill in all required fields:

- Full name, national ID, date of birth, gender
- Phone number (for WhatsApp notifications)
- Address
- Patient category: Walk-in, Referred, Emergency, Elderly, Pregnant, Child, Disabled, or Chronic
- Next of kin name and phone number
- Chief complaint (what's wrong)
- Target department: OPD, Casualty, Pharmacy, Lab, Radiology, or OI Clinic
- Notification consent checkbox

4 **Submit and print ticket**

Click "Register Patient". A ticket is generated (e.g., **OPD-205**) with a QR code. Print the 80mm ticket or share to the patient's phone. The patient appears in the queue immediately.

> **Form auto-save:** If your browser crashes or you accidentally close the tab, the form data is automatically saved. When you return, your entries will be restored.

Managing Patient Search

- Search by any identifier: name, phone, national ID, patient number, or QR code
- Click on a patient to see their visit history and current tickets
- Always search before creating a new record to avoid duplicates

4. For Triage Nurses

Performing a Triage Assessment

1 **Open the triage console**

Click "Triage" in the sidebar. You see all waiting patients sorted by arrival time.

2 **Select a patient**

Click on a patient to open the triage assessment form.

3 **Capture clinical data**

Record:

- **Chief complaint** — what the patient reports
- **Vital signs:** Blood pressure, pulse, temperature, respiratory rate, oxygen saturation
- **Pain level:** 0–10 scale
- **Symptoms and severity**
- **Emergency indicators** — system auto-detects critical thresholds

4 **Assign priority**

Choose one:

- **RED** — Resuscitation (immediate, life-threatening). Patient jumps to front of queue.
- **YELLOW** — Urgent (should be seen within 60 minutes)
- **GREEN** — Standard (routine queue order)
- **BLACK** — Deceased/expectant

The queue automatically re-sorts by priority. RED patients go to the front immediately.

Responding to Reassessment Alerts

Critical safety feature: If a RED or YELLOW patient has been waiting longer than the safe threshold, a reassessment alert appears on the staff dashboard. Click the alert to re-evaluate the patient's condition. You

> can upgrade their priority if their condition has worsened.

5. For Clinicians

Calling the Next Patient

1 **Open the staff dashboard**

Navigate to `/admin/dashboard`. You see the live queue for your department with priority colors and wait times.

2 **Click "Call Next"**

The system calls the highest-priority, longest-waiting patient. The patient's ticket appears on the public board as "Now Serving".

3 **Patient is notified**

A WhatsApp "called" notification is sent. Audio announcement plays (if enabled). The patient proceeds to your room.

After Consultation — Your Options

Action	What happens
Mark Complete	Patient visit is finished. Patient can now give feedback via portal.
Transfer to Department	Patient moves to Lab, Pharmacy, Radiology, or OI Clinic queue. Gets new ticket number and WhatsApp notification.
Order Tests	Create lab or radiology orders with test type and priority. Orders appear in the Lab/Radiology module.
Prescribe	Route prescription to pharmacy. Patient appears in Pharmacy queue.

Managing Missed Patients

1

Mark as missed

If a patient doesn't respond when called, click "Mark as Missed". The patient moves to the "Missed" lane on the board. A missed-turn WhatsApp notification is sent.

2

Recall when patient returns

When the patient returns, click "Recall" to reinsert them into the queue fairly — they don't go to the back of the line.

6. For Pharmacists

Pharmacy Workflow

1. **Open the Pharmacy module**

   Navigate to `/admin/pharmacy`. See all patients in the pharmacy queue with prescription status.

2. **Check prescription details**

   Click on a patient to see their prescription items, dosages, and quantities.

3. **Check stock availability**

   If an item is out of stock, flag it. The patient is automatically notified. The out-of-stock event is recorded for reports.

4. **Dispense and mark ready**

   Once medication is dispensed, mark the prescription as "Ready". The patient receives a WhatsApp notification: "Your prescription is ready at the Pharmacy window."

5. **Mark as collected**

   When the patient picks up their medication, mark as "Collected" to complete the pharmacy visit.

7. For Lab Technicians

Laboratory Workflow

1 Open the Laboratory module

Navigate to `/admin/laboratory`. See all pending test orders with patient and test details.

2 Update test lifecycle

Track each test through its lifecycle:

- **Pending** — Order received from clinician
- **Sample Collected** — Specimen obtained from patient
- **In Progress** — Test is running
- **Completed / Result Ready** — Results available

3 Notify when results are ready

When you mark a test as "Result Ready", the system automatically:

- Sends a WhatsApp notification to the patient
- Routes the patient back to the doctor's queue for follow-up

> **Turnaround time tracking:** The system records the time at each lifecycle stage. Lab turnaround reports show average time from order to result, helping identify slow tests.

8. For OI Clinic Staff

OI Clinic Workflow

The OI (Opportunistic Infection) Clinic handles sensitive services including HIV/TB care. Patient privacy is protected through confidential handling.

Service Streams

Stream	Description
ART	Antiretroviral Therapy management and refills
TB	Tuberculosis screening and treatment
Viral Load	Viral load monitoring and testing
Counselling	Patient counselling sessions
Cervical Screening	Cervical cancer screening

> **Confidentiality:** OI Clinic patients have masked routing on public display boards. Their personal details are only visible to authorized OI Clinic staff.

Managing OI Visits

- View patients by service stream
- Create new OI visits linked to patient tickets
- Update visit status and outcomes
- View OI summary dashboard for the day

9. For Administrators

Creating Staff Accounts

1. **Go to staff registration**

   Navigate to `/admin/staff-register`. This page is only accessible to administrators.

2. **Fill in staff details**

   Enter: full name, username, email, phone, employee ID, department, role, password (10+ chars with 3 of 4 character types), and optionally a profile photo.

3. **Submit**

   Click "Create staff account". The new staff member can now log in with their credentials. The creation is recorded in the audit trail.

Viewing the Audit Trail

- Navigate to `/admin/audit`
- See every action taken in the system: who did it, when, what changed
- The audit log is SHA-256 hash-chained — any tampering is mathematically detectable
- Filter by action type, staff member, or date range

System Configuration

- Navigate to `/admin/settings`
- Configure hospital information (name, location, contact)
- Manage departments and their settings
- Configure notification preferences
- Reset configuration to defaults if needed

Multi-Hospital Management

- Navigate to the hospitals management section
- View, create, and update hospital profiles

- Each hospital has its own configuration and departments

Backup and Sync

- **Export** — Download all system data as a JSON backup
- **Import** — Restore from a previous backup file
- **Backup** — Create a server-side backup
- **Status** — Check sync status and data integrity

10. Reports & Analytics

WaitLess generates 10 types of operational reports, all exportable to CSV, Excel (XLSX), and PDF.

Report	What it shows
Average Wait Time	Average waiting time per department with historical trends
Department Performance	Patients served, service time, queue length per department
No-Show Report	Missed turns correlated with notification delivery status
Triage Report	Triage assessments with timing, outcomes, and override tracking
Lab Turnaround	Time from test order to result ready, per test type
Staff Workload	Patients handled per staff member, service time, productivity
Out-of-Stock	Pharmacy stock flags with frequency and affected patients
Peak Hours	Patient arrival patterns by hour and day for resource planning
Congestion Report	AI-powered queue congestion prediction per department
Resource Allocation	AI-powered suggestions for staff deployment

Generating a Report

1. **Go to Reports**

   Navigate to `/admin/reports`. Select the report type from the sidebar.

2. **Select date range**

   Choose the start and end dates for the report period.

3. **View and export**

   Results appear in tables and charts. Click "Export CSV", "Export Excel", or "Export PDF" to download.

AI Analytics

- **Wait Time Prediction** — Predicts how long new patients will wait based on historical data
- **Bottleneck Analysis** — Identifies which departments are congested and recommends actions
- **Resource Allocation** — Suggests where to deploy staff for maximum impact

11. Notifications & Display Boards

WhatsApp Notifications

The system sends WhatsApp messages automatically at key points in the patient journey. You can monitor delivery status on the dashboard.

Trigger	Message sent
Patient registered	Confirmation with ticket code and tracking URL
Near turn (2-3 ahead)	"Your turn is approaching. Please return to [department]."
Patient called	"[Ticket], please proceed to [room]."
Missed turn	"You missed your turn. Please see reception."
Transferred	"You have been moved to [department]. New ticket: [code]."
Prescription ready	"Your prescription is ready at the Pharmacy window."
Lab results ready	"Your lab results are ready. Please return to the doctor."

Notification Retry

> **Automatic retry:** If a WhatsApp message fails to send (e.g., internet outage), it enters a retry queue. The system retries at 15 seconds, 60 seconds, and 5 minutes. You can also manually retry from the dashboard notification panel.

Display Boards

Screen	URL	Where to display
Public Queue Board	`/queue`	Large TV in main waiting area
Department Board	`/department-board`	TV at each department's waiting area

> **Setup tip:** Set display board browsers to kiosk mode (full screen, no address bar) for a clean public display. The boards update in real time via Server-Sent Events — no page refresh needed.

12. Appointments

Viewing Appointments

- Navigate to `/admin/appointments`
- View all appointments across departments
- Filter by date, department, or status
- See appointment summary statistics

Managing Appointments

- View appointment details and patient information
- Update appointment status (confirmed, cancelled, completed, no-show)
- View available slots per date and service category
- Track appointment reminders sent to patients

Follow-Up Appointments

- Create follow-up appointments linked to completed visits
- View all follow-ups for a specific patient ticket
- Track reminder delivery for follow-up appointments

13. Troubleshooting

"Unable to connect to the server"

The backend server is not running or is unreachable. Check:

- Is the backend server process running? (Should be on port 4242)
- Is the hospital LAN connected?
- Is the correct API URL configured in the frontend?

WhatsApp messages not being sent

- Check WhatsApp status at `/admin/notifications`
- Verify WHATSAPP_PROVIDER is set to "cloud" (not "mock") in backend .env
- Ensure WHATSAPP_ACCESS_TOKEN and WHATSAPP_PHONE_NUMBER_ID are set
- Check that the patient has WhatsApp consent and a valid phone number
- Messages in the retry queue will send automatically when connectivity returns

Staff account creation fails

- Password must be 10+ characters with at least 3 of: uppercase, lowercase, number, symbol
- Username must be 3-32 characters, only letters, numbers, dots, underscores, hyphens
- Check for duplicate username, email, or employee ID
- Only administrators can create staff accounts

Display board not updating

- Check that the backend server is running (SSE events stream from /api/events)
- Refresh the display board page
- Check browser console for EventSource connection errors

Patient not appearing in queue

- Verify the patient was registered to the correct department
- Check the queue board filter settings
- Ensure the patient's status is "waiting" (not "completed" or "missed")

Form data lost after browser crash

WaitLess auto-saves form drafts. When you return to the registration or triage page, your previous entries should be restored. If not, the auto-save may have been interrupted — re-enter the data.

AI chatbot not responding

- The chatbot requires the backend server to be running
- Check that the /api/ai/chatbot endpoint is accessible
- The chatbot uses keyword matching — try rephrasing your question
- Topics supported: wait times, ticket tracking, prescriptions, lab tests, appointments, emergency guidance

---

WaitLess — Digital Hospital Queue Management System
Built for Chinhoyi Provincial Hospital • Version 1.0 • August 2026
Replacing confusion with a queue patients can trust.